

TypoG

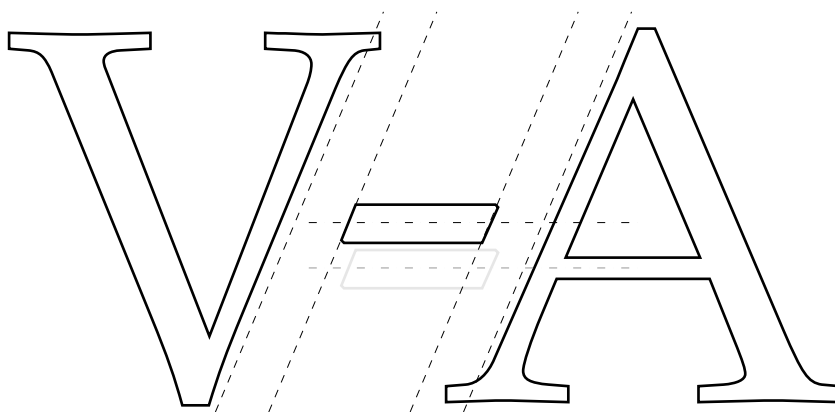
A L^AT_EX Package for Micro-Typographic Enhancements

Ch. L. Spiel*

v0.6 2026/03/21

Abstract

The `typog` package provides macros and environments to improve micro-typographic quality. It enables fine-tuning of hyphenation, spacing, font adjustments, and alignment. Features include control over T_EX's paragraph justification, customizable spacing, font characteristic adjustments, and specialized environments for precise paragraph layout and alignment.



This package is copyright © 2024–2026 Ch. L. Spiel. It may be distributed and/or modified under the conditions of the [L^AT_EX Project Public License](#) (LPPL), either version 1.3c of this license or – at your option – any later version. This work has the LPPL maintenance status “author-maintained”.

* cspiel@users.sourceforge.org

Contents

Hoffentlich wird es nicht so schlimm, wie es schon ist!
— KARL VALENTIN

Quick Reference [v](#)

1

Introduction [1](#)

- 1.1 Overview [1](#)
- 1.2 Prerequisites [1](#)

2

Package Options [2](#)

3

Macros and Environments [6](#)

- 3.1 Setup and Reconfiguration [6](#)
- 3.2 Information [7](#)
 - 3.2.1 Font Information [7](#)
 - 3.2.2 Paragraph- and Page-Breaking Trace [8](#)
- 3.3 Hyphenation [11](#)
- 3.4 Disable/Break Ligatures [16](#)
- 3.5 Manual Italic Correction [17](#)
- 3.6 Apply Extra Kerning or Spacing [18](#)
 - 3.6.1 Slash [18](#)
 - 3.6.2 Hyphen [19](#)
 - 3.6.3 En-Dash and Em-Dash [20](#)
- 3.7 Raise Selected Characters [24](#)
 - 3.7.1 Capital Hyphen [24](#)
 - 3.7.2 Capital En-Dash and Capital Em-Dash [25](#)
 - 3.7.3 Number Dash (Figure Dash) [26](#)
 - 3.7.4 Colon [27](#)
 - 3.7.5 Multiplication Sign [27](#)
 - 3.7.6 Guillemets [27](#)
 - 3.7.7 Inverted Marks [29](#)
- 3.8 Vertically Adjust Label Items [31](#)
- 3.9 Align Last Line [34](#)

Table of Contents continued on next page.

3.10	Fill Last Line	37
3.10.1	Problem Description	37
3.10.2	Manual Changes	38
3.10.3	Multi-Purpose Environments	40
3.10.4	Specialized Environments	40
3.10.5	Consistent Spacing of Last Line	42
3.11	Spacing	42
3.11.1	Looser/Tighter	42
3.11.2	Wide Space	44
3.11.3	Narrow Space	45
3.12	Microtype Front-End	46
3.12.1	Tracking	46
3.12.2	Font Expansion	47
3.12.3	Character Protrusion	48
3.13	Sloppy Paragraphs	48
3.14	Vertically Partially-Tied Paragraphs	50
3.15	Breakable Displayed Equations	54
3.16	Setspace Front End	55
3.17	Smooth Ragged	58

4 Limitations and Known Problems 62

5 Other Packages for Fine L^AT_EX Typography 63

A typog-grep 64

B e-_T_EX: Breaking Paragraphs into Lines 71

Table of Contents continued on next page.



Package Code 73

C.1	Setup and Reconfiguration	78	C.10	Fill Last Line	103
C.2	Information	79	C.11	Spacing	104
C.3	Hyphenation	81	C.12	Microtype Front-End	106
C.4	Disable/Break Ligatures	81	C.13	Sloppy Paragraphs	110
C.5	Manual Italic Correction	82	C.14	Vert. Tie Paragraphs	112
C.6	Apply Extra Kerning	83	C.15	Breakable Disp. Eqs.	116
C.7	Raise Selected Characters	88	C.16	Setspace Front End	116
C.8	Vert. Adjust Label Items	95	C.17	Smooth Ragged	120
C.9	Align Last Line	102			

Change History 127

References 129

Index 132

List of Tables

1	Hyphens and automatic hyphenation	13
2	Comparison of \rightspacedendash-comma combinations	22
3	Suggested raise amounts for \figuredash	26
4	Suggested raise amounts for guillemets	28
5	Label item adjustment suggestions	35
6	Spacing changes made by loosespacing	43
7	Spacing changes made by tightspacing	43
8	\fontdimen<number> parameters	44
9	Comparison of some space sizes	45
10	Shrink values of setfontshrink	47
11	Stretch values of setfontstretch	47
12	Shrink and stretch values of setfontexpand	47
13	Parameter adjustments of \slightlyloppy	49
14	Partial paragraph line counts	53
15	Env. breakabledisplay and \interdisplaylinepenalty	54

Quick Reference

In the multi-column parts of the Quick Reference, we reduce the line spacing from 125% to 120% with `\setbaseline-skippercentage`, and the inter-word spacing by 1.25% with `tightspacing`. List `\items` get tied with `\vtietop`.

This is an alphabetically sorted list of all user macros and environments defined by package `typog` along with the page numbers of their descriptions. A list of all [package options](#) can be found on pages 2 to 5. The [Index](#) on pages 132 to 137 may provide some more detailed insights.

If a line break occurs between the macro and its arguments or between any of the arguments, we indicate the break with a triangle at the end of the initial line and at the beginning of the following line.

A

<code>\Adjustedlabelitemi</code>	Typeset an uppercase-adjusted <code>\label-itemi</code> .	32
<code>\adjustedlabelitemi</code>	Typeset a lowercase-adjusted <code>\label-itemi</code> .	32
<code>\Adjustedlabelitemii</code>	Typeset an uppercase-adjusted <code>\label-itemii</code> .	32
<code>\adjustedlabelitemii</code>	Typeset a lowercase-adjusted <code>\label-itemii</code> .	32
<code>\Adjustedlabelitemiii</code>	Typeset an uppercase-adjusted <code>\label-itemiii</code> .	32
<code>\adjustedlabelitemiii</code>	Typeset a lowercase-adjusted <code>\label-itemiii</code> .	32
<code>\Adjustedlabelitemiv</code>	Typeset an uppercase-adjusted <code>\label-itemiv</code> .	32
<code>\adjustedlabelitemiv</code>	Typeset a height-adjusted <code>\label-itemiv</code> .	32
<code>\allowhyphenation</code>	(Re-)enable automatic hyphenation.	11

B

<code>breakabledisplay</code> [<i><level></i>]	Adjust the penalty associated with <code>\allowdisplaybreaks</code> .	54
--	---	----

<code>\breakpoint*</code>	Insert an empty discretionary.	14
---------------------------	--------------------------------	----

<code>\breakpoint</code>	Insert an empty discretionary and re-enable automatic hyphenation.	14
--------------------------	--	----

C

<code>\capitalcolon</code>	Typeset a vertically adjusted colon.	27
----------------------------	--------------------------------------	----

<code>\capitaldash*</code>	Alias for <code>\capitalendash*</code> .	25
----------------------------	--	----

<code>\capitaldash</code>	Alias for <code>\capitalendash</code> .	25
---------------------------	---	----

<code>\capitalemdash*</code>	Typeset a vertically adjusted <code>\textemdash</code> .	25
------------------------------	--	----

<code>\capitalemdash</code>	Typeset a vertically adjusted <code>\textemdash</code> that is breakable.	25
-----------------------------	---	----

<code>\capitalendash*</code>	Typeset a vertically adjusted <code>\textendash</code> .	25
------------------------------	--	----

<code>\capitalendash</code>	Typeset a vertically adjusted <code>\textendash</code> that is breakable.	25
-----------------------------	---	----

<code>\capitalhyphen*</code>	Typeset a vertically adjusted hyphen character.	24
------------------------------	---	----

<code>\capitalhyphen</code>	Typeset a vertically adjusted hyphen character that is breakable.	24
-----------------------------	---	----

<code>\capitalinverted- exclamationmark{<number>}</code>	Typeset an inverted ('upside-down') exclamation mark that is level with the baseline. 29
<code>\capitalinverted- questionmark{<number>}</code>	Typeset an inverted ('upside-down') question mark that is level with the baseline. 29
<code>\capitaltimes</code>	Typeset a vertically adjusted <code>\texttimes</code> . 27
<code>covernextindentpar[<dim>]</code>	Extend the last line of a paragraph. 40
D	
<code>\Doubleguillemetleft</code>	Typeset left double guillemets vertically adjusted for uppercase. 28
<code>\Doubleguillemetright</code>	Typeset right double guillemets vertically adjusted for uppercase. 28
<code>\doubleguillemetleft</code>	Typeset left double guillemets vertically adjusted for lowercase. 27
<code>\doubleguillemetright</code>	Typeset right double guillemets vertically adjusted for lowercase. 27
F	
<code>\figuredash*</code>	Typeset a <code>\textendash</code> vertically adjusted for figures (numerals). 26
<code>\figuredash</code>	Typeset a breakable <code>\textendash</code> vertically adjusted for figures (numerals). 26
<code>\fontsizeinfo{<csname>}</code>	Store the current em-height and <code>\baselineskip</code> in a pair of macros. 8
H	
<code>hyphenmin[<left-min>]{<right-min>}</code>	Set the values of <code>\lefthyphenmin</code> and <code>\righthyphenmin</code> . 15
I	
<code>\itcorr*{<strength>}</code>	Apply italic correction in the form of a <code>\kern</code> scaled by <code>\textitaliccorrection</code> . 17
<code>\itcorr{<strength>}</code>	Apply italic correction in the form of a <code>\kern</code> scaled by <code>\fontdim1</code> or <code>\textitaliccorrection</code> . 17
K	
<code>\kernedhyphen*[<raise>]▷ ◁{<left-kern>}{<right-kern>}</code>	Typeset an unbreakable hyphen and apply kerning to its left and right. 19
<code>\kernedhyphen[<raise>]▷ ◁{<left-kern>}{<right-kern>}</code>	Typeset a breakable hyphen and apply kerning to its left and right. 19
<code>\kernedslash*</code>	Typeset an unbreakable forward slash and apply kerning to both its left and right sides. 18
<code>\kernedslash</code>	Typeset a breakable forward slash and apply kerning to both its left and right sides. 18
L	
<code>lastlinecenteredpar</code>	Center the last lines of a paragraph. 36
<code>lastlinefitpar</code>	Match the spacing of last line and next-to-last line. 42
<code>lastlineflushrightpar</code>	Alias for <code>lastlinerraggedleftpar</code> . 34
<code>lastlinerraggedleftpar</code>	Align the last line of paragraph flush right. 34
<code>\leftkernedhyphen*[<raise>]{<left-kern>}</code>	Typeset a hyphen and apply kerning to its left-hand side. 19
<code>\leftkernedhyphen[<raise>]{<left-kern>}</code>	Typeset a hyphen, apply kerning to its left-hand side, and insert a breakpoint after it. 19

- `\leftspaceddash*` [*<raise>*]
Alias for `\leftspacedendash*`.
- `\leftspaceddash` [*<raise>*]
Alias for `\leftspacedemdash`.
- `\leftspacedemdash*` [*<raise>*]
Typeset an em-dash with some space around it. Prohibit line breaks before and after it.
- `\leftspacedemdash` [*<raise>*]
Typeset an em-dash with some space around it. Allow line breaks at its left-hand side.
- `\leftspacedendash*` [*<raise>*]
Typeset an en-dash with some space around it. Prohibit line breaks before and after it.
- `\leftspacedendash` [*<raise>*]
Typeset an en-dash with some space around it. Allow line breaks at its left-hand side.
- `loosespacing` [*<level>*]
Increase the width of the space character. 43
- `\lowercaseadjustlabelitems` {*<levels>*}
Activate the lowercase height-adjustment values inside `itemize` environments. 31
- ## N
- `\narrowspace*`
Typeset a narrow space whose width depends on `\fontdimen7`. 45
- `\narrowspace`
Typeset a narrow space whose width depends on `\fontdimen7` or `\fontdimen2`. 45
- `\noadjustlabelitems` {*<levels>*}
Deactivate height-adjustment of label items. 31
- `nocharprotrusion`
Deactivate character protrusion. 48
- `nofontexpand`
Alias for `nofontexpansion`. 48
- `nofontexpansion`
Deactivate font expansion. 48
- `\nolig*` [*<kerning>*]
Break a ligature. 16
- `\nolig` [*<kerning>*]
Break a ligature and introduce a hyphenation opportunity. 16
- ## O
- `openlastlinepar` [*<dim>*]
Open a paragraph's last line if it is almost full or completely filled. 40
- ## P
- `prolongpar`
Increase the `\looseness` of a paragraph. 40
- ## R
- `\resetbaselineskip`
Reset `\baselineskip` to its original value. 55
- `\rightkernedhyphen*` [*<raise>*] {*<right-kern>*}
Typeset a hyphen and apply kerning to its right-hand side. 19
- `\rightkernedhyphen` [*<raise>*] {*<right-kern>*}
Typeset a hyphen, apply kerning to its right-hand side, and insert a breakpoint after it. 19
- `\rightspaceddash*` [*<raise>*]
Alias for `\rightspacedendash*`.
- `\rightspaceddash` [*<raise>*]
Alias for `\rightspacedendash`.
- `\rightspacedemdash*` [*<raise>*]
Typeset an em-dash with some space around it. Prohibit line breaks before and after it.
- `\rightspacedemdash` [*<raise>*]
Typeset an em-dash with some space around it. Allow line breaks at its right-hand side.
- `\rightspacedendash*` [*<raise>*]
Typeset an en-dash with some space around it. Prohibit line breaks before and after it.
- `\rightspacedendash` [*<raise>*]
Typeset an en-dash with some space around it. Allow line breaks at its right-hand side.

S

<code>\setbaselineskip- percentage{⟨percentage⟩}</code>	Set <code>\baselineskip</code> as a percentage relative to the point size of the font. 56	<code>\singleguillemetrigh</code>	Typeset right single guillemets vertically adjusted for lowercase. 27
<code>\setbaselineskip{⟨baselineskip⟩}</code>	Set <code>\baselineskip</code> using an absolute length. 55	<code>slightlyloppypar[⟨sloppiness⟩]</code>	Format a paragraph with given sloppiness. 49
<code>setfontexpand[⟨level⟩]</code>	Simultaneously set font stretch and shrink limits. 47	<code>\slightlyloppy[⟨sloppiness⟩]</code>	Format using the given sloppiness. 49
<code>setfontshrink[⟨level⟩]</code>	Set font shrink limits. 47	<code>smootheraggedright[⟨option⟩...]</code>	Format with one of the three smooth-ragged-right generators. 60
<code>setfontstretch[⟨level⟩]</code>	Set font stretch limits. 47	<code>\smootheraggedrightfuzzfactor{⟨factor⟩}</code>	Relative amount of glue in smooth-ragged-right typeset lines. 60
<code>setfonttracking{⟨delta⟩}</code>	Override the default tracking for all fonts. 46	<code>\smootheraggedright- generator{⟨generator⟩}</code>	Name of the basis environment ('generator') used in the environments <code>smootheragged-right</code> and <code>smootheraggedrightpar</code> . 60
<code>\setleadingpercentage{⟨percentage⟩}</code>	Set <code>\baselineskip</code> as a percentage using the leading. 56	<code>\smootheraggedrightleftskip</code>	Value for <code>leftskip</code> that <code>smootheragged-right</code> and <code>smootheraggedrightpar</code> pass to the smooth-ragged-right generator. 60
<code>\setleading{⟨leading⟩}</code>	Set <code>\baselineskip</code> using the leading. 56	<code>smootheraggedrightpar[⟨option⟩...]</code>	Format a paragraph using one of the three smooth-ragged-right generators. 58
<code>shortenpar</code>	Decrease the <code>\looseness</code> of a paragraph. 40	<code>\smootheraggedrightparindent</code>	Value for <code>parindent</code> that <code>smootheragged-right</code> and <code>smootheraggedrightpar</code> pass to the smooth-ragged-right generator. 60
<code>\Singleguillemetleft</code>	Typeset left single guillemets vertically adjusted for uppercase. 28	<code>\smootheraggedrightparwidth</code>	Value for the width of the smooth-ragged-right margin in environments <code>smooth-raggedright</code> and <code>smootheraggedrightpar</code> . 61
<code>\Singleguillemetrigh</code>	Typeset right single guillemets vertically adjusted for uppercase. 28		
<code>\singleguillemetleft</code>	Typeset left single guillemets vertically adjusted for lowercase. 27		
<code>smootheraggedrightshapequintuplet[⟨option⟩...]{⟨width1⟩}...{⟨width5⟩}</code>	Prescribe five line lengths for formatting paragraphs. 58		
<code>smootheraggedrightshapeseptuplet[⟨option⟩...]{⟨width1⟩}...{⟨width7⟩}</code>	Prescribe seven line lengths for formatting paragraphs. 58		
<code>smootheraggedrightshapetriplet[⟨option⟩...]{⟨width1⟩}{⟨width2⟩}{⟨width3⟩}</code>	Prescribe three line lengths for formatting paragraphs. 58		

<code>\spacedcapitalemdash*</code>	Typeset a height-adjusted em-dash with some space around it. Prohibit line breaks before and after the dash. 25	<code>\typoggetnth{<dest>}{<key>}{<index>}</code>	Retrieve single item of a typog compound configuration value. 7
<code>\spacedcapitalemdash</code>	Typeset a height-adjusted em-dash with some space around it. 25	<code>\typogget{<key>}</code>	Retrieve a typog configuration value. 6
<code>\spacedcapitalendash*</code>	Typeset a height-adjusted en-dash with some space around it. Prohibit line breaks before and after the dash. 25	<code>typoginspectpar[<option>]{<id>}</code>	Turn on tracing of paragraphs and pages for a paragraph. 8
<code>\spacedcapitalendash</code>	Typeset a height-adjusted em-dash with some space around it. 25	<code>typoginspect[<option>]{<id>}</code>	Turn on tracing of paragraphs and pages. 8
<code>\spacedddash*{<raise>}</code>	Alias for <code>\rightspacedendash*</code> . 21	<code>\typoglowercaseadjust-check[<factor>]{<sample>}</code>	Typeset all four label items adjusted for lowercase with an indicator line. 34
<code>\spacedddash[<raise>]</code>	Alias for <code>\rightspacedendash</code> . 21	<code>typogsetup{<keys>}</code>	Configure package typog. 6
<code>\spacedemdash*{<raise>}</code>	Alias for <code>\rightspacedemdash*</code> . 23	<code>\typoguppercaseadjust-check[<factor>]{<sample>}</code>	Typeset all four label items adjusted for uppercase with an indicator line. 34
<code>\spacedemdash[<raise>]</code>	Alias for <code>\rightspacedemdash</code> . 23	U	
<code>\spacedendash*{<raise>}</code>	Alias for <code>\rightspacedendash*</code> . 21	<code>\uppercaseadjustlabelitems{<levels>}</code>	Activate the uppercase height-adjustment values inside <code>itemize</code> environments. 31
<code>\spacedendash[<raise>]</code>	Alias for <code>\rightspacedendash</code> . 21	V	
<code>\splicevtietop{<lines>}</code>	Inside of a <code>list</code> -like environment, fuse the first lines. 50	<code>vtiebotdisptoppar[<num-before-lines>]> <[<num-after-lines>]</code>	Fuse a display with its preceding and following lines. 52
T		<code>vtiebotdisp[<num-lines>]</code>	Fuse a display with its preceding lines. 51
<code>tightspacing[<level>]</code>	Decrease the width of the space character. 43	<code>vtiebotpar[<num-lines>]</code>	Fuse the final lines of a paragraph. 51
<code>\typogadjuststairs[<factor>]> <[<step>]{<count>}{<sample>}</code>	Generate ‘stairs’ of vertically shifted label items. 33	<code>\vtiebot[<num-lines>]</code>	Fuse final lines. 51
<code>\typogfontsize</code>	The default font’s quad size. 57	<code>vtietoppar[<num-lines>]</code>	Fuse the first lines of a paragraph. 50
		<code>vtietop[<num-lines>]</code>	Fuse first lines. 50
		<code>\vtietop[<num-lines>]</code>	Fuse first lines. 50

W**`\widespace*`**

Typeset a wide space whose width depends on `\fontdimen7`. **44**

`\widespace`

Typeset a wide space whose width depends on `\fontdimen7` or `\fontdimen2`. **44**

1 Introduction

“Good typography” is the minimum acceptable solution;
 “fine typography” is what we aspire to.
 — ILENE STRIZVER

L^AT_EX is the beginning of good typesetting—not the end. This package provides some tools for even better looking documents. When applied correctly, its effects appear subtle and inconspicuous. However, it’s a long ball game.

1.1 Overview

The primary focus of typog are micro-typographic improvements, where it tries to hit it out of the ballpark in terms of typographic polish.

Section 3.1 presents how to reconfigure package typog after it has been loaded and how to access its configuration values for introspection or inclusion in the user’s own code. Section 3.2 addresses the need for more information in the typesetting process whether during the draft phase or in the final printed manuscript. Section 3.3 expands the hyphenation facilities of L^AT_EX. Sections 3.4 and 3.5 treat the breaking of ligatures and the manual application of italic correction—in a generalized way. Section 3.6 introduces macros to kern or space some punctuation signs. Section 3.7 deals with vertically positioning glyphs in a more visually pleasing manner. Section 3.8 also deals with vertical alignment and explains how to height-adjust the labels in `itemize` lists to perfection, whether the items are followed by uppercase or by lowercase letters. Sections 3.9 and 3.10 discuss much-needed macros for better control of the last line of a paragraph. Section 3.11 covers the manipulation of paragraph spacing. Section 3.12 details the microtype front end: font tracking (3.12.1), font expansion (3.12.2), and character protrusion (3.12.3). In Sec. 3.13, we address some shortcomings of spacing control with a replacement for the macro `\sloppy` and the related environment `sloppy`. Section 3.14 presents several specialized functions to avoid club or widow lines in a paragraph. As a simple extension of displayed mathematical equations, we define a breakable variant in Sec. 3.15. Section 3.16 introduces the setspace front end. In the last part, Sec. 3.17, we introduce a novel way of generating ragged paragraphs, which is still experimental.

1.2 Prerequisites

Package typog requires e-T_EX; it relies on the L^AT_EX3 interface. Parts of it are based on packages microtype [26] and setspace [30]. However, if the functionality is not used, typog can be used without microtype. The same holds true for the setspace front end.

The package was tested with pdfTeX 3.141592653-2.6-1.40.24 from the TeX Live distribution of 2022, as shipped by Debian.

Throughout the whole document we indicate actual uses of the package’s macros and environments in the margin. All these notes are examples themselves, as they are typeset using `slightlyloppy`, `loose-spacing`, and `smoothragged-rightpar`. ¶ The title page has already demonstrated the effect of `lastlinecentered-par` in justified paragraphs for the abstract and the copyright notice.

2 Package Options

The typog package is loaded as usual via `\usepackage`. It can be configured during loading with package `<OPTIONs>` or later on with `typogsetup`, as described in Sec. 3.1. Since typog provides front ends to packages microtype and setspace, we mention them in the synopsis; they are *not* automatically loaded by typog.

```
\usepackage[...]{microtype} % Only required for macros and
                             % environments in Sec. 3.12.

\usepackage[...]{setspace} % Only required for macros in Sec. 3.16.

\usepackage[<OPTION>...]{typog}
```

The package `<OPTIONs>` also serve as configuration `<key>`s unless noted otherwise. This means, they can be set with `typogsetup` and their values can be retrieved with `\typogget`. Options that rely on package microtype are indicated with “microtype required”.

`breakpenalty=<penalty>`

Penalty for a line break at various points. Default value: 50, initialized by the current `\exhyphenpenalty`: 50.

`debug, nodebug`

Write some package-specific debug information to the *log* file. Opposite: `nodebug`. The default is not to record debug information.

These two options cannot be used with `typogsetup` or `\typogget`.

`emdashspace=<glue>`

SINCE V0.5

Set the horizontal skip that is inserted before and after the em-dash in macros `\spacedemdash` and `\spacedcapitaldash` to `<glue>`. Default value: $\frac{56}{1000}$ em.

`endashspace=<glue>`

SINCE V0.5

Set the horizontal skip that is inserted before and after the en-dash in macros `\spacedendash` and `\spacedcapitalendash` to `<glue>`. Default value: $\frac{200}{1000}$ em plus $\frac{100}{1000}$ em minus $\frac{67}{1000}$ em.

`ligaturekern=<dim>`

Set `<dim>` of the kern that is inserted to split a ligature in macro `\nolig`. See Sec. 3.4. Default value: $\frac{33}{1000}$ em.

`lowercaselabelitemadjustments={<dim-1>, ..., <dim-4>}`

SINCE V0.4

Vertical shifts `<dim-N>` to apply to `\labelitem<N>`, where `<N>`=1, 2, 3, or 4 is the nesting level of the `itemize` list. Empty list elements are ignored. The special value `*` instructs typog to preserve `<dim-N>` at that position. The adjustments apply to the lowercase setting (`\lowercaseadjustlabelitems`). See Sec. 3.8 (in particular subsection ‘Setup’ and Tab. 5 on p. 35) and also configuration option `uppercaseadjustlabelitem`.

All four lengths default to 0 pt.

This subsection is type-set with all typog parameters reset to their defaults by wrapping it in a `typogsetup` environment with an empty argument.

Important

Configuring `lowercaselabelitemadjustments` (or `uppercase-labelitemadjustments`) does *not* activate the correction mechanism. Use one of the macros `\lowercaseadjustlabelitems` or `\uppercaseadjustlabelitems` for that purpose. ■

SINCE V0.5

`lowerslash=<dim>`

Lower the slash typeset by `\kernedslash`. Positive lengths `<dim>` lower the glyph, negative ones raise it. This is the opposite ‘direction’ of `\raisebox`. See Sec. 3.6.1. Default value: 0 pt.

RENAMED IN V0.5

`mathitaliccorrection=<dim>`

Italic correction in math mode. See Sec. 3.5 and also the complementary configuration option `textitaliccorrection`. Default value: 0.4mu.¹

SINCE V0.6

`pdfsubstitutes, nopdfsubstitutes`

Setup substitutes for selected macros that make them work correctly for example in the PDF-outline. The default is not to setup any substitution.

See Section 4 for a complete list of macros that are substituted if option `pdfsubstitutes` is given.

These two options cannot be used with `typogsetup` or `\typogget`.

`raise*=<dim>`

Set the length by which selected characters (dash, hyphen, times, and number dash) are raised. Default value: 0 pt.

Only the raise amounts for guillemets and inverted marks are unaffected by this option.

This option neither can be used with `typogsetup` nor with `\typogget`; however, the specific options influenced by it can.

`raisecapitaldash=<dim>`

Set the length that the `\textendash` is raised in `\capitaldash`. See Sec. 3.7.2. Default value: 0 pt.

`raisecapitalguillemets=<dim>`

Set the length that single and double guillemets are raised in the uppercase versions of the guillemet macros. See Sec. 3.7.6. Default value: 0 pt.

`raisecapitalhyphen=<dim>`

Set the length that the hyphen character `⁂` is raised in `\capitalhyphen`. See Sec. 3.7.1. Default value: 0 pt.

`raisecapitaltimes=<dim>`

Set the length that the multiplication symbol `⌘` is raised in `\capital-times`. See Sec. 3.7.5. Default value: 0 pt.

We access all the (default) configuration values with `\typogget`.

¹ Note that 1 mu is 1/18 em of the mathematical font’s em.

`raisefiguredash=<dim>`

Set the length that the `\textendash` is raised in `\figuredash` and `\capitalcolon`. See Sec. 3.7.3. Default value: 0 pt.

`raiseguillemets=<dim>`

Set the length that single and double guillemets are raised in the lowercase versions of the guillemet macros. See Sec. 3.7.6. Default value: 0 pt.

`raiseinvertedmarks={<dim-1>, <dim-2>, <dim-3>}`

SINCE V0.5

Set the lengths by which the macros `\capitalinvertedexclamationmark` and `\capitalinvertedquestionmark` raise their associated inverted exclamation marks and inverted question marks. Each dimension corresponds to the optional indices of the macros. See Sec. 3.7.7.

A `<dim-N>` of 0 pt means to “auto level” the mark; if a (quasi-)zero manual correction is desired, use, for example, 1 sp. Empty list elements are ignored. The special value `*^` instructs typog to preserve `<dim-N>` at that position. Default values: 0 pt, 0 pt, 0 pt.

`shrinklimits={<limit-1>, <limit-2>, <limit-3>}` microtype required

`stretchlimits={<limit-1>, <limit-2>, <limit-3>}` microtype required

Set the three limits, given in $\frac{1}{1000}$ em, of shrinkability and stretchability for the levels. They are used in `setfontshrink` (shrinklimits only), `setfontstretch` (stretchlimits only), and `setfontexpand` (both sets). See Sec. 3.12.2.

New `<limit-#>` values replace old ones. If one or more limits of the triple should remain unchanged pass a `*^` instead of a number.

Defaults for `shrinklimits` are 5, 10, 20 and those for `stretchlimits` are 5, 10, 20.

Both options can be used when loading the package and in the document preamble, but *not* in the document body.

`slashkern=<dim>`

Set the size of the kerns before and after `\kernedslash`. See Sec. 3.6.1. Default value: $\frac{50}{1000}$ em.

`textitaliccorrection=<dim>`

RENAMED IN V0.5

Italic correction fallback value; used if `\fontdimen1` is zero. See Sec. 3.5 on manual italic correction and also the complementary configuration option `mathitaliccorrection`. Default value: $\frac{20}{1000}$ em.

`trackingttspacing={<outer-spacing>}` microtype required

Set the outer spacing of all typewriter fonts when used in the `settracking` environment as described in Sec. 3.12.1.

The argument `<outer-spacing>` gets passed to microtype’s `\SetTracking` option `outer spacing` [26, Sec. 5.3]. If the argument contains commas, enclose the whole argument in curly braces. Default argument value: 300, 90, 60.

The option can be used when loading the package and in the document preamble, but *not* in the document body.

By default, this option is unset.

SINCE V0.4

`uppercaselabelitemadjustments={⟨dim-1⟩, ..., ⟨dim-4⟩}`

Vertical shifts $\langle dim-N \rangle$ to apply to `\labelitem⟨N⟩`, where $\langle N \rangle=1, 2, 3$, or 4 is the nesting level of the `itemize` list. Empty list elements are ignored. The special value $\times$ instructs typog to preserve $\langle dim-N \rangle$ at that position. The adjustments apply to the uppercase setting (`\uppercaseadjustlabelitems`). See Sec. 3.8 (in particular subsection ‘Setup’ and Tab. 5 on p. 35) and also configuration option `lowercaseadjustlabelitem`.

All four lengths default to 0 pt.

Important

Configuring `uppercaselabelitemadjustments` (or `lowercase-labelitemadjustments`) does *not* activate the correction mechanism. Use one of the macros `\uppercaseadjustlabelitems` or `\lowercaseadjustlabelitems` for that purpose. ■

3 Macros and Environments

Easy things should be easy, and
hard things should be possible.
— LARRY WALL

This is the “User Manual” section of the documentation, where we describe all user-relevant macros and environments that are defined in package `typog`.

We follow the naming convention that every environment whose name ends with `...par` issues a `\par` at its end. Environments with different name suffixes never close with `\par`.

3.1 Setup and Reconfiguration

`typogsetup (env.)`

Configure the package with the given $\langle keys \rangle$. An empty argument of `typogsetup` resets all $\langle keys \rangle$ to their default values.

```
\begin{typogsetup}{\langle keys \rangle} ... \end{typogsetup}
```

The package can be (re-)configured at any point with `\typogsetup{\langle keys \rangle}`, or – for localized changes – as

```
\begin{typogsetup}{\langle keys \rangle}
...
\end{typogsetup}
```

where $\langle keys \rangle$ have the same format as the package options described in Sec. 2.

Tip

Use `\PassOptionsToPackage{\langle keys \rangle}{typog}` to pass $\langle keys \rangle$ to `typog` before loading it and `\typogsetup{\langle keys \rangle}` after `\usepackage{typog}`. ■

Use Cases

`\typogsetup` can substitute configuring the package at load time or serve as an addition. ¶ Using the `typogsetup` environment allows to fine-tune the parameters for a specific use, e.g., display-sized text. ¶ It even is conceivable that a well-established `typog`-configuration could be attached to font-changing macros like `\rm`, `\sf`, or `\tt`. ■

`\typogget`

Sometimes the user needs to access configuration values of package `typog`. This can be done in a safe way without resorting to code that is bracketed by `\makeatletter` and `\makeatother` using of the following macro.

```
\typogget{\langle key \rangle}
```

Retrieve the configuration value that is associated with $\langle key \rangle$. For a list of available $\langle key \rangle$ s see Sec. 2.

Use Case

Raise glyphs by the same amount as configured with `typog`.

```
\newcommand*\seesubst
{\raisebox{\typogget{raisecapitalguillemets}}%
{\rightarrowhead}}
\renewcommand*\labelitemi
{\raisebox{\typogget{raisecapitaldash}}{\cdot}}
```

The latter only is useful inside of an `itemize` environment of course. Compare with the solution in Sec. 3.8 offered by `typog` since v0.4. ■

`\typoggetnth`
SINCE v0.4

If a configuration item is associated with a list of values as `lowercase-labelitemadjustments`, `shrinklimits`, `stretchlimits`, `trackingtt-spacing`, and `uppercase-labelitemadjustments` are, it may be convenient to fetch a particular list element of it.

```
\typoggetnth{<cname>}{<key>}{<index>}
\typoggetnth{<dimen-register>}{<key>}{<index>}
```

Retrieve the configuration value—which is a comma-separated list—that is associated with `<key>` and store the item having position `<index>` in `<dimen-register>` or the parameter-less, global macro `<cname>`. The destination `<dimen-register>` may be predefined like, e. g., `\dimen0` or user-defined. Dimensions can also be stored in a macro by using the `<cname>` form of `\typoggetnth` but not vice versa.

Index into the list either from left to right with positive indices starting at 1 up to the length of the list, or from right to left with negative indices starting at -1 down to the negative length.

Important

Macro `\typoggetnth` *only* works with `<key>`s that are associated with a list of values. ■

3.2 Information

Never forget: The visual output counts;
it must always be checked, [...].
— UDO WERMUTH [33]

The em-dash at then end of
the quote is height-adjusted with
`\capitalemdash*`.

We define some functions for inspecting the typesetting process.

3.2.1 Font Information

`\fontsizeinfo`

Capture the font size² and line spacing³ at the point where `\fontsizeinfo` is called in macro `<cname>`. Both dimensions are measured in points (pt) and the results are rounded to tenths.

- 2 We use `\fontdimen6`, the em-height as the font size.
- 3 The line spacing simply is `\baselineskip`.

```
\fontsizeinfo{<cname>}
```

The call to `\fontsizeinfo` defines a pair of macros to access the stored values. The unstarred version `\cname` expands to the lengths including their units (i. e., pt), the starred version `\cname*` omits the units. The separating slash is `\kernedsl`[ash](#), which is introduced in Sec. [3.6.1](#).

Note

The `\baselineskip` can contain a rubber (stretch/shrink) component; however, `\fontsizeinfo` will not display these parts. ■

Use Cases

Colophon. ■ Font test pages. ■

3.2.2 Paragraph- and Page-Breaking Trace

`typoginspect (env.)`
`typoginspectpar (env.)`

The environments `typoginspect` and `typoginspectpar` turn on the tracing of paragraphs and pages; optionally they display the parbox' contents. These environments help users identify typographic issues quantitatively without getting distracted by unrelated information in the trace or the *log*-file.

```
\begin{typoginspect}[<option>]{<id>} ... \end{typoginspect}
\begin{typoginspectpar}[<option>]{<id>}
...
\end{typoginspectpar}
```

The `<id>` is an arbitrary string that identifies the results in the *log* file. If the mandatory argument is empty, `typog` constructs a unique value.

Option

tracingboxes[`=<size>`]

Specify the maximum box breadth and box depth reported in the log. If `<size>` is omitted the maximum values are assumed; this is similar to the `\tracingboxes` macro [\[1, p. 312\]](#).

Caution

The end-of-trace marker sometimes gets placed too early and the trace seems truncated. $\text{\LaTeX}$ reliably logs the requested trace information, but the write operations for trace data and `\immediate\write` which is used to print the end-tag are not synchronized. The workaround in such a situation is to enclose more text in the `typoginspect` environment (respecting the nesting of other environments of course). ■

L^AT_EX log-file and trace. The trace data in the *log*-file is bracketed by XML-tags.

```
<typog-inspect_id="⟨id⟩"_job="⟨jobname⟩"_line="⟨line-number⟩"_page="⟨page-number⟩">
...
</typog-inspect>
```

where the $\langle id \rangle$ is the user-supplied, unique⁴ identifier of the group, $\langle jobname \rangle$ is the value of `\jobname`, $\langle line-number \rangle$ records the `\inputlineno` of the `\begin` of the group, and $\langle page-number \rangle$ gets replaced with the current value of the page counter.

- Any text tool can be used to extract the tags. EMACS users will find the function (`occur` $\langle regex \rangle$) to be useful.
- As long as the tags are not nested `sed` or `perl` extract the information gathered by `typoginspect`, for example:

```
sed -ne '/<typog-inspect_id="..."/, \#</typog-inspect>#p'
< jobname.log
```

or

```
perl -ne '$a=0 if /<\/typog-inspect>; \
print $_ if $a; \
$a=1 if /<typog-inspect_id="..."/' \
< jobname.log
```

- The companion program **typog-grep** is tailored to extract the information marked up by `typoginspect` and `typoginspectpar`, even if the environments are nested.

The complete manual page of **typog-grep** is reproduced in [Appendix A](#).

Tips

- It may be necessary to run whatever L^AT_EX engine with a larger log-file line length, to prevent wrapped lines. With short lines the wannabe XML opening tags can get wrapped and thus become unrecognizable to dumb post-processors. To avoid wrapped lines, prepend

```
/usr/bin/env max_print_line=2147483647
```

to the command-line. The value $2147483647 = 2^{31} - 1$ effectively disables all line wrapping by L^AT_EX.

As both **pdf_latex** and **lua_latex** support changing their configuration on a by-call basis with option `-cnf-line=⟨STRING⟩` an alternative to the above example is to add

```
-cnf-line=max_print_line=2147483647
```

to the command-line.

- If more trace information is needed just add `\tracing...` calls right after `\begin{typoginspect}` or `\begin{typoginspectpar}`.

⁴ It has turned out advantageous to use unique $\langle id \rangle$ s. However, $\langle id \rangle$ s are *not required* to be distinct.

- As the overhead of `\typoginspect` is relatively low, hairy parts of a document can permanently be furnished with them, for example, the Index.
- Any labeled part can treat their ids to `<id>`. Think of `\captions` or any theorem-like environment and their associated, unique `\labels`. ■

Investigating the badness of a paragraph. It is generally unnecessary to determine the *exact* classification of a paragraph’s badness [17, p. 97n], though the curious user can switch on logging of T_EX’s line-break information with `\tracingparagraphs=1`⁵ or simply use the `typoginspect` environment and check the suffixes

`@@<breakpoint-number> line <line-number>.<suffix>`

of each line in the paragraph, where for `<suffix>` the following mapping holds [17, p. 99]:

`0` $\mapsto$ very loose, `1` $\mapsto$ loose, `2` $\mapsto$ decent, and `3` $\mapsto$ tight.

Example

`@@17: line 15.1- t=142289 s=93.58414 a=2.86073 -> @@16`

1. The feasible breakpoint `@@` number 17 in the paragraph leads to
2. `line` 15, which is the loose `.1` last `-` line of the paragraph.
3. Up to this breakpoint the paragraph has picked up total demerits `t` of 142289.
4. The following two values only show up if `\lastlinefit` $\neq$ 0:
 - (a) The shortfall `s` and
 - (b) glue `a` or `g`.⁶
5. The best⁷ way to get here, i. e., `@@17` is via `->` breakpoint `@@` 16. ■

Note

When package `microtype`’s font expansion jumps in the reports on “Loose `\hbox` (badness ...)” and “Tight `\hbox` (badness ...)” contain the amount of shrinking or expansion as parenthesized values (units are thousandths of the current font’s em) like, e. g.,

`\T1/erewhon-LF/m/n/9/@/@ (-13) ...`

or

`\T1/erewhon-LF/m/n/9/@/@/10ls (+7) ...`

An `ls` appended to the font name specification indicates that `microtype`’s letter spacing is active and changed the tracking by that many thousands on an em as indicated before `ls`. ■

⁵ Reference 32 provides an exceptionally detailed discussion of the output of `\tracingparagraphs`.

⁶ The author is unaware of any descriptions of `s`, `a`, or `g` and the interested reader is referred to the source code, e. g., *pdf_{tex}.web*; search for `print("_s=")`. In the weaved documentation the first relevant section is §1851.

⁷ ‘Best’ means the minimum-demerits path in the graph of the feasible breakpoints, which has been constructed for the paragraph.

Investigating page-breaks. Use `\tracingpages=1` or the `typoginspect` environment to switch on tracing of T_EX's page-break information [17, p. 112n].⁸

The first time vertical material enters a new page, T_EX logs

```
%% goal height=<text-height>, max depth=<max-depth>
```

where $\langle text-height \rangle$ is the total height T_EX wants to achieve and $\langle max-depth \rangle$ is the maximum depth of the hbox in the last line of the page is allowed to have without considering $\langle text-height \rangle$ to be exceeded. For example:

```
%% goal height=598.0, max depth=5.0
```

For every vertical breakpoint T_EX records

```
% t=<total-height> g=<goal-height> b=<badness> p=<penalty>
    c=<cost>
```

Here, $\langle total-height \rangle$ and $\langle goal-height \rangle$ are the current total height of the page and the current goal height to achieve with respect to this vertical breakpoint.

The value of $\langle penalty \rangle$ and $\langle cost \rangle$ can be infinite, which would be indicated with an asterisk $\star$ instead of a numerical value. The best vertical breakpoint found so far on the current page is indicated by a trailing sharp-sign $\#$.

Example

```
% t=351.3 plus 11.0 minus 1.0 g=553.9 b=10000 p=-300 c=100000#
```

1. At this vertical breakpoint the total page height t is 351.3 pt. We have picked up glue with 11 pt stretchability and 1 pt shrinkability along the way.
2. The current goal height g is 553.9 pt. If the initial goal height was 598 pt we can deduce that some space for other vertical material was subtracted.
3. The badness b of this vertical break is horrendous which is expected for the first lines on a page since breaks so early are rightfully considered infinitely bad.
4. The penalty p at this point actually is a bonus.
5. As the badness is 10000 the cost for a break is calculated to 100000. ■

3.3 Hyphenation

T_EX's and thus L^AT_EX's hyphenation algorithm is highly sophisticated, yet the document author sometimes lacks convenient macros to solve seemingly trivial typographic tasks. For example, to hyphenate a compound word connected by a hyphen.

`\allowhyphenation`

T_EX inhibits breaks of the component words by default. The following macro rectifies the problem.

```
\allowhyphenation
```

Macro `\allowhyphenation` re-enables automatic hyphenation after T_EX has turned it off, e. g. in a hyphenated compound.

The admittedly simple rules for when T_EX auto-hyphenates and when it does not give rise to so many different, yet interesting cases that we devote Tab. 1 to

⁸ See also the discussion of the T_EX output routines by SOLOMON [28].

them. The seemingly special cases shown there are not that uncommon, e. g., consider ‘spin-½’ which is coded as `\mbox{spin-\textfrac{1}{2}}`. A line break between the text and the fraction would garble the term.

Use Cases

- All examples from the bottom of Tab. 1 on p. 13.
- Establish additional hyphenation opportunities if these have been canceled by bricolage that is necessary for parenthesized German compounds like, e. g., “(Linear-)Impulsvektor”, “(Links-)Konjugation”, or “(Unter-)Gruppe”. All of these can be fixed by blunting the hyphen with `\mbox` and inserting a breakpoint with `\hskip` (or, as horizontal glue carries a penalty of zero, with `\breakpoint`, which is associated with `\breakpenalty`), like

`(Linear\mbox{-})\hskip0pt Impulsvektor`

where “Linear” won’t get hyphenated. Macro `\allowhyphenation` rectifies this situation:

`(Linear\allowhyphenation\mbox{-})\hskip0pt Impulsvektor`

Typographically the last code example raises the same concerns as does `\hyp`, this is, whether the hyphenation opportunities (here: *Li-ne-ar*), if chosen, distract from the construction of the compound that already contains a hyphen.

- Mend line breaks of index-entries in a narrow index:
`Halbgruppe, Transformations\allowhyphenation\mbox{-}\,---`
 The first part, ‘Transformations’ is allowed to be hyphenated, but a break after the hyphen is prohibited as it results in a prowling em-dash at the beginning of the next line.
- Re-enable hyphenation when a macro decays into a `\hbox`:
`Einselement\allowhyphenation\rlap{,}\footnote{...}`
 where `\rlap` is equivalent to something like `\makebox[0pt]{#1\hss}`.
- Use `\allowhyphenation` to turn on hyphenation of the first word of a paragraph as, e. g., in a narrow index or a `\marginpar`:
`\marginpar{\allowhyphenation Kontakttransformationen}`
 A common trick to sweet-talk T_EX into hyphenating the first word of a paragraph is to put `\hskip0pt` in front of it.
- Enable automatic hyphenation in the rare but weird cases when T_EX does not hyphenate a word that is hyphenatable despite the result is an overfull box.⁹ ■

Whenever using `\-`, the short-hand form of `\discretionary{-}{ }{ }`, authors writing in a foreign language should reconsider whether it really beats `\hyphenation` or `\babelhyphenation`¹⁰ in the particular situation. However, sometimes `\-` actually *is* the way to go.

Let us assume we mark up proper names with

`\DeclareRobustCommand*{\propername}[1]
{\mbox{\textsc{#1}}}`

⁹ The author of typog has no idea, why this happens, but he has been successful in fixing the problems with code like `(\allowhyphenation Superpositionsprinzip)` or `(\allowhyphenation Superauswahlregel)`. Elucidation by a T_EX-savant is highly welcome.

¹⁰ `\babelhyphenation` is the multi-lingual extension of T_EX’s `\hyphenation` and it is defined in package `babel` [6].

A `\narrowsspace` compensates the visual gap between the colon and the italic ‘L’.

TABLE 1: $\text{\TeX}$ offers plenty of possibilities to hyphenate a compound. ¶
 We use the sample ‘hyphenated-compound’ to show various code examples and the results that they produce. The parts are automatically hyphenated like this: ‘hyphenated’ $\rightarrow$ ‘hy-phen-ated’ and ‘compound’ $\rightarrow$ ‘com-pound’.

$\text{\LaTeX}$ -Code	Result	Note
hyphenated-compound	hyphenated-compound	Most frequently used code; the <code>hyphen</code> expands to <code>\discretionary{-}{-}{-}</code> rendering the parts unbreakable
hyphenated\mbox{-}% compound	hyphenated-compound	Suppress hyphenation with the <code>\mbox</code> in the compound
\mbox{hyphenated-% compound}	hyphenated-compound	Avoid line break and thus hyphenation
hyphenated\hyp compound	hy- phen- ated- com- pound	Macro <code>\hyp</code> defined in package <code>hyphenat</code> [39]
hyphenated% \allowhyphenation-% compound	hy- phen- ated- compound	Macro <code>\allowhyphenation</code> of package <code>typog</code> ; only unblock hyphenation of the first part
hyphenated-% \allowhyphenation compound	hyphenated- com- pound	Macro <code>\allowhyphenation</code> of package <code>typog</code> ; only unblock hyphenation of the second part
hyphenated% \allowhyphenation \mbox{-}% compound	hy- phen- ated-compound	Macro <code>\allowhyphenation</code> of package <code>typog</code> ; hyphenate first part and keep the original <code>hyphen</code> unbreakable
hyphenated% \allowhyphenation-% \allowhyphenation compound	hy- phen- ated- com- pound	Macro <code>\allowhyphenation</code> of package <code>typog</code> ; hyphenate both parts, similar to <code>\hyp</code> shown above

and we want to have breakable “ABELsche Gruppe” or “EUKLIDischer Vektorraum” without dropping the markup. To that end we define commands that insert a hyphenation point at the right place:

```
\newcommand*\abelsche{
    {\propername{Abel}\-sche}
\newcommand*\euklidischer{
    {\propername{Euklid}i\-scher}
```

which are impossible to encode with `\hyphenation` or `\babelhyphenation` as these expect only letters and dashes as their arguments with spaces separating the words.

$\TeX$ never hyphenates the initial word in a paragraph even with `\allowhyphenation`. Start the paragraph with `\hskip 0pt` to enable hyphenation even for the first word.

Tip — Typewriter Fonts

Sometimes it is desired to get a breakable typewriter font. $\LaTeX$ suppresses any hyphenation for fonts in `\ttfamily` by un-defining their `\hyphenchars`. If these are reassigned, the usual line breaking and hyphenation occurs again.

So, a fictitious macro ‘`\code`’ to typeset short pieces of code could look like this:

```
\newcommand*\code{[1]
    {\ttfamily
    \hyphenchar\font=‘\-\relax #1}} ■
```

`\breakpoint`
`\breakpoint*`

The empty discretionary construct [17, p. 95], `\discretionary{}{}{}{}`, is helpful. It deserves its own macro—with a descriptive name.

```
\breakpoint    \breakpoint*
```

The starred form inserts an empty discretionary, which disables automatic hyphenation. The unstarred form inserts an empty discretionary and immediately re-enables automatic hyphenation.

The difference between `\breakpoint` and the $\LaTeX$ macro `\allowbreak` is not only that the former has a starred form, but the penalty associated with `\breakpoint` is the current¹¹ `\exhyphenpenalty`, whereas `\allowbreak` statically assigns a zero penalty.

Use Case

Prefixes that end in a hyphen inside of a pair of parenthesis:

```
\mbox{(pre-)}\breakpoint* \propername{Hilbert} space ■
```

`hyphenmin (env.)`
SINCE V0.3

Set the values of `\lefthyphenmin` and `\righthyphenmin` confined to an environment.

¹¹ At this point in the document `\exhyphenpenalty=50` holds.


```
\begin{hyphenmin} [⟨left-hyphen-minimum⟩] {⟨hyphen-minimum⟩}  
...  
\end{hyphenmin}
```

Without optional argument `hyphenmin` sets both `\lefthyphenmin` and `\righthyphenmin` to `⟨hyphen-minimum⟩`. When called with an optional argument it sets `\lefthyphenmin` to `⟨left-hyphen-minimum⟩` and `\righthyphenmin` to `⟨hyphen-minimum⟩`.¹²

Use Case

If the hyphen minimums were *increased* e. g. in the preamble: Reduce the hyphen minimum in the index or other multi-column environments with narrow lines to regain hyphenation possibilities. ¶ Use a large `⟨hyphen-minimum⟩` to disable hyphenation. ■

¹² The current values for `\lefthyphenmin` and `\righthyphenmin` in this document are 2 and 3.

3.4 Disable/Break Ligatures

`\nolig*` Break a ligature without introducing a hyphenation opportunity.

```
\nolig*[\langle kerning \rangle]
```

Inserting `\nolig*` disables a ligature at the given point by a kern. Set the size of the kern with `ligaturekern` or override this value with `\langle kerning \rangle` as thousandths of the current font's em.

Use Cases

`\nolig*` can be useful in headings, where additional hyphenation points are unwelcome. ¶ In fonts with an overly rich set of ligatures `\nolig*` offers a straightforward means to suppress unwanted ligatures at non-hyphenatable positions. ¶ Rectify the appearance of a pseudo ligature, i. e., two adjacent characters that look like a ligature, but actually are not. ■

`\nolig` Break a ligature and introduce a hyphenation opportunity.

```
\nolig[\langle kerning \rangle]
```

Inserting `\nolig` disables a ligature at the given point as `\nolig*` does and introduces a hyphenation opportunity with penalty `breakpenalty`.

Important — hyperref bookmarks

If a `\nolig` – whether starred or un-starred – occurs in an argument that is processed with package `hyperref` for inclusion into the document's PDF-bookmarks an additional argument is necessary to parse the macro. This argument either is `\relax` or the empty group `{}`.

```
\nolig*[\langle kerning \rangle]\relax    \nolig[\langle kerning \rangle]\relax
\nolig*[\langle kerning \rangle]{ }    \nolig[\langle kerning \rangle]{ }
```

The prototypical places where this processing-for-PDF-bookmarks happens are the sectioning macros, e.g., `\chapter`, `\section`, `\subsection`, etc.

L^AT_EX will trip with “Undefined control sequence” on `\typog@missing@-` argument if the extra argument is not passed.

Alternatively use `\texorpdfstring` [25, Sec. 4.1.2, p. 22]. ■

Use Cases

`\nolig` can be used with just about any ligature that needs to be split into its parts. It also has proven beneficial in separating pairs of characters that are kerned too tightly, e.g., the ‘ij’, as in ‘bijection’, which is particularly distracting here, for it occurs at the boundary of two syllables. ■

3.5 Manual Italic Correction

`\itcorr` The italic correction offered by T_EX or L^AT_EX sometimes needs manual adjustment.

`\itcorr*`

```
\itcorr{<strength>}      \itcorr*{<strength>}
```

In text mode, macro `\itcorr` inserts a kern whose width is proportional to `\fontdim1`, which is the font’s italic correction. If `\fontdim1` happens to be zero (e.g. for an upright font), `\itcorr` uses the value set with `textitalic-correction` instead of `\fontdim1`. The starred version always uses `textitalic-correction`. In math mode macro `\itcorr` uses the value set with `mathitalic-correction`¹³ in both the starred and the unstarred form.

Typical slant angles of serif italic fonts range from 8° to 18° and thus values for `textitalic-correction` from .14 to .32. Note: `<strength>` can be negative, and fractional `<strength>`s are allowed.

Use Cases

Stronger or weaker correction than `\.` ¶ Correct a non-slanted or non-italicized font. ¶ Negative correction at the left-hand side¹⁴ of italic, i. e., compensate “shift-to-the-right effect” of italic. ¶ Positive correction at the left-hand side of italic, e. g., an opening parenthesis or square bracket followed by an italic *f* (before: 8, after: 7) or *y* (before: 4, after: 1) reaching far to the left below the baseline. ■

The `<strength>` parameter explained. T_EX records the slant angle α of a font in `\fontdim1` as $1\text{pt} \times \sin \alpha$. Rephrased the formula means: *How much horizontal space is required for a letter slanted with α that is 1pt high?* Thus, `\itcorr{<strength>}` calculates

$$\langle strength \rangle \times 1\text{pt} \times \sin \alpha.$$

A well-chosen `<strength>` should be the absolute minimum value which avoids that the glyphs typeset in italic collide with other – usually non-italic – letters or symbols unless this disturbs the consistency of the overall tracking.

Correction of the right-hand side and $\alpha > 0$: A reasonable first guess of `<strength>` is the highest point where the rightmost part of the letter would touch a rule angled at α with respect to the baseline. The correction of the left-hand side and $\alpha > 0$ considers the lowest ‘touching’ point below the baseline on the left-hand side of the letter. Negative values of α exchange the reference points.

Figure 1 shows how `<strength>` and α are related. Moreover, it demonstrates how intricate italic correction is.

¹³ Separate adjustments may be desirable if the math font’s italic have markedly different slants.

¹⁴ Groff has the machinery for left-italic-correction. Its font-metrics files support per glyph left-italic-correction values and users can access them conveniently via `\,`.

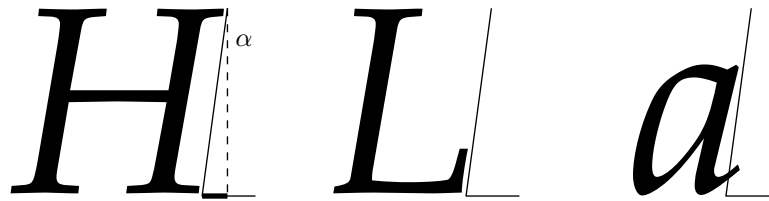


FIGURE 1: Some letters of an italic font. We use the capital `H` to measure the angle α between the plumb-line (drawn dashed) and a tangent to the rightmost parts of the glyph. The length of the plumb-line is proportional to `<strength>` and the short, thick part of the baseline symbolizes the resulting italic correction. ¶ The middle example, the capital `L`, shares α with `H` but obviously needs a far smaller `<strength>` or even no correction at all. ¶ The `a` at the right-hand side is an example of why `TEX` allows to assign an italic correction to each individual character of a font. Not only features the lowercase `a` a larger α – despite being a member of the same font – but its serif adds as much to the width as the slanted stem.

3.6 Apply Extra Kerning or Spacing

Package `typog` provides two sets of macros to kern some of the punctuation symbols. One is for forward slashes the other, more extensive one, for hyphens and dashes.

3.6.1 Slash

`\kernedslash` Macro `\kernedslash` expands to a forward slash (`/`) with some extra space around it.

`\kernedslash*`

```
\kernedslash \kernedslash*
```

The starred form is unbreakable, the non-starred version introduces a break point with penalty `breakpenalty` after the slash. Configure the kerning around the slash with `slashkern`.

The kerned slash can typeset lowered (or raised), with the offset from the baseline configured using `lowerslash`.

Tip

Define specialized macros for slashes surrounded by lowercase letters or small caps if they are needed often or require a specific value for `slashkern`.

```
\newcommand*{\lowercasekernedslash}
{\begin{typogsetup}{lowerslash=.1em,
slashkern=.02em}

\kernedslash
\end{typogsetup}}
```

Generic solutions could build upon `\fontdimen5` and `\fontdimen6` (see Tab. 8 on p. 44) or measure the height and depth of the slash-glyph (abstracted in `\typog@forwardslash`) and compare it to e.g. the height of selected lowercase characters. ■

We center the last lines of each figure and table caption using `lastlinecenteredpar`.

OPTION `lowerslash`
INTRODUCED IN V0.5

If the word following the slash should not be hyphenated append `\nobreak` after `\kernedslash*`.

Use Cases

`\kernedslash` improves the appearance of pairs of years typeset in lining numerals: `\year_1/\year_2`. ¶ The macro has proven helpful in many cases where the right hand side of the slash starts with a capital as, for example, `\city/\state-code` (US-specific) or `\anything/\noun` (any language that capitalizes `\noun`). ¶ Use option `lowerslash` to adjust the slash to surrounding lowercase letters. ¶ Correct a too high raising slash of a font (e.g. Cochineal) with option `lowerslash`. ■

3.6.2 Hyphen

`\kernedhyphen` Macros `\kernedhyphen*` and `\kernedhyphen` expand to a hyphen (⁀) with
`\kernedhyphen*` given kerning to its left and to its right.

```
\kernedhyphen[⟨raise⟩]{⟨left-kerning⟩}{⟨right-kerning⟩}
\kernedhyphen*[⟨raise⟩]{⟨left-kerning⟩}{⟨right-kerning⟩}
```

Typeset an unbreakable hyphen with `\kernedhyphen*` or a breakable hyphen (like `\hyp` of package `hyphenat` [39]) with `\kernedhyphen` and apply some kerning to left and to the right of it. The values `⟨left-kerning⟩` and `⟨right-kerning⟩` are multiplied with one thousandth of the current font's em to get the size of the kern.

The optional argument `⟨raise⟩`, also given in $\frac{1}{1000}$ em, allows to adjust the height of the hyphen similar to the macros described in Sec. 3.7. In text mode, the special argument `_*` for `⟨raise⟩` applies the current value of `raisecapital-hyphen`. The default for `⟨raise⟩` is zero.

We also define specialized versions for kerning on the left-hand side or the right-hand side only. These macros work like their two-argument counterparts and set the opposite kerning to zero.

```
\leftkernedhyphen[⟨raise⟩]{⟨left-kerning⟩}
\leftkernedhyphen*[⟨raise⟩]{⟨left-kerning⟩}
\rightkernedhyphen[⟨raise⟩]{⟨right-kerning⟩}
\rightkernedhyphen*[⟨raise⟩]{⟨right-kerning⟩}
```

Use Cases

Composites in the form `⟨math⟩-⟨noun⟩` in languages where nouns are capitalized. ¶ Composites where one or both sides of the hyphen are typeset in different fonts, like, `⟨small-caps⟩-⟨roman⟩`. ■

`\leftkernedhyphen`
`\leftkernedhyphen*`
`\rightkernedhyphen`
`\rightkernedhyphen*`

3.6.3 En-Dash and Em-Dash

The macros introduced in this subsection allow for fine-tuning of the whitespace before and after en-dashes and em-dashes. Each macro set is designed for a specific use. The spaced en-dash macros are intended to be used where an en-dash is surrounded by normal spaces:

$$\langle \text{WORD} \rangle_-_-\langle \text{WORD} \rangle.$$

Typographic improvement may come from reducing the width of the space around the en-dash with relative to the normal space.¹⁵ On the other hand, the em-dash macros assume that there is no space between the $\langle \text{WORD} \rangle$ s and the em-dash:

$$\langle \text{WORD} \rangle-\langle \text{WORD} \rangle.$$

The spaced em-dash macros add some (hairline) space between the em-dash and the $\langle \text{WORD} \rangle$ s, which helps prevent the em-dash from visually colliding with adjacent words.¹⁶

Line Breaking Around Dashes. Package `typog` offers spaced en- and em-dashes with breakpoints either on the left-hand side or the right-hand side (or none for any of the starred versions) and this is reflected in the macro names by `left` and `right`. All spaced dashes apply equal amounts of space on both sides of the dash; ‘left’ and ‘right’ refer to the breakpoints.

A single dash is used – among many other cases – to indicate a pause (e. g. announcing an action to be continued), instead of a comma, or even instead of a period. The typographic rules favor either no line break at all or a break at the right-hand side [4, 6.82] of the dash, which also is $\text{\TeX}$ ’s default.

Dashes are often used for appositions or explanatory phrases, as in the first sentence above. In this instance there exist two camps: one emphasizes the insertion and thus couples the lead-in and lead-out dashes to it (see item 1 below) [13, p. 173]. The other avoids a dash at the beginning of a line at the cost of severing it from the insertion (item 2). We demonstrate the alternatives with en-dashes.

1. One rule set prefers insertions to be preceded and followed by dashes. The elementary way to solve this is to say

```
before\_--~insertion~--\_after
```

For this solution `typog` offers `\leftspacedendash` and `\rightspacedendash`:

```
before\leftspacedendash
insertion\rightspacedendash after
```

15 The current font’s space width is $\frac{233}{1000}$ em plus $\frac{116}{1000}$ em minus $\frac{78}{1000}$ em compared to typogs `en-dashspace` width of $\frac{200}{1000}$ em plus $\frac{100}{1000}$ em minus $\frac{67}{1000}$ em.

16 Some fonts offer dashes with no side bearings to facilitate drawing horizontal lines by abutting these dashes.

The insertion, “among . . .”, is separated by spaced en-dashes.

2. Another school insists to avoid dashes at the start of lines. Now the straightforward solution looks like this:

before~--_insertion~--_after

For this scenario both dashes behave like `\rightspacedendash` and the alias `\spacedendash` comes in handy:

before\spacedendash
insertion\spacedendash after

The four macros `\leftspacedendash`, `\leftspacedendash*`, `\rightspacedendash`, and `\rightspacedendash*` all expand to an en-dash `–` that is surrounded by whitespace. They differ in the ways they handle line breaking before or after the en-dash.

```
\leftspacedendash
\leftspaceddash
\leftspacedendash*
\leftspaceddash*
\rightspacedendash
\rightspaceddash
\spacedendash
\spaceddash
\rightspacedendash*
\rightspaceddash*
\spacedendash*
\spaceddash*
ALL SINCE V0.5
```

<code>\leftspacedendash[⟨raise⟩]</code>	<code>\leftspaceddash</code> (alias)
<code>\leftspacedendash*[⟨raise⟩]</code>	<code>\leftspaceddash*</code> (alias)
<code>\rightspacedendash[⟨raise⟩]</code>	<code>\rightspaceddash</code> (alias)
	<code>\spacedendash</code> (alias)
	<code>\spaceddash</code> (alias)
<code>\rightspacedendash*[⟨raise⟩]</code>	<code>\spacedendash*</code> (alias)
	<code>\spaceddash*</code> (alias)

Typeset an unbreakable en-dash with any of the starred variants. The unstarred `\left...`-macros have breakpoints at their left-hand sides and the `\right...`-macros breakpoints at their right-hand sides.

Configure the size of the space before and after the dash with option `en-dashspace`.

The optional argument `⟨raise⟩` allows adjusting the height of the en-dash above the baseline just as the macros in Sec. 3.7; it is meant for one-off solutions. If an en-dash is needed that is both raised and spaced prefer macro `\spacedcapitalendash`.

Important — hyperref bookmarks

If any of `\leftspacedendash`, `\rightspacedendash`, `\spacedendash`, `\spaceddash`, or `\spacedemdash` (see below)—whether starred or unstarred—occur in an argument that is processed with package `hyperref` for inclusion into the document's PDF-bookmarks an additional argument is necessary to parse the macro. This argument either is `\relax` or the empty group `{}`.

We showcase the modifications of the plain calls for macro `\spaceddash`. They apply to all macros that generate spaced en- and em-dashes.

<code>\spaceddash*[⟨raise⟩]\relax</code>	<code>\spaceddash[⟨raise⟩]\relax</code>
<code>\spaceddash*[⟨raise⟩]{}</code>	<code>\spaceddash[⟨raise⟩]{}</code>

The prototypical places where this processing for PDF bookmarks happens are the sectioning macros, e.g., `\chapter`, `\section`, `\subsection`, etc.

L^AT_EX will trip with “Undefined control sequence” on `\typog@missing@-` argument if the extra argument is not passed.

Alternatively use `\texorpdfstring` [25, Sec. 4.1.2, p. 22]. ■

`\swapedashskip`
SINCE V0.5

If a `\rightspacedendash` is immediately followed by another punctuation mark as, e. g., a comma, the space at the right-hand side of the en-dash is ill-placed. It either should be removed by following the macro with an `\unskip` or `\swapedashskip`. Furthermore, `\rightspacedendash` should be made unbreakable.

`\swapedashskip{<arg>}`

Use `\swapedashskip{<arg>}`, where `<arg>` typically is a punctuation mark, to remove the right-hand space of any spaced en-dash macro and put exactly that amount of space on the right-hand side of `<arg>`. Table 2 compares the effects on the spacing.

TABLE 2: Comparison of uncorrected and corrected `\rightspacedendash`-comma combinations for a space width of $\frac{1}{3}$ em and a (exaggerated) `endashspace` of $\frac{1}{2}$ em. We have added “book struts” to the left and to the right of the results to indicate the width of the output.

Code	Result
<code>\rightspacedendash*, \space</code>	┘ — , ┘
<code>\rightspacedendash*\unskip, \space</code>	┘ —, ┘
<code>\rightspacedendash*\swapedashskip,</code>	┘ —, ┘

The four macros `\leftspacedemdash`, `\leftspacedemdash*`, `\rightspacedemdash`, and `\rightspacedemdash*` all expand to an em-dash “—” that is surrounded by whitespace. They differ in the ways they handle line breaking before or after the em-dash.

Let us pick up the alternatives explained in the section on en-dashes and adapt them to em-dashes. Note that by joining words with em-dashes T_EX turns off automatic hyphenation, which usually is the right thing to do to avoid confusion with dashes and hyphens. The spaced em-dash macros duplicate this behavior.

1. Tie the em-dashes to the insertion.

before\breakpoint\mbox{---}
insertion---after

Macro `\breakpoint` was introduced in Section 3.3.

before\leftspacedemdash
insertion\rightspacedemdash after

2. By default T_EX inserts breakpoints after em-dashes. So that we can just write

before---insertion---after

`\leftspacedemdash`
`\leftspacedemdash*`
`\rightspacedemdash`
`\rightspacedemdash*`
`\spacedemdash`
`\spacedemdash*`
ALL SINCE V0.5

The spaced solution with package `typog` looks like this:

```
before\spacedemdash
insertion\spacedemdash after
```

<code>\leftspacedemdash[⟨raise⟩]</code>	<code>\leftspaceddash (alias)</code>
<code>\leftspacedemdash*[⟨raise⟩]</code>	<code>\leftspaceddash* (alias)</code>
<code>\rightspacedemdash[⟨raise⟩]</code>	<code>\rightspaceddash (alias)</code>
	<code>\spacedemdash (alias)</code>
<code>\rightspacedemdash*[⟨raise⟩]</code>	<code>\spacedemdash* (alias)</code>

Typeset an unbreakable em-dash with any of the starred variants. The unstarred `\left...`-macros have breakpoints at their left-hand sides and the `\right...`-macros breakpoints at their right-hand sides.

The size of the space before and after the dash is configured with option `emdashspace`.

The optional argument `⟨raise⟩` allows adjusting the height of the em-dash above the baseline just as the macros in Sec. 3.7; it is meant for one-off solutions. If an em-dash is needed that is both raised and spaced prefer macro `\spacedcapitalemdash`.

The same *Important note* on hyperref bookmarks holds for `\spacedemdash` as for `\spacedendash`.

Note

Alternatively use package `microtype` to achieve a similar effect. Macro `\SetExtraKerning` [26, Sec. 5.4] allows modifying the side-bearings of any glyph. ■

Tip

The macros rely on `\textemdash` and `\textendash` to typeset em-dashes and en-dashes. If different glyphs are desired redefine them (preferably locally).

Example with the fictitious macro `\fancydash`:

```
\let\origspacedemdash=\spacedemdash
\renewcommand*{\spacedemdash}
{
  \begingroup
    \typogsetup{emdashspace=.2pt}%
    \let\textemdash=\fancydash
    \origspacedemdash
  \endgroup} ■
```

3.7 Raise Selected Characters

Usually all hyphens and dashes of a font are designed to join lowercase letters. This holds also true for most of our `\labelitem(N)` markers, bullets, stars, and even fancy dingbats. If these hyphens and dashes connect uppercase letters (or lining numerals) they sometimes appear to low; they disrespect the glyphs' symmetry axis. A similar situation arises if `itemize` list markers precede an uppercase letter, a lining numeral, or a big mathematical operator.

We introduce a set of macros for the most common cases that typeset these characters at a user definable, adjusted height above the baseline. Readers familiar with OpenType fonts will be reminded of the case feature. Users can base their own definitions of raised characters on their associated dimensions.¹⁷

Caution

The height adjustment disables a font's built-in kerning. ■

General note for all raised hyphen-like macros: Prefer the starred version if applied in front of any punctuation.

3.7.1 Capital Hyphen

`\capitalhyphen`
`\capitalhyphen*`

In many fonts the height of the hyphen character `-` above the baseline is optimized for lowercase letters. In languages that capitalize their nouns as, e.g., German, this may be too low for compounds involving capitals.

`\capitalhyphen` `\capitalhyphen*`

The unstarred version introduces a hyphenation opportunity right after the hyphen character (with penalty `breakpenalty`) whereas the starred version does not. The actual amount the hyphen gets raised in `\capitalhyphen` is determined by `raisecapitalhyphen`.

Use Cases

In languages that capitalize their nouns, the typical use-case is between an *⟨abbreviation⟩* and a *⟨noun⟩* when *⟨abbreviation⟩* is a string of uppercase letters. The same holds true for a connection of an uppercase variable in mathematical mode and a *⟨noun⟩* starting with a capital letter. ¶ Abbreviated compound first names (e.g., A.-M. Legendre) can be joined with the starred version. ¶ Also, the starred form is suited for ISO 8601-formatted dates if they are composed with lining-style numerals. ■

¹⁷ Also compare with Ex. 12 in reference 38 for an attempt to automate vertical alignment.

3.7.2 Capital En-Dash and Capital Em-Dash

`\capitalendash` The situation of the en-dash `_` is almost identical to the one of the hyphen character `_` described in the previous section or the number dash to be introduced in the next section.

`\capitalendash*`

`\capitaldash`

`\capitaldash*`

```
\capitalendash    \capitaldash (alias)
\capitalendash*   \capitaldash* (alias)
```

The unstarred version introduces a hyphenation opportunity right after the dash (with penalty `breakpenalty`) whereas the starred version does not. The actual amount the hyphen gets raised in `\capitaldash` is determined by `raisecapitaldash`.

Use Cases

Letter ranges as used in the title of an index. ¶ Any mixed letter-digit ranges (of capital letters and lining-style numerals) as in e. g., Sec. B-2. ■

`\capitalemdash` For completeness we also introduce a raised em-dash `_`. It behaves just like its en-dash sibling.

`\capitalemdash*`

```
\capitalemdash    \capitalemdash*
```

Use Cases

Item symbols in itemized lists if the item text starts with an uppercase letter. ¶ Theorem headings, like, e. g., Definition 6.2 — LIE Algebra. ■

`\leftspacedcapitalendash` Combine raising an en-dash and adding some extra space around it.

`\leftspacedcapitaldash`

`\leftspacedcapitalendash*`

`\leftspacedcapitaldash*`

`\rightspacedcapitalendash`

`\rightspacedcapitaldash`

`\rightspacedcapitalendash*`

`\rightspacedcapitaldash*`

ALL SINCE V0.5

```
\leftspacedcapitalendash    \leftspacedcapitaldash (alias)
\leftspacedcapitalendash*   \leftspacedcapitaldash* (alias)
\rightspacedcapitalendash   \rightspacedcapitaldash (alias)
                             \spacedcapitalendash (alias)
                             \spacedcapitaldash (alias)
\rightspacedcapitalendash*   \spacedcapitalendash* (alias)
                             \spacedcapitaldash* (alias)
```

These macros unite the features of `\capitalendash` with those of `\spaced-endash` as described in Sec. 3.6.3.

`\leftspacedcapitalemdash` Combine raising an em-dash and adding some extra space around it.

`\leftspacedcapitalemdash*`

`\rightspacedcapitalemdash`

`\rightspacedcapitalemdash*`

`\spacedcapitalemdash`

`\spacedcapitalemdash*`

ALL SINCE V0.5

```
\leftspacedcapitalemdash
\leftspacedcapitalemdash*
\rightspacedcapitalemdash    \spacedcapitalemdash (alias)
\rightspacedcapitalemdash*   \spacedcapitalemdash* (alias)
```

These macros unite the features of `\capitalemdash` with those of `\spaced-emdash` as described in Sec. 3.6.3.

Tip

The macros rely on `\textemdash` and `\textendash` to typeset em-dashes and en-dashes. If different glyphs are desired redefine them (preferably locally).

See the [tip](#) on p. 23 for a code example. ■

3.7.3 Number Dash (Figure Dash)

`\figuredash`
`\figuredash*`

`\figuredash` yields 12–34–56–78 for sans-serif and 12–34–56–78 for the roman typeface.

The en-dash often gets used as separator for numerical ranges. In most fonts it has the correct height above baseline for oldstyle numerals, e. g. 12–34–56–78, but with lining numerals – depending on the font – it may look like it suffers from “broken suspenders”: 12–34–56–78. The situation is similar to `\capitaldash` and `\capitalhyphen` discussed in Secs. 3.7.1 and 3.7.2.

`\figuredash` `\figuredash*`

The unstarred version introduces a hyphenation opportunity right after the en-dash with penalty `breakpenalty` whereas the starred version does not. The actual amount the en-dash gets raised in `\figuredash` is determined by `raisefiguredash`.

Values of .05 em to .1 em are typical for fonts that need this kind of correction and .1 em is a good starting point. Table 3 summarizes some findings.

TABLE 3: Suggested values for raising `\figuredash`, which actually is an en-dash, between lining numerals of some selected fonts in multiples of 1 em.

Font	Raise
Alegreya, Arvo, Bitter, Clara, EB Garamond, Gentium, Ibarra Real Nova, INRIA Serif, Libertine, Libertinus, Merriweather, PT Serif, Roboto Slab, Spectral, STIX, and many more	.0
Charis SIL, fbb*, Source Serif Pro	.05
Libre Baskerville, Crimson Pro, Erewhon, Droid Serif	.0667
GFS Artemisia, Libre Caslon, Coelacanth, Crimson Pro, Crimson Text, T _E X Gyre Pagella, Quattrocento, TX Fonts, ADF Venturis, and many more	.1

* Free Bembo.

Other macros may be redefined with `\figuredash` for a consistent appearance of the copy, like, for example, `\citedash` (package `cite` [3]), or `\crefrangeconjunction` (package `cleveref` [11]).

Use Case

The key customers of `\figuredash` are the PAGES entries of bibliography databases. ¶ In an index generated with `makeindex` the range delimiter `delim_r` is a candidate for `\figuredash*`. ■

3.7.4 Colon

`\capitalcolon`

SINCE V0.6

The `\capitalcolon` macro is a variation of the `\figuredash` theme.

`\capitalcolon`

Expands to a raised colon. More exactly, it expands to a raised `\textcolon` and the latter by default expands to `:`. Macro `\textcolon` can be overridden by the user to substitute different colons. The raise amount is the same as for `\figuredash`, i. e., `raisefiguredash`.

Use Case

Ratios, like, e. g. 1:2 when using lining numerals. ¶ Time of day e. g. 9:00–5:00, again with lining numerals. ■

3.7.5 Multiplication Sign – Times ^x

`\capitaltimes`

The `\capitaltimes` macro is a variation of the `\capitalhyphen` theme.

`\capitaltimes`

In text mode, it expands to an appropriately raised `\texttimes`, and in math mode to a raised `\times` binary operator, where `raisecapitaltimes` determines the amount of upward-shifting applied; it never inserts any break points.

Use Case

Prime use are two- or higher-dimensional shape specifications with lining numerals or uppercase letters in mathematical mode as, for example, matrix or tensor sizes. ■

3.7.6 Guillemets

The sentence uses `\leftspaced-endash` and `\rightspaced-endash`.

Another possible typographic problem this package addresses is that both sets – single and double quotes – of guillemets may suffer from a too small distance to the baseline.

For the implementation typog relies on the T1¹⁸ font encoding not on package babel.

`\singleguillemetleft`

`\singleguillemetright`

`\doubleguillemetleft`

`\doubleguillemetright`

Lowercase Versions.

`\singleguillemetleft` `\singleguillemetright`
`\doubleguillemetleft` `\doubleguillemetright`

For consistency and easy accessibility we define height-adjusted left and right single guillemets as `\singleguillemetleft` and `\singleguillemetright`;

18 Font encoding T1 can be forced via `\usepackage[T1]{fontenc}` in the document preamble.

double guillemets are available with `\doubleguillemetleft` and `\doubleguillemetright`. Their heights above the baseline are collectively adjusted with [raiseguillemets](#).

`\Singleguillemetleft`
`\Singleguillemetright`
`\Doubleguillemetleft`
`\Doubleguillemetright`

Uppercase Versions.

<code>\Singleguillemetleft</code>	<code>\Singleguillemetright</code>
<code>\Doubleguillemetleft</code>	<code>\Doubleguillemetright</code>

The companion set of single, double, left, and right quotes corrected for uppercase letters or lining numerals is `\Singleguillemetleft` and `\Singleguillemetright` and `\Doubleguillemetleft` and `\doubleguillemetright`. Mnemonic: These macros start with an uppercase letter. Their height above the baseline is adjusted with [raisecapitalguillemets](#). Values of .025 em to .075 em are typical for fonts that need this kind of correction. Table 4 summarizes some findings.

TABLE 4: Suggested values for raising guillemets of some selected fonts in multiples of 1 em.

Font	Uppercase	Lowercase
EB Garamond, Libertinus, Merriweather, and many more	.05	.0
Gentium	.05	.025
GFS Artemisia, GFS Didot	.0625	.05
Charis SIL	.0667	.0333
ADF Baskervald	.0667	.04

Tip

Define shorthand macros that simplify the application of guillemets, like, e. g.,

```
\newcommand*{\singlequotes}[1]
  {\singleguillemetright #1%
   \singleguillemetleft}
\let\sq=\singlequotes
```

and similar definitions for `\Singlequotes`, `\doublequotes`, and `\Doublequotes`.

Users working according to the French typesetting conventions will want to add extra spacing between the guillemets and the macro argument already in these macros. ■

Whether the guillemets must be height-adjusted for lowercase letters depends on the font. Careful judgment at various magnifications with a variety of samples is necessary.

Interaction with package `csquotes`. The users of package `csquotes` can hook up the guillemets as defined by `typog` with `\DeclareQuoteStyle`:

```
\DeclareQuoteStyle{typog-guillemets}
  {\doubleguillemetright}%    opening outer mark
  {\doubleguillemetleft}%    closing outer mark
  {\singleguillemetright}%    opening inner mark
  {\singleguillemetleft}%    closing inner mark
```

As always, the influence of package `babel` on `csquotes` has to be put into consideration. See Sec. 8 of the `csquotes` manual for a description of its configuration possibilities.

Use Case

All-capital words as for example acronyms put in guillemets that are raised somewhat almost always look better, whether using the French typographic convention (guillemets pointing outward plus some extra kerning) or the other way round (guillemets pointing inward). ■

Anticipated Changes & Possible Extensions

A correction in the other direction, i. e., lowering certain characters may also be desirable, to visually align them to the surrounding copy. Parentheses and in particular square brackets around all-lowercase text come into mind. ■

3.7.7 Inverted Exclamation Mark and Inverted Question Mark

The Spanish (Castilian, Asturian, etc.) language requires exclamatory sentences and questions to be followed by exclamation marks and question marks and recommends that they are preceded by inverted (or ‘upside-down’) versions of these punctuation characters.

Usually the horizontally-mirrored versions’ designs follow that of the lowercase letters. In particular the inverted marks exhibit descenders (see left-hand side of Fig. 2). For all-uppercase or all-smallcaps phrases, e. g. in headlines the descending parts of these marks disturb the tight block structure and it may be preferable to have inverted marks that align with the baseline.

`\capitalinvertedexclamationmark` The macros `\capitalinvertedexclamationmark` and `\capitalinvertedquestionmark` simultaneously provide bottom-aligned inverted marks for up to three different sets of exclamation marks or question marks, e. g., for uppercase, med-caps, and small-caps text¹⁹ or just for some stylistic alternatives offered by particularly well equipped fonts.

BOTH SINCE V0.5

```
\capitalinvertedexclamationmark{<number>}
\capitalinvertedquestionmark{<number>}
```

Typeset inverted exclamation marks and inverted question mark. Select the appropriate mark and the associated raise-amount with `<number>=1, 2, or 3`. We sketch the result of the automated correction process in Figure 2.

¹⁹ The small exclamation and question marks even made it into Unicode.

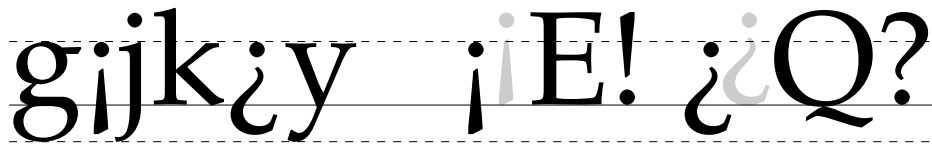


FIGURE 2: The left-hand sample shows the inverted exclamation marks and question marks wedged between some lowercase letters. The baseline is drawn with a solid line and the ex-height as well as the descender depth are indicated with dashed lines. ¶ On the right hand side we display the same marks in conjunction with capital letters. The gray glyphs demonstrate the result of shifting them up such that they exactly touch the baseline, which mimics the auto-raise mode described in the text. ¶

The sample font once again is URW Palladio .

Inverted exclamation and inverted question marks by the same $\langle number \rangle$ are raised by the same amount which is configurable with option `raiseinvertedmarks`, where $\langle number \rangle$ corresponds to the dimension at position $\langle number \rangle$. If the dimension is equal to 0 pt the marks are auto-raised as shown in Fig. 2. This may amount to different lengths for the inverted exclamation mark and inverted question mark even at the same $\langle number \rangle$. A nonzero dimension raises the marks by exactly that length. To get a near-zero length use e. g. 1 sp.

Tip

The letters $\mathcal{V}$, $\mathcal{W}$, and $\mathcal{Y}$ following an inverted question mark may need some manual kerning as well as $\mathcal{J}$ following an inverted exclamation mark. Macro `\itcorr` (p. 17) offers a solution that is almost portable across font changes and a $\langle strength \rangle$ of -1 is a good starting point. ■

The user-side macros `\capitalinvertedexclamationmark{\langle number \rangle}` and `\capitalinvertedquestionmark{\langle number \rangle}` call the parameter-less, internal macros

```
\typog@inverted@exclamationmark@<roman-numeral>
\typog@inverted@questionmark@<roman-numeral>
```

where $\langle roman-numeral \rangle = i, ii$, and iii correspond to $\langle number \rangle = 1, 2$, and 3 . Their defaults are `\textexclamdown` and `\textquestiondown` for all three numbers. Users can redefine the internal macros to hook up special fonts or characters.

3.8 Vertically Adjust Label Items of Environment `itemize`

Perfection of planned layout
is achieved only by institutions
on the point of collapse.
— CYRIL NORTHCOTE PARKINSON

The symbols that L^AT_EX uses to distinguish the items of `itemize` lists do not always align well in the vertical direction with the following text. Sometimes the label is too low, especially if followed by an uppercase (initial) letter. In rare occasions the label is placed too far above the baseline. If any label has been taken from a math-font vertical alignment with the text font is almost purely accidental.²⁰

`\uppercaseadjustlabelitems` Package typog lets the user vertically align the `itemize` labels for subsequent uppercase or lowercase letters, where the designations ‘uppercase’ and ‘lowercase’ are just names for two four-tuples of lengths (technically: `\noadjustlabelitems` `\dimens`) to shift the labels up or down.

ALL THREE SINCE V0.5

```
\uppercaseadjustlabelitems{<levels-to-adjust>}
\lowercaseadjustlabelitems{<levels-to-adjust>}
\noadjustlabelitems{<levels-to-adjust>}
```

Apply uppercase adjustment, lowercase adjustment or no adjustment to the labels in `itemize` environments at the `<levels-to-adjust>`. The adjustment values themselves, this is the vertical shifts are configured with options `uppercase-labelitemadjustments` and `lowercaselabelitemadjustments`. They are doubly font dependent: on the one hand the font where the label itself comes from and on the other hand the font of the copy.

The argument `<levels-to-adjust>` is a – possibly empty – comma separated list of the levels the adjustments are to be applied to. The levels themselves are given as *decimal* numbers, this is, 1, 2, 3, 4 or the special value `*` which stands for all four levels. An empty argument list also has a special meaning. Used within any `itemize` environment it automatically applies the adjustment to exactly this level.

Example

With the flexible syntax the following settings are possible.

- ▷ Correct all `itemize` labels for uppercase letters.

```
\uppercaseadjustlabelitems{*}
```

- ▷ Adjust nesting levels 1, 2, and 3 for uppercase letters and level 4 for lowercase.

```
\lowercaseadjustlabelitems{4}
\uppercaseadjustlabelitems{2,3,1}
```

- ▷ Within an `itemize` environment just turn off any correction for this level whatever it may be.

20 The exception being mathematics typeset as text via package `mathastext` [8].

```

\begin{itemize}
\noadjustlabelitems{}
\item ...
\end{itemize}

```

- ▷ Override `\labelitemi` with a right-pointing triangle and adjust its vertical position inside of a `typogsetup` environment.

```

\begin{typogsetup}
{uppercaselabelitemadjustments={.1em}}
\renewcommand*\labelitemi{{\small$\rhd$}}
\begin{itemize}
\uppercaseadjustlabelitems{}
\item ...
\end{itemize}
\end{typogsetup}

```

The observant reader will have noticed that the itemized list in this example uses this code. ■

Setup. To assist the user in finding the desired adjustments of the labels of `typog` provides macros that help setting up `lowercaselabelitemadjustments` and `uppercaselabelitemadjustments`. Their intended uses are in the draft phase of a document or in non-printed sections of the text.

The macros assume a ‘correct’ height that they derive from the measured height of a sample text scaled by a user-defined factor, which defaults to $\frac{1}{2}$.²¹ The then correct height gets indicated by a thin horizontal line parallel to the baseline. Thus, at sufficiently high magnifications it is possible to judge whether a label gets typeset too high or too low with respect to this reference line.

Note

The macros use the actual height of a given sample text. So, a lowercase sample should not contain any letters with ascenders.

Swashes whether upper- or lowercase always need special attention. ■

Sometimes uppercase-adjusted or lowercase-adjusted label items are needed outside of `itemize` environments.

```

\Adjustedlabelitemi
\Adjustedlabelitemii
\Adjustedlabelitemiii
\Adjustedlabelitemiv
\adjustedlabelitemi
\adjustedlabelitemii
\adjustedlabelitemiii
\adjustedlabelitemiv

```

ALL EIGHT SINCE V0.5

```

\Adjustedlabelitemi      \adjustedlabelitemi
\Adjustedlabelitemii     \adjustedlabelitemii
\Adjustedlabelitemiii    \adjustedlabelitemiii
\Adjustedlabelitemiv     \adjustedlabelitemiv

```

Typeset label items that are height-adjusted according to `uppercaselabelitemadjustments` or `lowercaselabelitemadjustments` either for uppercase (macros starting with a capital `A`) or lowercase (macros starting with

²¹ The default factor of $\frac{1}{2}$ hearkens back to STRIZVER’s suggestion that “[b]ullets should be centered on either the cap height or x-height of the neighboring text, [...]” [29, p. 220].

a small `a`). These macros can be used anywhere. They are *not* controlled by `\uppercaseadjustlabelitems`, `\lowercaseadjustlabelitems`, nor `\noadjustlabelitems`.

Use Cases

Free, user-defined lists. ¶ Injection of typog's height-adjustment functionality in other packages as, for example, tasks [24]. ■

`\typogadjuststairs`
SINCE V0.5

To get a quick overview how the four `itemize` labels align vertically `\typogadjuststairs` draws them at user-defined steps, typically $\frac{1}{4}$ pt, $\frac{1}{3}$ pt, or $\frac{1}{2}$ pt. It ignores any existing adjustments and in that way can be utilized as a first configuration step or, for a small $\langle\text{step-size}\rangle$ and a high $\langle\text{number-of-steps}\rangle$, for an easy refinement.

```
\typogadjuststairs[ $\langle\text{scale-factor}\rangle$ ]=.5]
                    { $\langle\text{step-size}\rangle$ } { $\langle\text{number-of-steps}\rangle$ }
                    { $\langle\text{sample}\rangle$ }
```

Generate stairs of $\langle\text{number-of-steps}\rangle$ vertically shifted label items; use the next odd number, if $\langle\text{number-of-steps}\rangle$ is even. Draw a reference hairline at $\langle\text{scale-factor}\rangle$ times the height of $\langle\text{sample}\rangle$, where $\langle\text{scale-factor}\rangle$ defaults to .5. The stairs start at a vertical shift of

$$-\frac{\langle\text{number-of-steps}\rangle - 1}{2} \times \langle\text{step-size}\rangle$$

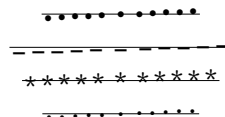
and repeat up to

$$\frac{\langle\text{number-of-steps}\rangle - 1}{2} \times \langle\text{step-size}\rangle.$$

The central step – which is always surrounded by a bit more space – shows the neutral, uncorrected alignment, this is, 0 pt. `\typogadjuststairs` never prints the contents of $\langle\text{sample}\rangle$.

Example

Play ball!



This is the result of `\typogadjuststairs{.25pt}{11}{ABC}` with the document's definitions of `\labelitem<N>`. The middle (6th) label item in each line is the uncorrected one. ■

`\typoguppercaseadjustcheck`
`\typoglowercaseadjustcheck`
BOTH SINCE V0.5

For a quick and easy check how the four label items vertically align *as configured* use `\typoguppercaseadjustcheck` and `\typoglowercaseadjustcheck`. Experienced users with a keen eye for type can apply these

macros even in the initial setup. An accurate determination of `\uppercaselabelitemadjustments` and `\lowercaselabelitemadjustments` is preferably done at a high magnification (e. g., 400% to 600% on a 100 dpi screen) with a representative sample of initial letters.

```
\typoguppercaseadjustcheck[⟨scale-factor⟩=.5]{⟨sample⟩}
\typoglowercaseadjustcheck[⟨scale-factor⟩=.5]{⟨sample⟩}
```

Typeset all four label items adjusted for uppercase or for lowercase with an indicator line at `⟨scale-factor⟩` times the `⟨sample⟩`'s actual height. The default `⟨scale-factor⟩` is .5. Both macros refer to the currently configured values for the uppercase or lowercase adjustments but they are independent of any settings done with `\uppercaseadjustlabelitems`, `\lowercaseadjustlabelitems`, or `\noadjustlabelitems`. Again, `⟨sample⟩` does not get printed.

Example

Uppercase check with `\typoguppercaseadjustcheck{ABCXYZ}`:

ABGH•—•QWYZ, 0123•—•4567

and similarly for lowercase: `\typoglowercaseadjustcheck{acexyz}`:

ace•—•mno, bdf•—•gkj, 0123•—•4567,

where we have bracketed the macro calls with selected uppercase and lowercase letters, or suitable figures. ■

In Table 5 on p. 35 we collected some suggestions for adjustment values in the *default* case when the label items are not redefined by the user and expand like

```
\labelitemi  ⌢ \labelitemfont \textbullet,
\labelitemii ⌢ \labelitemfont \bfseries \textendash,
\labelitemiii ⌢ \labelitemfont \textasteriskcentered, and
\labelitemiv ⌢ \labelitemfont \textperiodcentered.
```

They display as $\bullet$, --- , $\ast$, and $\cdot$.

3.9 Align Last Line of a Paragraph

The usual algorithms of L^AT_EX typeset the last line of a paragraph flush with the left margin unless `center`, `raggedleft` or `Centering`, `FlushRight` (package `ragged2e` [27]) are in effect. For an instructive discussion consult Ch. 17, “Paragraph End”, of reference 12. The following environments adjust the last lines of paragraphs in different ways.

The environment `lastlinerraggedleftpar` adjusts the various skips such that the last lines of the paragraphs gets typeset flush with the right margin.

`lastlinerraggedleftpar`
(env.)
`lastlineflushrightpar`
(env.)

```
\begin{lastlinerraggedleftpar}
...
\end{lastlinerraggedleftpar}
lastlineflushrightpar (alias)
```

TABLE 5: Some suggested values for the vertical adjustments of label items. The table assumes that the default definitions (of class article) for `\labelitem{N}` are in effect. The `itemize`-list levels `i`, `ii`, `iii`, and `iv` are referred to with $N = 1, 2, 3, 4$. All lengths are given as printer points (pt) and relate to a document font size of 10 pt.

Font Name	Uppercase Adjustments				Lowercase Adjustments			
	1	2	3	4	1	2	3	4
ADF Accanthis	1.0	.75	1.125	1.125	.0	-.25	-.125	.0
ADF Venturis	.75	.5	.75	.875	-.125	-.25	-.125	-.125
Aleo	1.25	1.0	1.25	1.25	.0	-.25	.0	.125
Charis SIL	1.0	1.0	1.0	1.125	.0	-.25	.0	.125
CM Roman	1.0	.75	1.0	1.0	-.25	-.5	-.25	-.25
Domitian	1.0	.75	1.0	1.0	.0	-.25	-.25	.0
Cochineal	.75	.75	.75	.75	-.25	-.5	-.25	-.25
EB Garamond	1.0	.75	.75	1.0	-.25	-.5	-.25	-.25
etbb [*]	1.0	.75	1.0	1.0	-.25	-.5	-.25	-.25
Extended Charter [†]	1.0	.75	1.0	1.0	.0	-.25	.0	.0
Gentium Plus	.75	.5	.75	.75	-.25	-.5	-.25	.0
GFS Bodoni	1.0	.75	1.0	1.0	.0	-.25	-.125	.0
GFS Didot	1.25	1.0	1.25	1.25	.0	.0	.0	.0
IBM Plex Serif	1.125	1.0	1.125	1.25	.25	.25	.25	.25
KP Serif [‡]	1.0	.75	1.0	1.0	-.25	-.5	-.25	-.25
Libertinus Serif	.75	.5	.75	.875	-.25	-.5	-.25	-.25
ML Modern	1.	.75	1.0	1.0	-.25	-.5	-.25	-.25
Source Serif Pro	1.0	.75	1.0	1.0	.0	-.25	.0	.0
Spectral	1.0	.75	.75	1.0	.0	-.25	-.25	.0
STIX	1.0	.75	1.0	1.0	-.25	-.5	-.25	.0
URW Palladio [§]	1.0	.75	1.0	1.0	.0	.0	.0	.0
Utopia	.75	.5	.75	.75	-.25	-.5	-.25	-.25

* EDWARD TUFTE's Bembo in package ETbb. Note the two initial capital letters in the filename.

† Found in package XCharter. Again note the two initial capitals in the filename.

‡ In package kpfons.

§ Contained in package mathpazo.

|| Utopia is available through package fourier or package mathdesign. For the latter pass option `adobe-utopia` to the package.

Note

Depending on the font packages loaded (and on their load order) the three macros `\textendash`, `\textasteriskcentered`, `\textperiodcentered` may be redefined and thus expand to different glyphs which need different adjustment values than the ones mentioned in Table 5. To ensure a set of correction values works use macros `\typoguppercaseadjustcheck` or `\typoglowspaceadjustcheck` inside of the target document. ■

The name `lastlineflushrightpar` is an alias for `lastlinerragged-leftpar`.

`lastlinecenteredpar`
(*env.*)

Center the last lines of the paragraphs enclosed by this environment.²²

```
\begin{lastlinecenteredpar} ... \end{lastlinecenteredpar}
```

Use Cases

`lastlineflushrightpar`: Narrow, justified parts of the text put flush against the right margin. ¶ `lastlinecenteredpar`: Table or figure captions typeset justified as centered boxes. ■

²² Also compare with the approach taken in reference [35](#).

3.10 Fill Last Line of a Paragraph

The problem of when and how to ‘fill’ the last line of a paragraph is quite intricate. We first define the problem then we proceed to general purpose functions and we close the section with specific environments to control the length of the last line.

3.10.1 Problem Description

Depending on the value of `\parindent`, either zero or nonzero, there may be the need to control the length of the last line of a paragraph.

1. `\parindent > 0` [35, O1]

If the last line of a paragraph is shorter than the `\parindent` of the following paragraph a visual gap tears open.



A similar issue occurs with displayed math in a flush left²³ setting, e.g., `amsmath` [2] and option `fleqn`.²⁴

A possible remedy is to reflow the paragraph in a way that its last line is clearly wider than `\parindent`, a typical suggestion being twice the `\parindent`.



2. `\parindent = 0` [35, O2]

If the last line of a paragraph is completely filled with text, i.e., flush with the right margin, it may become hard to spot the start of the following paragraph unless `\parskip` is large.²⁵



²³ The common practice of centering displayed equations does not call for the manipulations of a paragraph's last line discussed here.

²⁴ For displayed equations and `amsmath` the relevant parameter is `\mathindent`.

²⁵ Package `parskip` defines `\parskip` as 6 pt plus 2 pt for a base size of 10 pt.

A possible, more legible solution is to reformat the paragraph such that its last line leaves a marked gap with respect to the right margin.



Recommended gap widths range from two ems to twice the typical `\parindent`²⁶ value [9].

Tip

In theory both problems, O1 and O2 can be resolved by either shortening or prolonging the last line of the paragraph. The user to decide which direction to go and to choose the method that yields the most pleasing typographic results.

$\text{\TeX}$ evaluates paragraphs as a whole. Therefore, even changes intended for just the last line may affect the entire paragraph and potentially degrade its appearance.

Prudent users check the appearance of the problematic, original paragraph against one or more corrected versions—at a minimum, visually. Quantitative comparisons can be performed using `\tracingparagraphs`. ■

Important

For the techniques in the following two subsections to work the paragraphs treated with them should have certain advantageous properties.

- Technically, the paragraphs need to contain enough glue (see for example Sec. 3.13) to achieve a low badness such that the desired paragraph end is deemed feasible by $\text{\TeX}$.
- Aesthetically, the paragraphs must be long enough to absorb the change in last-line fill level otherwise their gray-values visibly deviate from the average. ■

3.10.2 Manual Changes

Most O1 or O2 situations can be navigated with do-it-yourself methods. Here are some common recipes.

1. End-of-paragraph intervention.

(a) Tie `~`

Tie the last words.

The problem with the tie may be a hyphenation of one of the words that participates in the tie. The next item avoids this disadvantage.

(b) `\mbox`

Join the last words or inline equation at the end of the paragraph with an `\mbox`.

This itemize list demonstrates vertically adjusted label items (Sec. 3.8).

26 For example, $\text{\LaTeX}$'s class article uses a `\parindent` of 25 pt.

(c) `\linebreak`

Add a `\linebreak` to the back part of the paragraph (approximately where the `\mbox` of item 1b would start) in a way that the last line receives the desired length [37]. As a result, the penultimate lines may become unsightly. Counteract this degradation e. g. with recipes 2a to 2c.

Tying and the use of `\mbox` can be generalized. They are useful not only at the end of paragraphs but also effective for grouping logical units of sentences or inline equations, ensuring the key information remains in the reader's focus. Of course, binding text segments together must stop when overfull lines begin to appear.

2. Uniform paragraph change.

(a) Vary spacing.

Modify the inter-word spacing, for example, with the macros introduced in Sec. 3.11.1.

Enclose the paragraph in either `loosespacing` or `tightspacing`. Increase the spacing $\langle level \rangle$ until the last line gets the desired length.

(b) Vary font tracking.

Enclose the paragraph in a `setfonttracking` group. See Sec. 3.12.1. Increase or decrease the tracking in steps of $\frac{1}{1000}$ em until the last line looks good.

(c) Vary font expansion.

Enclose the paragraph in a `setfontexpand` group. See Sec. 3.12.2.

3. A combination of any of the above items.

4. Some curveballs.

(a) If the paragraph already suffers from one of the problems that $\text{\TeX}$ addresses with `\doublehyphendemerits`, `\finalhyphendemerits`, or `\adjdemerits`, crank up one or all of these values to 10000 and observe whether the length of last line changes in the desired direction.

(b) If any influential microtype features have been enabled try with one more more of them *disabled*. See, e. g., environment `nofontexpansion` in Sec. 3.12.2.

(c) If inside of `\Centering`, `\RaggedLeft`, or `\RaggedRight` environments (all defined by package `ragged2e` [27]), slightly nudging the associated margin glues may influence the line breaks in the enclosed paragraphs. See Sec. 3.2 of the `ragged2e` documentation for the names of the influential skips.

3.10.3 Multi-Purpose Environments

`shortenpar` (*env.*) The two environments `shortenpar` and `prolongpar` can be employed in quite general situations when a paragraph should be typeset one line longer or shorter, e. g., to avoid a widow line²⁷ or a club line²⁸ [17, p. 104 and 22]. (See also Sec. 3.14 for special functions to avoid clubs or widows.) ‘Accidentally’, they also change the length of the last line of the paragraph.

```
\begin{shortenpar} ... \end{shortenpar}
```

Environment `shortenpar` decreases the `\looseness` of the paragraph.²⁹ This environment works well when the last line is short or the paragraph overall is loosely set.

```
\begin{prolongpar} ... \end{prolongpar}
```

This environment increases the `\looseness` of the paragraph, which is why it works best with decent or tight last lines that are nearly full.

3.10.4 Specialized Environments

We introduce environments not just skips to get the correct behavior – set up all paragraph parameters *before* the paragraph ends – and, at the same time, limit the range of this parameter change.

`covernextindentpar` (*env.*) Environment `covernextindentpar` can be helpful for [case 01](#), i. e., a too short last line.

```
\begin{covernextindentpar}[\langle dim \rangle]
...
\end{covernextindentpar}
```

The environment forces $\text{\TeX}$ to extend the last line of a paragraph such that it takes at least $2\text{\parindent}$ (if $\text{\parindent} \neq 0$), 2em (if $\text{\parindent} = 0$), or $\langle dim \rangle$ if called with an optional argument.

`openlastlinepar` (*env.*) The next environment, `openlastlinepar`, takes care of [case 02](#), i. e., a last line in a paragraph that is almost full or completely filled.

```
\begin{openlastlinepar}[\langle dim \rangle] ... \end{openlastlinepar}
```

It may resolve [case 02](#) as it attempts to prevent a completely filled line by introducing a partly unshrinkable `\parfillskip`. Without optional argument the threshold of unused last-line length is either $2\text{\parindent}$ (if $\text{\parindent} \neq 0$)

²⁷ The last line of a paragraph becomes a ‘widow’ (ger. *Hurenkind*) if it starts the following page or column.

²⁸ The first line of a paragraph is called ‘club’ or ‘orphan’ (ger. *Schusterjunge*) if it appears at the bottom of the page or column.

²⁹ `\looseness` is a $\text{\TeX}$ primitive [17, p. 103n]. A thorough discussion of the interaction of `\linepenalty` and `\looseness` can be found in reference 34.

or 2em (if `\parindent = 0`). The optional argument $\langle dim \rangle$ directly sets the gap threshold.

Note that using this environment can succeed in avoiding a fully filled last line; however, the result might still fall into case [type O1](#).

3.10.5 Consistent Spacing of Last Line

Since e-TeX the paragraph-breaking algorithm can be instructed to match the spacing of the last line of a paragraph to the next-to-last line with the built-in `\lastlinefit` [7, Sec. 3.8].³⁰ Package `typog` wraps this control in an environment to limit the scope as its effects are not always beneficial to the appearance of the body.

`\lastlinefitpar (env.)`
SINCE V0.5

Typeset a single paragraph with `\lastlinefit` set to a given value.

```
\begin{lastlinefitpar}[<value>] ... \end{lastlinefitpar}
```

The parameter range of `<value>` is zero to thousand. It controls the fraction of thousandths of the next-to-last-line's spacing to be used in the last line. This means that `<value> = 0` requests no transfer (as if `\lastlinefit` is not used at all) and `<value> = 1000` initiates exact transfer of the spacing. The default `<value>` is 1000.

Note that different `<value>`s can lead to different breakpoints in the affected paragraph. e-TeX still considers the *whole* paragraph when calculating the optimal line breaks.

Use Cases

Adapt the spacing of the last line of a paragraph to a very loose or very tight next-to-last line or to a whole paragraph that is very loose or very tight. ■

3.11 Spacing

90% of design is typography.
 And the other 90% is whitespace.
 — JEFFREY ZELDMAN

The functions described in this section rely only on plain L^AT_EX. No extra packages are required. Compare to the microtype-based functionality of Sec. 3.12.

3.11.1 Looser or Tighter Spacing

Never try to adjust lines by squeezing or stretching the tracking.
 Go for the subtle solution: adjust word spacing instead.
 — JAN MIDDENDORP [21, p. 119]

The environments in this section directly influence the spacing, this is, they change the width and stretchability of the horizontal space.

On the one hand, they act gently by adjusting the spacing only by a small amount. On the other hand, they operate decidedly in controlling the glue associated with the adjusted space. The latter is also important to ensure the monotonicity of the different `<level>`s. However, the strictly managed stretchability/

³⁰ The actual algorithm is somewhat involved. So, for ease of reference, we have included the relevant part of the e-TeX-manual as Appendix B starting on p. 71.

shrinkability may lead to many overfull boxes with `\fussy`, or when applied to short lines.

`loosespacing` (env.)

`tightspacing` (env.)

Environments `loosespacing` and `tightspacing` introduce four $\langle level \rangle$ s of ‘looseness’ or ‘tightness’, where $\langle level \rangle = 0$ disables the functionalities. The higher the $\langle level \rangle$ the looser or tighter the text will be typeset.

```
\begin{loosespacing}[\langle level \rangle] ... \end{loosespacing}
```

Environment `loosespacing` increases the width of a space by the percentages given in Tab. 6.

$\langle level \rangle$	Adjustment %	Note
0	n/a	neutral
1	+5	default
2	+10	
3	+20	
≥ 4	+30	

TABLE 6: Adjustments made by environment `loosespacing` to `\spaceskip`. The mapping of $\langle level \rangle$ to the exact skip definitions are $1 \mapsto 1.05^{+.5}_{-.1}$, $2 \mapsto 1.1^{+.5}_{-.1}$, $3 \mapsto 1.2^{+.6}_{-.2}$, and $\geq 4 \mapsto 1.3^{+.8}_{-.3}$, where all factors scale with `\dimen2`, the current font’s space-width.

The default level of `loosespacing` is 1.

```
\begin{tightspacing}[\langle level \rangle] ... \end{tightspacing}
```

Environment `tightspacing` decreases the width of a space by the percentages given in Tab. 7.

$\langle level \rangle$	Adjustment %	Note
0	n/a	neutral
1	-1.25	default
2	-2.5	
3	-5	
≥ 4	-10	

TABLE 7: Adjustments made by environment `tightspacing` to `\spaceskip`. The mapping of $\langle level \rangle$ to the exact skip definitions are $1 \mapsto .9875^{+.0125}_{-.5}$, $2 \mapsto .975^{+.025}_{-.5}$, $3 \mapsto .95^{+.05}_{-.5}$, and $\geq 4 \mapsto .9^{+.1}_{-.5}$, where all factors scale with `\dimen2`, the current font’s space-width.

The default level of `tightspacing` is 1.

Note

At a given $\langle level \rangle$ the changes of `loosespacing` are much larger than those of `tightspacing`. ■

Use Cases

Tighten spacing in narrow lines as, for example, in multi-column environments. ¶
Nudge line breaks or hyphenation points. ¶ Separate clashing descenders and ascenders. ¶ Eliminate rivers. ■

3.11.2 Wide Space

The `\widespace` macro and its companion `\narrowospace` derive their appearances from several of the current font's `\fontdimen⟨number⟩`s. T_EX addresses the latter by integers, which is totally non-memnonic. Therefore, we play softball by first presenting Tab. 8 that associates the `\fontdimen⟨number⟩`s with their meanings and also reports on their current values (for this document).³¹

#	Description	Value
1	Slant per 1 pt height	0
2	Interword space width	23.3
3	Interword stretch	11.6
4	Interword shrink	7.8
5	x° height	47.5
6	<code>\quad</code> height	100
7	Extra space width	3.9

TABLE 8: All T_EX font parameters normalized to the font's quad-size. The first column `#` states the index of the `\fontdimen⟨number⟩` parameter: `⟨number⟩`. Column 2 presents short descriptions of `\fontdimen⟨number⟩`. As examples, the values for the current font are shown in column 3 as percentages.

`\widespace`
`\widespace*`
STARRED FORM
SINCE V0.2

Typeset a wide, sentence-ending space as if in `\nonfrenchspacing` mode. Consult Table 9 for a comparison of the various sizes.

`\widespace` `\widespace*`

The unstarred macro `\widespace` inserts a space that is as wide as the font's sentence-ending space in `\nonfrenchspacing` mode, this is

$\text{\fontdimen2} + \text{\widespacestrength} \times \text{\fontdimen7}.$

Its width is independent of any `\frenchspacing` or `\nonfrenchspacing` settings, but depends on `\widespacestrength` which defaults to 1. The latter can be overridden by the user to get a more or less pronounced effect.

If `\fontdimen7` happens to be zero `\widespace` uses

$\text{\widespacescale} \times \text{\fontdimen2}$

as width instead, where `\widespacescale` defaults to 1.125. The stretchability and shrinkability of `\widespace` always are scaled with `\widespacescale`. The `\widespacescale` too can be redefined by the user to achieve different effects.

The starred form, `\widespace*`, unconditionally uses the `\fontdimen7 = 0` code-path.

The sentence that ends with
 'l.' uses `\widespace` after the
 period.

31 The association is given in Appendix F (p. 433) of reference 17. For a concise and understandable explanation of the T_EX `\fontdimen` parameters consult reference 10.

Use Case

Useful as a sentence-ending space if, for example, the sentence ends in an abbreviation with a period or decimal number without trailing digits *and* the next sentence should be delimited in a clearer way. ¶ Open tight lines with a series of `\widespaces`.³² ■

3.11.3 Narrow Space`\narrowospace`

Typeset a narrow space. Consult Table 9 for a comparison of the various sizes.

`\narrowospace*`

SINCE v0.2

<code>\narrowospace</code>	<code>\narrowospace*</code>
----------------------------	-----------------------------

The unstarred macro `\narrowospace` inserts a narrow space with the width

$$\text{\fontdimen2} - \text{\narrowospacestrength} \times \text{\fontdimen7}$$

if `\fontdimen7` is different from zero or otherwise

$$\text{\narrowospacestretch} \times \text{\fontdimen2}.$$

The starred version, `\narrowospace*`, unconditionally uses the `\fontdimen7 = 0` code-path. Refer to Table 8 for the meanings of the various `\fontdimen` parameters.

The stretchability and shrinkability of `\narrowospace` always get scaled with `\narrowospacestretch`. Both factors, `\narrowospacestrength` and `\narrowospacestretch` can be redefined by the user; their defaults are .5 and .9375.

Use Cases

Tighten loose lines with a series of `\narrowospace`s.³³ ¶ Compensate the visual gap that may occur when switching from upright shape to italic shape. ■

TABLE 9: Exemplary comparison of standard `\space` versus `\narrowospace` and `\widespace`. All values are relative to the size of the current font's quad-size and shown as a percentage of it. `\narrowospace` and `\widespace` use the package's defaults. ¶ The upper values in the 'Width' column for `\narrowospace` and `\widespace` refer to the `\fontdimen7 ≠ 0` case and the lower ones to the `\fontdimen7 = 0` code-path.

Macro	Width	Stretch	Shrink
<code>\narrowospace</code>	21.4 21.8	10.9	7.3
<code>\space</code>	23.3	11.6	7.8
<code>\widespace</code>	27.2 26.2	13.1	8.7

32 See also "Investigating the badness of a paragraph" on Page 10.

33 Footnote 32 again applies.

3.12 Microtype Front-End

The manual uses `\spaced-`
`emdash` for the em-dash.

The functionalities are just front ends of selected macros in package `microtype`—welcome syntactic sugar.

Important

All macros and environments introduced in this section require that package `microtype` [26] has been loaded, preferably *before* package `typog`

```
\usepackage[⟨microtype-options⟩...]{microtype}
```

```
\usepackage[⟨typog-options⟩...]{typog}
```

in the document preamble. ■

3.12.1 Tracking

Caution

The tracking changes may interfere with implicit changes of tracking declared with `\SetTracking`. Explicit calls to `\textls` remain in effect. ■

`setfonttracking`
(env.)

Override the default tracking for all fonts.

```
\begin{setfonttracking}{⟨delta⟩} ... \end{setfonttracking}
```

The environment `setfonttracking` manages a group for `\lsstyle` of package `microtype`. The change `⟨delta⟩` in tracking is given as multiples of $\frac{1}{1000}$ em. Positive as well as negative values of `⟨delta⟩` are allowed.

See Sec. 5.3, ‘Tracking’, and 7, ‘Letterspacing revisited’, in the documentation of `microtype` [26] for a detailed explanation.

For font combinations involving monospaced fonts (T_EX lingo: typewriter) an overly large spacing may show up at the borders where fonts change. This is caused by the calculation of the “outer spacing” described in Sec. 5.3 of the `microtype` manual.

Use configuration variable `trackingttspacing` to reduce the outer spacing to a reasonable value either directly at package-load time

```
\usepackage[trackingttspacing={250, 75, 50}]{typog}
```

or using `\typogsetup` in the document *preamble* (after loading `microtype` and `typog`)

```
\typogsetup{trackingttspacing={250, 75, 50}}
```

If the argument of option `trackingttspacing` is omitted the outer spacing defaults to 300, 90, 60.

Use Cases

Nudge line breaks or hyphenation points. ¶ Avoid clashes of descenders and ascenders, e.g., for `\smashed` symbols of inline math.—Think of integrals. ¶ Control the length of the last line in a paragraph. ■

3.12.2 Font Expansion

`setfontshrink` (*env.*) Adjust the limits of either only stretchability or only shrinkability and zero the other component, i. e., shrinkability and stretchability.

`setfontstretch` (*env.*)

```
\begin{setfontshrink}[\langle level \rangle] ... \end{setfontshrink}
\begin{setfontstretch}[\langle level \rangle] ... \end{setfontstretch}
```

A $\langle level \rangle$ of zero is a no-op. Tables 10 and 11 summarize the values for stretch and shrink in these environments.

$\langle level \rangle$	stretch	shrink	Note
0	n/a	n/a	no operation
1	0	5	default
2	0	10	
3	0	20	

TABLE 10: Preconfigured values for shrink inside of environment `setfontshrink` as $\frac{1}{1000}$ em. Note that all stretch values are zero, so the fonts only can shrink.

$\langle level \rangle$	stretch	shrink	Note
0	n/a	n/a	no operation
1	5	0	default
2	10	0	
3	20	0	

TABLE 11: Preconfigured values for stretch inside of environment `setfontstretch` as $\frac{1}{1000}$ em. Note that all shrink values are zero, so the fonts only can stretch.

The three (nonzero) shrink limits of `setfontshrink` can be configured with package option `shrinklimits` and – in the same way – the three (nonzero) stretch limits of `setfontstretch` with package option `stretchlimits`.

Use Cases

Nudge line breaks or hyphenation points. ¶ Control the length of the last line in a paragraph. ■

`setfontexpand` (*env.*) Manipulate both, stretch and shrink values at the same time.

```
\begin{setfontexpand}[\langle level \rangle] ... \end{setfontexpand}
```

Table 12 gives an overview of the values associated with $\langle level \rangle$.

$\langle level \rangle$	stretch	shrink	Note
0	n/a	n/a	no operation
1	5	5	default
2	10	10	
3	20	20	

TABLE 12: Preconfigured values for shrink and stretch inside of environment `setfontexpand` as $\frac{1}{1000}$ em. Note that both shrink and stretch values are nonzero, so the fonts can shrink or expand.

The six shrink and stretch limits of `setfontexpand` can be configured with package options `shrinklimits` and `stretchlimits`.

Notes

- Environment `setfontexpand` shares its `shrinklimits` with `setfontshrink` and its `stretchlimits` with `setfontstretch`.
- These environments do not nail down any font’s expansion but only set up its available range. See Sec. 3.3, “Font Expansion”, in the microtype documentation [26].

Moreover, a text may not ‘respond’ neither to `setfontshrink`, `setfontstretch`, nor `setfontexpand` because $\TeX$ already considers it optimal without expansion or within the previous expansion limits, e. g., those set at microtype load time as opposed to typog’s load time. ■

Use Cases

Nudge line breaks or hyphenation points. ¶ Control the length of a paragraph, e. g., to avoid a widow. ■

`nofontexpansion`
(env.)
`nofontexpand` (env.)

Disable the microtype feature ‘expansion’ inside of the environment.

```
\begin{nofontexpansion} ... \end{nofontexpansion}
nofontexpand (alias)
```

The name `nofontexpand` is an alias for `nofontexpansion`.

Use Cases

Nudge line breaks or hyphenation points. ¶ Prevent severe scaling effects in paragraphs strongly manipulated by other means, e. g., `shortenpar` or `prolongpar`. ■

3.12.3 Character Protrusion

`nocharprotrusion`
(env.)

Disable the microtype feature ‘protrusion’ inside of the environment.

```
\begin{nocharprotrusion} ... \end{nocharprotrusion}
```

Use Cases

Table of Contents or similar tables with aligned section numbers. ¶ Any table with left- or right-aligned numerals in particular tabular numerals. ¶ Index. ■

3.13 Sloppy Paragraphs

`\slightlyloppy`
`slightlyloppypar`
(env.)

Experienced $\LaTeX$ users know that `\sloppy` is more of a problem by itself and not really a viable solution of the “overfull box” syndrome.

We define the macro `\slightlyloppy` and the associated environment, `slightlyloppypar`, with a user-selectable $\langle sloppiness \rangle$ parameter. The constructions recover the known settings `\fussy` ($\langle sloppiness \rangle = 0$) and `\sloppy` ($\langle sloppiness \rangle \geq 8$), and introduce seven intermediate $\langle sloppiness \rangle$ levels.³⁴ The default $\langle sloppiness \rangle$ is 1.

³⁴ Also compare with the findings for `\emergencystretch` in reference 33.

```

\slightlyloppy[⟨sloppiness⟩]
\begin{slightlyloppypar}[⟨sloppiness⟩]
...
\end{slightlyloppypar}

```

Table 13 summarizes the adjustments that `\slightlyloppy` makes depending on the `⟨sloppiness⟩` level.

TABLE 13: Adjustments made by `\slightlyloppy` to various $\text{\TeX}$ parameters at different levels of `⟨sloppiness⟩`.

<code>⟨sloppiness⟩</code>	<code>\tolerance</code>	<code>\hfuzz</code> <code>\vfuzz</code> pt	<code>\emergencystretch</code> <code>G</code> em	Note
0	200	.1	0	$\text{\TeX}$: <code>\fussy</code> default
1	330 [†]	.15	.375 [‡]	
2	530 [†]	.2	.75 [‡]	
3	870 [†]	.25	1.125 [‡]	
4	1410 [†]	.3	1.5 [‡]	
5	2310 [†]	.35	1.875 [‡]	
6	3760 [†]	.4	2.25 [‡]	
7	6130 [†]	.45	2.625 [‡]	
≥ 8	9999	.5	3	$\text{\TeX}$: <code>\sloppy</code>

[†] All intermediate levels set `\pretolerance = \tolerance/2`.

[‡] The intermediate levels scale the amount of available glue G (indicated in column 4 of the table) for `\emergencystretch` with the actual line length, this means, in these levels

$$\text{\emergencystretch} = G \times \frac{\text{\linewidth}}{\text{\textwidth}}.$$

to prevent excessive stretchability in narrow lines.

Environment `slightlyloppypar[⟨sloppiness⟩]` mimics $\text{\LaTeX}$'s `sloppypar`, while offering the flexibility of `\slightlyloppy`.

Use Cases

Drop-in replacement for `\sloppy`, whether explicit or implicit (think of `\parbox`). ¶
Initial paragraphs in theorem environments (e. g., as defined by `amsmath` or `amsthm`), where the theorem head already takes a lot of space. ¶ Bibliographies as environment `thebibliography` sets `\sloppy`. ■

3.14 Vertically Partially-Tied Paragraphs

L^AT_EX provides several macros and environments to tie material vertically—most prominently `samepage` and `minipage`.³⁵ Typog’s macros and environments constitute more sophisticated but weaker forms of these. They tie only the first or last couple of lines in a paragraph while the rest of the paragraph gets broken into pages by T_EX in the usual way.

The macros and environments described in this section locally set e-T_EX penalty arrays [7, Sec. 3.8]. In addition the environments `vtietoppar`, `vtiebotpar`, and `vtiebotdisptoppar` explicitly issue a `\par` at the end of the group.

Avoid a club line in each partial paragraph.

`\vtietop`

`vtietoppar (env.)`

```
\vtietop[⟨number-of-lines⟩]
\begin{vtietoppar}[⟨number-of-lines⟩] ... \end{vtietoppar}
```

Vertically tie the first *⟨number-of-lines⟩* in a paragraph. Zero or one for *⟨number-of-lines⟩* are no-ops. Up to nine lines can be fused. The default is to link three lines.

Use Cases

String together the first paragraph right after a sectioning macro. ¶ Tie the first line of an itemized, enumerated, or a description list with the paragraph following `\item`. ■

`\splicevtietop`

Inside of a `list` a one-off solution simply concatenates `\item[...] \vtietop` to fuse the line with the `item#`, the representation of the `enum#`, or the description term with the first paragraph. For a systematic use prefer `\splicevtietop` and apply it as the first thing in the `list` body.

```
\splicevtietop[⟨number-of-lines⟩]
```

Use this macro *inside* of a `list`-like environment to equip each `\item` with `\vtietop[⟨number-of-lines⟩]`. The default *⟨number-of-lines⟩* is three as for any of the `vtie...` functions.

Example for a description list and plain L^AT_EX:

```
\begin{description}
  \splicevtietop[2]
  \item[...]
\end{description}
```

Alternatively with package `enumitem` [5]:

```
\begin{description}[first=\splicevtietop[2]]
  \item[...]
\end{description}
```

or shorter and with the default *⟨number-of-lines⟩*, 3, using the `enumitem` style³⁶ `vtietop`:

³⁵ A valuable complement to these is package `needspace` [41] which takes a different approach and reliably works in *mixed* horizontal and vertical mode situations.

³⁶ The documentation of `enumitem` prosaically calls them ‘keys’ (Section 3) not ‘styles’.

`vtietop` (*enumitem* key)

```

\usepackage{enumitem}
\begin{description}[vtietop]
  \item[...]
\end{description}

```

`\vtiebot`

Avoid a widow line in each partial paragraph.

`vtiebotpar` (*env.*)

```

\vtiebot[⟨number-of-lines⟩]
\begin{vtiebotpar}[⟨number-of-lines⟩] ... \end{vtiebotpar}

```

Vertically tie the last *⟨number-of-lines⟩* in a paragraph. Zero or one for *⟨number-of-lines⟩* are no-ops. Up to nine lines can be fused. The default is to link three lines.

Example — Ornamental separator.

Ornamental separators should never be stranded. Therefore, macro `\ornamentalseparator` precedes the `\medskip` by `\nobreaks`:

```

\newcommand*{\ornamentalseparator}{%
  \nobreak\medskip
  \centerline{*\:*\:*\}%
  \nobreak\medskip}

```

However, this definition only ties the last line of the previous paragraph and the first line of the following paragraph to the separator. By enclosing the previous and following paragraphs in `vtiebotpar` and `vtietoppar` environments more than just single lines can be tied to it, thereby increasing its textual context.

```

\begin{vtiebotpar}[⟨number-of-bottom-lines⟩]
  % previous paragraph
\end{vtiebotpar}

\ornamentalseparator

\begin{vtietoppar}[⟨number-of-top-lines⟩]
  % following paragraph
\end{vtietoppar}

```

If the paragraphs wrapped in `vtiebotpar` and `vtietoppar` are long, the amount of text around `\ornamentalseparator` can be controlled by varying *⟨number-of-bottom-lines⟩* and *⟨number-of-top-lines⟩*. ■

`vtiebotdisp` (*env.*)

Avoid a display widow line in each partial paragraph.

```

\begin{vtiebotdisp}[⟨before-disp-number-of-lines⟩]
  ...
\end{vtiebotdisp}

```

Vertically tie the last *⟨before-disp-number-of-lines⟩* in a paragraph before a display. Zero or one for *⟨before-disp-number-of-lines⟩* are no-ops. Up to nine lines can be fused. The default is to link three lines.

To use the environment, bracket the paragraph before the display (the one that needs protection) and the associated displayed math:

```
\begin{vtiebotdisp}
% vertically tied paragraph before the math display
\begin{equation}
% math
\end{equation}
\end{vtiebotdisp}
```

vtiebotdisptoppar
(env.)

Avoid a display widow, compound the display with its preceding *and* following paragraph, and avoid a club line in the paragraph right after the display.

```
\begin{vtiebotdisptoppar} [⟨before-disp-number-of-lines⟩]
[⟨after-disp-number-of-lines⟩]
...
\end{vtiebotdisptoppar}
```

Vertically tie the last $\langle\textit{before-disp-number-of-lines}\rangle$ in the paragraph before a display and the first $\langle\textit{after-disp-number-of-lines}\rangle$ in the paragraph after the display. Moreover, turn the paragraphs and the display into an unbreakable unit.³⁷

Zero or one for $\langle\textit{before-disp-number-of-lines}\rangle$ as well as $\langle\textit{after-disp-number-of-lines}\rangle$ are no-ops for the paragraph. Up to nine lines each can be fused.

Both optional arguments default to three. If only the first argument is given the second acquires the same value.

To use the environment, bracket the paragraphs before and after the display:

```
\begin{vtiebotdisptoppar}
% vertically tied paragraph before the math display
\begin{equation}
% math
\end{equation}
% vertically tied paragraph after the math display
\end{vtiebotdisptoppar}
```

See also Sec. 3.10.3 for other methods to avoid club or widow lines.

Partial Paragraphs And Counting Lines. The top-of-paragraph ties, `\vtietop` and `vtietoppar` count $\langle\textit{number-of-lines}\rangle$ from the beginning of every partial paragraph. Each displayed math in the paragraph resets the count. The bottom-paragraph ties, `\vtiebot`, `vtiebotpar`, `\vtiebotdisp`, and `vtiebotdisppar` count backward from the end of each partial paragraph. Again, each displayed math in the paragraph resets the count. According to T_EX's rules, a displayed math formula always is counted as *three* lines no matter its contents. Table 14 summarizes these rules with the help of an example.

37 The paragraphs and the display are concreted together by setting both `\predisplaypenalty` and `\postdisplaypenalty` to 10000.

TABLE 14: Exemplary, eight-line paragraph compounded of two partial paragraphs of three and two lines and a displayed math formula of arbitrary size sandwiched in between.

Continuous Line Number	Example Contents	$\backslash\text{vtietop}^\dagger$ Count	$\backslash\text{vtiebot}^\ddagger$ Count
1	Text line ₁	1	3
2	Text line ₂	2	2
3	Text line ₃	3	1
4	} Display math		
5			
6			
7	Text line ₄	1	2
8	Text line ₅	2	1

$\dagger$ This is e-TeX's counting scheme of $\backslash\text{clubpenalties}$; it also holds for vtietoppar .

$\ddagger$ The same counting scheme also holds for vtiebotpar , $\backslash\text{vtiebotdisp}$, and vtiebotdispar . It is implied by e-TeX's line counts of $\backslash\text{widowpenalties}$ and $\backslash\text{displaywidowpenalties}$ on which the functions of this package are based.

Tips

- The environments can be combined to arrive at paragraphs that simultaneously are protected against club lines and (display) widow lines.
- For very long derivations that are not interrupted and thus made breakable with the help of $\backslash\text{intertext}^{38}$ or $\backslash\text{shortintertext}^{39}$ it is desirable to make the display breakable. This is achieved with $\backslash\text{allowdisplaybreaks}$ or the environment breakabledisplay which will be described in Sec. 3.15. ■

Use Cases

Avoid widows and orphans, e. g., those turned up by package `widows-and-orphans` [23]. ¶ Extend the convention of using three to four lines instead of a single club or widow line to a flexible, context-dependent rule that aims to keep all the relevant information together for the reader—ideally, at least. ■

38 Introduced in package `amsmath` [2].

39 Defined in package `mathtools` [14].

3.15 Breakable Displayed Equations

`breakabledisplay`
(*env.*)

Package `amsmath` offers `\allowdisplaybreaks` to render displayed equations breakable at each of their lines. Environment `\breakabledisplay` is a wrapper around it which limits the macro's influence to the environment. Furthermore, the default $\langle level \rangle$ of `breakabledisplay` is 3 whereas that of `\allowdisplaybreaks` is 4. This results in fewer breaks in displayed equations, making `breakabledisplay` more suitable for automated page breaking.

```
\begin{breakabledisplay}[ $\langle level \rangle$ ]  
...  
\end{breakabledisplay}
```

Environment `breakabledisplay` simply passes on $\langle level \rangle$ to `\allowdisplaybreaks`. Table 15 shows the default penalties that `amsmath` associated with each of the $\langle level \rangle$ s.

TABLE 15: Penalties `\interdisplaylinepenalty` associated with different $\langle level \rangle$ s of environment `breakabledisplay`. Depending on the version of package `amsmath` the actual penalties may differ.

$\langle level \rangle$	<code>\interdisplay-</code> <code>linepenalty</code>	Note
0	10000	no operation
1	9999	
2	6999	default
3	2999	
4	0 [†]	

[†] This is the default of `\allowdisplaybreaks`.

Tips

- Terminating a line with `\*^` inhibits a break after this line.
- A `\displaybreak[ $\langle level \rangle$ ]` can be set for *each* line of the displayed equation separately. `LATEX` resumes with the original value of `\interdisplaylinepenalty` in the following lines. Note, however, that `\displaybreak` is only effective (immediately) in front of `\*^`. See the “User’s Guide for the `amsmath` Package” for a discussion [2, Sec. 3.9].
- If a discretionary break of the displayed equation is to be accompanied with some aid for the reader, team `\intertext` (or `\shortintertext`) with `\displaybreak` as, e. g.,

```
\newcommand*{\discretionarydisplaybreak}  
{\intertext{\hfill Eq.~cont.~on next page.}%  
 \displaybreak  
 \intertext{Eq.~cont.~from prev.~page.\hfill}} ■
```


Use Cases

Extremely long derivations without interspersed `\intertext` or `\shortintertext`.  Draft phase of a document. ■

3.16 Setspace Front End

Package `setspace` [30] is a base hit when it comes to consistently setting the line skip for a document via the macro `\setstretch`. The interface of `\setstretch` though is unintuitive as it asks for an obscure factor. The L^AT_EX user however prefers to keep her eyes on the ball and set the line skip directly (e. g. 12.5 pt) or the lines' leading to a length or percentage of the font's size.⁴⁰ This is where the following macros go to bat.

Important

All macros that are introduced in this section rely on macro `\setstretch`. So package `setspace` must have been loaded with

```
\usepackage{setspace}
```

in the document preamble. ■

`\setbaselineskip`
SINCE V0.3

Set the line skip using an absolute length—technically: a dimen.

```
\setbaselineskip{<baseline-skip>}
```

Set the `\baselineskip` to `<baseline-skip>`. This is what a non-initiated user expects from the assignment

```
\setlength{\baselineskip}{<baseline-skip>}
```

The `<baseline-skip>` can contain a rubber (stretch/shrink) component, however, `\setbaselineskip` will discard of it and issue a warning that only the fixed-length part will be used in the computation.

Example

- Let us assume we want to lighten the gray value of the copy a tad with the `\baselineskip` increased from 12 pt to 12.5 pt. To this end we say:

```
\setbaselineskip{12.5 pt}
```

- In a generic part of the document, where the actual `\baselineskip` is not known, we can refer to its current value and rescale it:

```
\setbaselineskip{\baselineskip * 12.5 / 12}
```

Care should be taken if code like the above is implicitly or explicitly repeated, because it results in a geometric series. ■

`\resetbaselineskip`
SINCE V0.3

Reset the `\baselineskip` to its original value.

```
\resetbaselineskip
```

⁴⁰ To find out about the current font's size and the `\baselineskip` in printable form check out Sec. 3.2.1 on p. 7.

This macro simply expands to `\setstretch{1}`. So, we rely on setspace's notion of what is a single-line `\baselineskip`.

`\setbaselineskippercentage` Set the `\baselineskip` with a relative value calculated as a percentage of the current font's point size.

SINCE V0.3

```
\setbaselineskippercentage{<baselineskip-percentage>}
```

Set `\baselineskip` to $\text{\typogfontsize} \times \langle \text{baselineskip-percentage} \rangle / 100$.

Example

We modify the previous example and assume a point size of 10 pt, but now write

```
\setbaselineskippercentage{125}
```

which sets `\baselineskip` to $10 \text{ pt} \times 125/100 = 12.5 \text{ pt}$. ■

`\setleading` Set the `\baselineskip` with an absolute length that gets *added to* `\typogfontsize`.

SINCE V0.3

```
\setleading{<leading>}
```

Set the `\baselineskip` to `\typogfontsize` plus `<leading>`. Note that `<leading>` can be negative, e. g. to set solid.

Example

Another solution of the previous example, given a point size of 10 pt is to write

```
\setleading{2.5pt}
```

which sets `\baselineskip` to $10 \text{ pt} + 2.5 \text{ pt} = 12.5 \text{ pt}$. ■

`\setleadingpercentage` Set the `\baselineskip` to `\typogfontsize` *plus* a relative value calculated as a percentage of `\typogfontsize`.

SINCE V0.3

```
\setleadingpercentage{<leading-percentage>}
```

Set `\baselineskip` to $\text{\typogfontsize} \times (1 + \langle \text{leading-percentage} \rangle / 100)$.

Example

We modify the previous example and again assume a point size of 10 pt, but now write

```
\setleadingpercentage{25}
```

which sets `\baselineskip` to $10 \text{ pt} \times (1 + 25/100) = 12.5 \text{ pt}$. ■

`\typogfontsize` The macros `\setbaselineskippercentage`, `\setleading`, and `\setleadingpercentage` all depend on the font size. By changing `\typogfontsize` they can be configured for different font sizes.

(dimen)
SINCE V0.3

The length `\typogfontsize` gets initialized at the end of the preamble to the default font's quad size:⁴¹

41 For an overview of the various `\fontdimen<number>` parameters consult Tab. 8 on p. 44.

```
\typogfontsize=\fontdimen6\font
```

which is also called its “nominal size” or its “point size”. This assignment can be repeated at any point in the document to record a reference font’s size. To set just `\typogfontsize` without changing the current font, encapsulate the font change in a group and export the new value:

```
\begingroup
\usefont{T1}{Arvo-TLF}{m}{n}\selectfont
\normalsize
\global\typogfontsize=\fontdimen6\font
\endgroup
```

An alternative to relying on the point size is using the actual size of an upper-case letter:

```
\settoheight{\typogfontsize}{CEMNORSUVWXZ}
```

With `\typogfontsize` defined this way it becomes trivial to set solid:

```
\setleading{0pt}
```

or

```
\setleadingpercentage{0}
```

Tip

All macros in this section actually accept expressions of their argument types, though the sick rules of $\text{\TeX}$ *<dimen>*- and *<skip>*-expressions apply.

Here are some forms that do work:

```
\setbaselineskip{12pt + 0.6667pt}
\setbaselineskip{12pt * 110 / 100}
\setbaselineskippercentage{100 + 25}
\setleading{1pt / -2.0}
\setleadingpercentage{10 - 25 / 2} ■
```

3.17 Smooth Ragged

The attention someone gives
to what he or she makes
is reflected in the end result,
whether it is obvious or not.
— ERIK SPIEKERMANN

Package `typog` implements a novel approach to typesetting ragged paragraphs. Instead of setting the glue inside a paragraph to zero and letting the line widths vary accordingly [36], we prescribe the line widths with T_EX's `\parshape` primitive and leave the stretchability and shrinkability of the glue unchanged.

Caution

None of the following environments work within lists (e.g. `description`, `enumerate`, or `itemize`). ■

<code>smoothraggedrightshapetriplet</code> (<i>env.</i>)	We introduce three environments that define three, five, or seven different line lengths (which T _E X will, of course, repeat for paragraphs longer than three, five, or seven lines): <code>smoothraggedrightshapetriplet</code> , <code>smoothraggedrightshapequintuplet</code> , and <code>smoothraggedrightshapeseptuplet</code> . They work for paragraph lengths up to 99, 95, and 98 lines, respectively.
<code>smoothraggedrightshapequintuplet</code> (<i>env.</i>)	
<code>smoothraggedrightshapeseptuplet</code> (<i>env.</i>)	

```
\begin{smoothraggedrightshapetriplet}[\option]...{\width1}{\width2}{\width3}
...
\end{smoothraggedrightshapetriplet}
\begin{smoothraggedrightshapequintuplet}[\option]...{\width1}...{\width5}
...
\end{smoothraggedrightshapequintuplet}
\begin{smoothraggedrightshapeseptuplet}[\option]...{\width1}...{\width7}
...
\end{smoothraggedrightshapeseptuplet}
```

The environments take $N = 3, 5$, or 7 mandatory line-width parameters, where each $\langle widthI \rangle$, $I = 1, \dots, N$, is a skip, i. e., a dimen that can include some glue.

Option `leftskip=\langle dim \rangle` sets the left margin of the smooth ragged paragraph to $\langle dim \rangle$; it is similar to the T_EX parameter `\leftskip`. Option `parindent=\langle dim \rangle` sets the first-line indent of the smooth ragged paragraph to $\langle dim \rangle$; it is similar to the T_EX parameter `\parindent`.

<code>smoothraggedrightpar</code> (<i>env.</i>)	Environment <code>smoothraggedrightpar</code> builds upon the three environments just described, using them as ‘generators’. It prescribes three, five, or seven relative line lengths that are known to yield convincing rags.
--	---

```
\begin{smoothraggedrightpar}[\option]...
...
\end{smoothraggedrightpar}
```

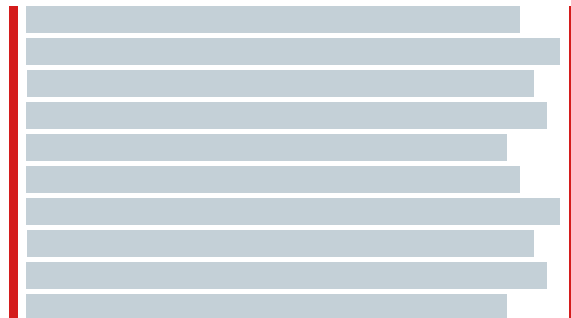
Typeset a single paragraph with a given rag width—set with the global parameter `smoothraggedrightwidth`—of the right margin, where with “rag width” we mean the length difference of the longest and the shortest lines. Select the generator with `smoothraggedrightgenerator`. The line lengths equally divide the ragged margin, i. e., they form an arithmetic sequence based on the generator size.

Option `linewidth=<dim>` overrides the length of the longest line with `<dim>`. The default line width is the current `\linewidth`.

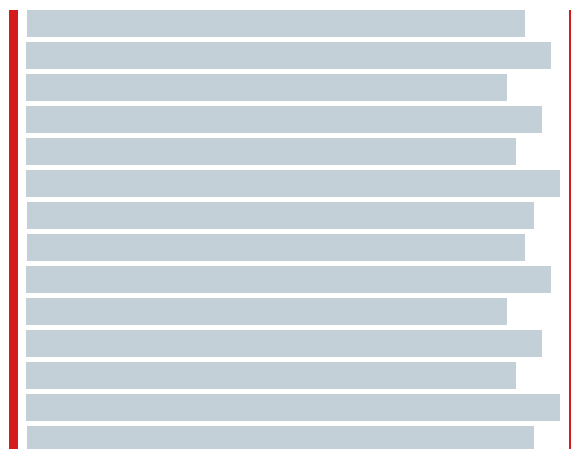
- The triplet generator repeats a *short line–long line–middle-length line* sequence. Shown below are two complete cycles.



- The quintuplet generator varies the triplet theme and avoids the ‘ladder’ pattern of lines 2–3–4 (or, if numbered by cycle: 1.2–1.3–2.1). Shown here are two cycles.



- The septuplet generator uses a permutation that appears ‘random’; at the very least, it conceals the cycle boundaries effectively. Shown here are two of them.



The graphical representations of the smooth-ragged-right-paragraph shapes shown above are based on the actual values used by the environments and not made up only for pretty figures.

`smoothraggedright`
(*env.*) Environment `smoothraggedright` is the multi-paragraph version of `smoothraggedrightpar`. It takes the same optional arguments.

```
\begin{smoothraggedright}[\langle option \rangle...]
...
\end{smoothraggedright}
```

`\smoothraggedrightfuzzfactor` The environments `smoothraggedright` and `smoothraggedrightpar` add glue to every line width⁴² to achieve a more convincing “ragged appearance” and to reduce the number of overfull lines. The algorithm divides the smooth margin into 3, 5, or 7 parts depending on the chosen `\smoothraggedrightgenerator` (see below).

```
\renewcommand*{\smoothraggedrightfuzzfactor}{\langle factor \rangle}
```

The `\smoothraggedrightfuzzfactor` is the amount of glue of each line expressed as a multiple of the distance between the division points. The default `\langle factor \rangle` of 1.0 means that glue is added just enough to prevent the lines from overlapping (assuming justification is feasible).

`\smoothraggedrightgenerator` This macro selects the generator to use.

```
\renewcommand*{\smoothraggedrightgenerator}{\langle generator \rangle}
```

Valid `\langle generator \rangle` names are

- triplet,
- quintuplet, and
- septuplet.

The default generator is triplet.

`\smoothraggedrightleftskip`
(*skip*) Value for `leftskip` to pass to the generator.

```
\smoothraggedrightleftskip=\langle dim \rangle
```

Default: 0 pt.

`\smoothraggedrightparindent`
(*skip*) Value for `parindent` to pass to the generator.

```
\smoothraggedrightparindent=\langle dim \rangle
```

Default: 0 pt.

⁴² The shortest line only gets stretchability, the longest receives only shrinkability. All other lines are both stretchable and shrinkable.

`\smoothraggedrightragwidth` Value for the width of the ragged right margin.
(*skip*)

```
\smoothraggedrightragwidth=<dim>
```

Default: 2 em.⁴³

Use Cases

Replacement for `\RaggedRight` [27]. ¶ A design alternative to fully justified paragraphs when used with a small rag width. ■

Anticipated Changes & Possible Extensions

Translate the code to `l3galley` which is part of the `l3experimental` package [19]. Galley code is supposed to work inside of lists, too. ■

Sample Smooth-Ragged-Right Paragraph at Full Text-Width

Throughout this manual, we have demonstrated how `smoothraggedright` environments work for very narrow columns—namely, inside the document’s margins: all marginal notes were typeset with `smoothraggedright` environments (quintuplet generator, 1.5 em rag width, at footnote size in addition to using environments `slightly sloppy` and `loosespacing`). In this paragraph, we utilize it with the quintuplet generator and a rag width of only 7 pt in the body that is 355 pt wide and averages around twelve words per line. There is much more glue to adapt to the line ends, and thus the desired rag is achieved more easily. The sloppiness is minimal; that is, `\fussy` is in effect, and character protrusion into the margins is disabled. A limitation of the current implementation is that it is ineffective inside of lists. Therefore, this paragraph has not been wrapped inside of an ‘Example’ because all of our examples are implemented as lists.

43 This is also the default rag width of `ragged2e`’s `\RaggedRight` [27].

4 Limitations and Known Problems

Here is a list of some known problems with typog.

Interference with KOMA classes [18] and package pdfcomment [16] if option pdf-substitutes is passed.

The following macros or just loading typog with option pdfsubstitutes can cause weird formatting in unexpected places.

<code>\Adjustedlabelitemi</code>	32	<code>\itcorr</code>	17
<code>\Adjustedlabelitemii</code>	32	<code>\kernedhyphen</code>	19
<code>\Adjustedlabelitemiii</code>	32	<code>\kernedslash</code>	18
<code>\Adjustedlabelitemiv</code>	32	<code>\leftkernedhyphen</code>	19
<code>\adjustedlabelitemi</code>	32	<code>\leftspacedemdash</code>	23
<code>\adjustedlabelitemii</code>	32	<code>\leftspacedendash</code>	21
<code>\adjustedlabelitemiii</code>	32	<code>\nolig</code>	16
<code>\adjustedlabelitemiv</code>	32	<code>\rightkernedhyphen</code>	19
<code>\breakpoint</code>	14	<code>\rightspacedemdash</code>	23
<code>\capitalcolon</code>	27	<code>\rightspacedendash</code>	21
<code>\capitalemdash</code>	25	<code>\Singleguillemetleft</code>	28
<code>\capitalendash</code>	25	<code>\Singleguillemetright</code>	28
<code>\capitalhyphen</code>	24	<code>\singleguillemetleft</code>	27
<code>\capitaltimes</code>	27	<code>\singleguillemetright</code>	27
<code>\Doubleguillemetleft</code>	28	<code>\spacedcapitalemdash</code>	25
<code>\Doubleguillemetright</code>	28	<code>\spacedcapitalendash</code>	25
<code>\doubleguillemetleft</code>	27	<code>\spaceddash</code>	21
<code>\doubleguillemetright</code>	27	<code>\spacedemdash</code>	23
<code>\figuredash</code>	26	<code>\spacedendash</code>	21

The simplest workaround is not to use pdfsubstitutes. This is the default anyhow.

The sentence uses `\spaced-`
endashes.

Use `\texorpdfstring` – which is part of the package hyperref [25] – in the relevant sectioning macros to restore string substitution on a case-by-case basis.

Example

```
\section{\(CP\) \texorpdfstring{\capitalhyphen}{-}%
Invariance}
```



5 Other Packages for Fine L^AT_EX Typography

Many other packages help produce better output from L^AT_EX. Here is a list—in alphabetical order—of the ones the author considers particularly valuable.

- | | |
|-----------|---|
| enumitem | Flexible and consistent definition of all basic L ^A T _E X list types, including inline lists [5]. |
| geometry | Powerful and sophisticated configuration of page layout [31]. Best used together with layout [20] to visualize the page geometries. |
| hyphenat | Provides hyphens that do not inhibit further auto-hyphenation of compound words [39].

Also see Section 3.3 (pp. 11–15) for this package’s macros and environments that are related to hyphenation. |
| microtype | Fine control of spacing, tracking, sidebearings, character protrusion into the margins, font expansion, and more [26].

See Section 3.12 (pp. 46–48) for a front end to microtype provided by this package and also the discussion in reference 15. |
| ragged2e | Improved versions of the raggedleft, raggedright, and center environments [27]. |
| setspace | Consistently sets the line spacing of a document (i. e., controls \baselineskip) [30].

See Section 3.16 (pp. 55–57) for a front end to setspace provided by this package. |

A typog-grep

The companion program **typog-grep** for analyzing the output of **typoginspect** and **typoginspectpar** has its own manual page. We reproduce it here for completeness of the documentation.

A.1 Name

typog-grep - specialized grep for typog-inspect elements in L^AT_EX log files

A.2 Synopsis

```
typog-grep -a | --all | --any [OPTION...] LOG-FILE...
```

```
typog-grep [OPTION...] REGEXP LOG-FILE...
```

The first form, “discovery mode”, shows all *IDs* of

```
<typog-inspect id="ID" ...>
```

elements in *LOG-FILE*.

The second form shows the contents, *LOG-DATA*, of the elements

```
<typog-inspect id="ID" ...>
LOG-DATA
</typog-inspect>
```

whose *IDs* match *REGEXP* in *LOG-FILE*.

If no *LOG-FILE* is given read from *stdin*. The filename – is synonymous to *stdin*.

A.3 Description

typog-grep is a tailored post-processor for L^AT_EX *log* files and the **typoginspect** environment as provided by the L^AT_EX package **typog**. It shares more with the venerable **sgrep** than with POSIX **grep**.

In the L^AT_EX source file the user brackets her text or code in a **typoginspect** environment:

```
\begin{typoginspect}{ID}
TEXT-OR-CODE-TO-INVESTIGATE
\end{typoginspect}
```

where *ID* is used to identify one or more bracketed snippets. *ID* does not have to be unique. The *REGEXP* mechanism makes it easy to select groups of related *IDs* if they are named accordingly.

In *LOG-FILE* the result of the environment shows up, packed with tracing information, as

```
<typog-inspect id="ID" job="JOB-NAME" line="LINE-NUMBER"
page="PAGE-NUMBER">
LOG-DATA
</typog-inspect>
```

where all the capital-letter sequences are meta-variables and in particular *JOB-NAME* is the expansion of `\jobname`, *LINE-NUMBER* is the L^AT_EX source file line number of the beginning of the `typoginspect` environment, and *PAGE-NUMBER* is the page where the output of *TEXT-OR-CODE-TO-INVESTIGATE* occurs.

typog-grep reveals the contents of *LOG-FILE* between `<typog-inspect id="ID" ...>` and `</typog-inspect>` excluding the XML-tags themselves. Access the *JOB-NAME*, *LINE-NUMBER*, and *PAGE-NUMBER* with the commandline options **--job-name**, **--line-number**, and **--page-number**. Use **--id** to show the name of the *IDs* that matched *REGEXP*.

`typoginspect` environments can be nested. **typog-grep** respects the nesting, i.e., if the *ID* of the nested environment does not match *REGEXP* it will not be included in the program's output.

A.4 Options

The list of options is sorted by the names of the long options.

-a, --all, --any

ID-discovery mode: Discover all `typog-inspect` elements independent of any matching patterns and print their *IDs*. The results are printed in their order of occurrence in the *LOG-FILES*. Pipe the output into **sort** to get alphabetically ordered *IDs*.

Augment with options **--job-name**, **--line-number**, **--log-line-number**, or **--page-number** for more information.

--color, colour WHEN

Colorize specific log contents for the matching *IDs*. The argument *WHEN* determines when to apply color: **always**, **never**, or **auto**. The setting **auto** checks whether standard output has been redirected. This is the default.

-C, --config *KEY=VALUE[:KEY=VALUE[:...]]*

Set one or more configuration *KEY* to *VALUE* pairs. See section A.5 for a description of all available configuration items. Use option **--show-config** to display the default configuration.

--debug

Turn on debug output on *stderr*.

-E, --encoding *ENCODING*

Set the *ENCODING* of *LOG-FILE* for the translation to UTF-8. The default is unset.

Use this option to get rid of pesky "<HEX-DIGITS>" escapes on UTF-8 terminals. See option **--show-encodings** for the known encodings and Encode::Supported for a summary of all encodings. See also section A.5.2.

Apply **iconv** (POSIX) or **recode** (GNU) on *LOG-FILE* before this tool to avoid having to use option **--encoding**.

-h, --help

Display brief help then exit.

-i, --[no-]id

Print the actual ID-name that matched *REGEXP*. Control the appearance of the matching *ID* with configuration item *id-heading*.

-y, --[no-]ignore-case

Match *IDs* while ignoring case distinctions in patterns and data.

-j, --[no-]job-name

Print the `\jobname` that **latex** associated with the input file.

-n, --[no-]line-number

Print the line number where the `typoginspect` environment was encountered in the **L^AT_EX** source file.

-N, --[no-]log-line-number

Print the line number of the *log*-file where the current line was encountered.

-p, --[no-]page-number

Print page number where the contents of the `typoginspect` environment starts in the typeset document.

-P, --[no-]pager

Redirect output from *stdout* to the configured pager.

--show-config

Show the default configuration and exit.

--show-encodings

Show all known encodings and exit.

-V, --version

Show version information and exit.

-w, --[no-]word-regexp

Match only whole words.

A.5 Configuration**id-format=FORMAT**

Control the *FORMAT* for printing matching ids in inline-mode, where *FORMAT* is passed to Perl's `printf`. Default: `%s`.

id-heading=0|1

Choose between printing the matching *IDs* with option **--id**: Inline (0) or heading before the matching data (1). Default: 0.

id-heading-format=FORMAT

Control the *FORMAT* for printing matching *IDs* in heading-mode, where *FORMAT* is passed to Perl's `printf`. Default: `--> %s <--`.

id-indent=INDENT

Indentation of nested typog-inspect tags. Only used in “discovery mode” (first form), i.e., if **--all** is active. Default: 8.

id-max-length=MAXIMUM-LENGTH

Set the maximum length of a matching *ID* for printing. If a matching *ID* exceeds this length it will be truncated and the last three characters (short of *MAXIMUM-LENGTH*) will be replaced by dots. Default: 40.

line-number-format=FORMAT

Control the *FORMAT* for printing TeX source line numbers, where *FORMAT* is passed to Perl's `printf`. Default: `%5d`.

log-line-number-format=FORMAT

Control the *FORMAT* for printing log line numbers, where *FORMAT* is passed to Perl's `printf`. Default: `%6d`.

page-number-format=FORMAT

Control the *FORMAT* for printing page numbers, where *FORMAT* is passed to Perl's `printf`. Default: `[%3d]`.

pager=PAGER

Name of pager application to pipe output into if run with option **--pager**. Default: `less`.

`pager-flags=FLAGS`

Pass *FLAGS* to *PAGER*. Default: `--quit-if-one-screen`.

Color Configuration

For the syntax of the color specifications consult the manual page of `Term::ANSIColor(pm)`.

`file-header-color`

Color of the filename header.

`fill-state-color`

Color of the messages that report “Underfull hbox” or “Overfull hbox”.

`first-vbox-color`

Color of the first vbox on a page.

`font-spec-color`

Color of font specifications.

`horizontal-break-candidate-color`

Color of lines with horizontal-breakpoint candidates @.

`horizontal-breakpoint-color`

Color of lines with horizontal breakpoints @@.

`id-color`

Color of matching *IDs* when printed inline.

`id-heading-color`

Color of matching *IDs* when printed in heading form.

`line-break-pass-color`

Color of the lines showing which pass (e.g., `@firstpass`) of the line-breaking algorithm is active.

`line-number-color`

Color of TeX-source-file line numbers.

`log-line-number-color`

Color of log-file line numbers.

`math-color`

Color used for math expressions including their font specs.

`page-number-color`

Color of page numbers of the final output.

`tightness-color`

Color of lines with Tight/Loose hbox reports.

`vertical-breakpoint-color`

Color of possible vertical breakpoints.

A.5.1 Brief summary of colors and attributes

Foreground Color

black, red, green, yellow, blue, magenta, cyan, white,
Prefix with `bright_` for high-intensity or bold foreground.

Foreground Grey

grey0, ..., grey23

Background Color

on_black, on_red, on_green, on_yellow, on_blue, on_magenta, on_cyan,
on_white

Replace `on_` with `on_bright_` for high-intensity or bold background.

Background Grey

on_grey0, ..., on_grey23

Text Attribute

bold, dark, italic, underline, reverse

A.5.2 Some common encodings

The following list shows some encodings that are suitable for option `--encoding`.

Latin-1, Western European

iso-8859-1, cp850, cp860, cp1252

Latin-2, Central European

iso-8859-2, cp852, cp1250

Latin-3, South European (Esperanto, Maltese)

iso-8859-3

Latin-4, North European (Baltics)

iso-8859-4

Cyrillics

iso-8859-5, cp855, cp866 (Ukrainian), cp1251

Arabic

iso-8859-6, cp864, cp1006 (Farsi), cp1256

Greek

iso-8859-7, cp737, cp1253

Hebrew

iso-8859-8, cp862, cp1255

Turkish

iso-8859-9, cp857, cp1254

Nordic

iso-8859-10, cp865, cp861 (Icelandic)

Thai

iso-8859-11, cp874

Baltic

iso-8859-13, cp775, cp1257

Celtic

iso-8859-14

Latin-9 (sometimes called Latin0)

iso-8859-15

Latin-10

iso-8859-16

A.6 Exit status

The exit status is 0 if at least one *ID* matched *REGEXP*, 1 if no *ID* matched *REGEXP*, and 2 if an error occurred.

A.7 Caveats

The end tag `</typog-inspect>` sometimes gets placed too early in the output and the trace *seems* truncated. However, L^AT_EX reliably logs the requested the trace information, but the write operations for trace data and the code which is used to print the end tag are not synchronized.

A.8 See also

`grep(1)`, `printf(3)`, `Encode::Supported(pm)`, `Term::ANSIColor(pm)`

B e-TeX: Breaking Paragraphs into Lines

This is an excerpt from the e-TeX manual [7], Sec. 3.8, “Breaking Paragraphs into Lines” that describes the `\lastlinefit` algorithm. Package `typog` warps `\lastlinefit` in environment `lastlinefitpar`, which was introduced in Sec. 3.10.5 on p. 42.

Traditional typesetting with lead type used to adjust (stretch or shrink) the interword spaces in the last line of a paragraph by the same amount as those in the preceding line. With TeX the last line is, however, usually typeset at its natural width due to infinitely stretchable `\parfillskip` glue. e-TeX allows interpolation between these two extremes by specifying a suitable value for `\lastlinefit`. For a value of 0 or less, e-TeX behaves as TeX, values from 1 to 1000 indicate a glue adjustment fraction f times 1000, values above 1000 are interpreted as $f = 1$.

The new algorithm is used only if

1. `\lastlinefit` is positive;
2. `\parfillskip` has infinite stretchability; and
3. the stretchability of `\leftskip` plus `\rightskip` is finite.⁴⁴

Thus the last line of a paragraph would normally be typeset at its natural width and the stretchability of `\parfillskip` glue would be used to achieve the desired line width. The algorithm proceeds as usual, considering all possible sequences of feasible break points and accumulating demerits for the stretching or shrinking of lines as well as for visually incompatible lines. When a candidate for the last line has been reached, the following conditions are tested:

5. the previous line was not “infinitely bad” and was stretched with positive finite stretchability or was shrunk with positive shrinkability;
6. the last line has infinite stretchability entirely due to `\parfillskip` glue;
7. if the previous line was stretched or shrunk the last line has positive finite stretchability or shrinkability.

If all three conditions are satisfied, a glue adjustment factor of f times that of the preceding line will be applied to the relevant stretch or shrink components of all glue nodes in the last line, and the corresponding demerits are computed. (The last line will, however, not be stretched beyond the desired line width.)

When all possible candidates for the last line of the paragraph have been examined, the one having fewest accumulated demerits is chosen. If e-TeX’s modified algorithm was applied to that last line, the actual stretching or shrinking is achieved by suitably modifying the `\parfillskip` glue node.

⁴⁴ As usual for parameters influencing TeX’s line-breaking algorithm, the values current at the end of the (partial) paragraph are used.

This paragraph benefits from being enclosed in a `covernextindentpar` environment.

All computations described so far are performed with machine-independent integer arithmetic. Note, however, that the actual stretching requires machine-dependent floating point arithmetic. Therefore, when a paragraph is interrupted by a displayed equation and the line preceding the display is subject to the adjustment just described, the display will in general be preceded by `\abovedisplayskip` and not by `\abovedisplayshortskip` glue.

Section 3.8 of the e-TeX manual closes with a description of the generalizations of `\clubpenalties`, `\displaywidowpenalties`, `\interlinepenalties`, and `\widowpenalties`. See also Sec. 3.14, “Vertically Partially-Tied Paragraphs”, p. 50 in this manual.

C Package Code

This is the “Reference Manual” section of the documentation where we describe the package’s code and explain its implementation details.

```

1 %<*package>
2 \NeedsTeXFormat{LaTeX2e}[2005/12/01]
3 \ProvidesPackage{typog}
4           [2026/03/21 v0.6 TypoGraphic extensions]
5
6 \RequirePackage{etoolbox}
7 \RequirePackage{everyhook}
8 \RequirePackage{xkeyval}
9

```

Declarations of Lengths, Skips, etc.

\typog@TYPOG Define a macro that unequivocally identifies this very package.

```
10 \newcommand*{\typog@TYPOG}{}

```

\typoglogo We have our own, low-key logo.

```
11 \newcommand*{\typoglogo}{\textsf{T}\kern-.12em\textsl{y}poG}}

```

\iftypog@debug Our switch for debug information.

```
12 \newif\iftypog@debug

```

\typog@typeout Our information printer. Just adds a prefix so that we can tease apart the *log* later.

```
13 \newcommand*{\typog@typeout}[1]{\typeout{typog: #1}}
14

```

\typog@typeout Our debug information printer.

```
15 \newcommand*{\typog@debug@typeout}[1]
16           {\iftypog@debug\typog@typeout{#1}\fi}
17

```

typog@@iteration (*counter*) We want our own counter (currently for keeping track of iterations) that does not get trampled underfoot too easily.

```
18 \newcounter{typog@@iteration}
19

```

\typog@trim@spaces Pull \tl_trim_spaces into the ‘classic’ namespace.

```
20 \ExplSyntaxOn
21 \let\typog@trim@spaces=\tl_trim_spaces:o
22 \ExplSyntaxOff
23

```

typog@setup@pdfsubstitutes Request the setup of PDF-substitutes.

```
24 \newif\iftypog@setup@pdfsubstitutes

```

`\typog@register@pdfsubstitute` We often need to register (simple) substitute commands suitable for PDF bookmarks. This is a convenient abbreviation for that task.

```

25 \newcommand*{\typog@register@pdfsubstitute}[1]{%
26   \iftypog@setup@pdfsubstitutes
27     \AtBeginDocument{%
28       \ifdefined\pdfstringdefDisableCommands
29         \pdfstringdefDisableCommands{#1}%
30       \fi}
31   \fi}
32
```

Some functionality depends on package `microtype`. To complicate matters for certain setup operations, e. g., `\SetExpansion`, `microtype` must be loaded *before* package `typog`, a fact that we encode in `\iftypog@microtype@preloaded`.

`\typog@microtype@preloaded`

```

33 \newif\iftypog@microtype@preloaded
34
```

`\typog@require@preloaded@microtype` It is easy to determine whether `microtype` has been sourced. We raise to the occasion and define a pair of check macros which simplify the test for the correct `microtype` load state.

```

35 \ifdefined\MT@MT
36   \typog@typeout{package microtype preloaded}%
37   \typog@microtype@preloadedtrue
38   \def\typog@require@preloaded@microtype{\relax}
39 \else
40   \typog@microtype@preloadedfalse
41   \def\typog@require@preloaded@microtype
42     {\PackageError{typog}%
43       {package microtype not (pre-)loaded}%
44       {package microtype must be loaded before pack-
45         age typog}}
46 \fi
47
```

`\iftypog@microtype@loaded`

```

47 \newif\iftypog@microtype@loaded
48
```

`\typog@require@microtype` This code duplicates `\typog@require@preloaded@microtype`; the only difference is that we call the test *after* the preamble was processed.

```

49 \AtBeginDocument{
50   \ifdefined\MT@MT
51     \typog@typeout{package microtype loaded}%
52     \typog@microtype@loadedtrue
53     \def\typog@require@microtype{\relax}
54   \else
55     \typog@microtype@loadedfalse
56     \def\typog@require@microtype
57       {\PackageError{typog}%

```

```

58             {package microtype not loaded}%
59             {require package microtype before package ty-
        pog}}}%
60     \fi
61 }
62

```

Our own state ...

onfig@mathitaliccorrection

```

63 \newmuskip\typog@config@mathitaliccorrection
64

```

Space around em-dash.

g@config@emdashspace (*dimen*)

```

65 \newlength{\typog@config@emdashspace}

```

Space around en-dash.

g@config@endashspace (*dimen*)

```

66 \newlength{\typog@config@endashspace}

```

Actual \labelitem⟨*N*⟩ corrections.

og@adjust@labelitemi (*dimen*)

```

67 \newdimen{\typog@adjust@labelitemi}

```

g@adjust@labelitemii (*dimen*)

```

68 \newdimen{\typog@adjust@labelitemii}

```

@adjust@labelitemiii (*dimen*)

```

69 \newdimen{\typog@adjust@labelitemiii}

```

g@adjust@labelitemiv (*dimen*)

```

70 \newdimen{\typog@adjust@labelitemiv}

```

Configuration constants for \labelitem⟨*N*⟩ corrections.

lowercase@labelitemi (*dimen*)

```

71 \newdimen{\typog@adjust@lowercase@labelitemi}

```

owercase@labelitemii (*dimen*)

```

72 \newdimen{\typog@adjust@lowercase@labelitemii}

```

wercase@labelitemiii (*dimen*)

```

73 \newdimen{\typog@adjust@lowercase@labelitemiii}

```

```

lowercase@labelitemiv (dimen)
74 \newdimen{\typog@adjust@lowercase@labelitemiv}

uppercase@labelitemi (dimen)
75 \newdimen{\typog@adjust@uppercase@labelitemi}

ppercase@labelitemii (dimen)
76 \newdimen{\typog@adjust@uppercase@labelitemii}

percase@labelitemiii (dimen)
77 \newdimen{\typog@adjust@uppercase@labelitemiii}

ppercase@labelitemiv (dimen)
78 \newdimen{\typog@adjust@uppercase@labelitemiv}
79

    Other lengths ...

onfig@textitaliccorrection
80 \newlength{\typog@config@textitaliccorrection}

\typog@config@ligaturekern
81 \newlength{\typog@config@ligaturekern}

\typog@config@lowerslash
82 \newlength{\typog@config@lowerslash}

og@config@raisecapitaldash
83 \newlength{\typog@config@raisecapitaldash}

fig@raisecapitalguillemets
84 \newlength{\typog@config@raisecapitalguillemets}

@config@raisecapitalhyphen
85 \newlength{\typog@config@raisecapitalhyphen}

g@config@raisecapitaltimes
86 \newlength{\typog@config@raisecapitaltimes}

pog@config@raiseguillemets
87 \newlength{\typog@config@raiseguillemets}

pog@config@raisefiguredash
88 \newlength{\typog@config@raisefiguredash}

\typog@config@slashkern
89 \newlength{\typog@config@slashkern}

```

```

\typog@config@breakpenalty
    90 \newcommand*{\typog@config@breakpenalty}{\exhyphenpenalty}

\typog@dim@unit We would like to express the argument values for example of \kernedhyphen*
                and \kernedhyphen as multiples of a thousandth of an em. Therefore, we define
                a dimen as “base unit” which simplifies matters greatly.
    91 \newlength{\typog@dim@unit}
    92 \setlength{\typog@dim@unit}{.001em}

g@config@trackingttspacing
    93 \newcommand*{\typog@config@trackingttspacing}{300, 90, 60}

\typog@default@shrink@i The default configuration for shrink values.
    94 \newcommand*{\typog@default@shrink@i}{5}

\typog@default@shrink@ii
    95 \newcommand*{\typog@default@shrink@ii}{10}

\typog@default@shrink@iii
    96 \newcommand*{\typog@default@shrink@iii}{20}

\typog@shrink@i Configurable shrink values. Initialized from the typog@default@shrink@ set.
    97 \newcommand*{\typog@shrink@i}{}

\typog@shrink@ii
    98 \newcommand*{\typog@shrink@ii}{}

\typog@shrink@iii
    99 \newcommand*{\typog@shrink@iii}{}

\typog@default@stretch@i The default configuration for stretch values.
    100 \newcommand*{\typog@default@stretch@i}{5}

\typog@default@stretch@ii
    101 \newcommand*{\typog@default@stretch@ii}{10}

\typog@default@stretch@iii
    102 \newcommand*{\typog@default@stretch@iii}{20}

\typog@stretch@i Configurable stretch values. Initialized from the typog@default@stretch set.
    103 \newcommand*{\typog@stretch@i}{}

\typog@stretch@ii
    104 \newcommand*{\typog@stretch@ii}{}

\typog@stretch@iii
    105 \newcommand*{\typog@stretch@iii}{}

```

C.1 Setup and Reconfiguration

`typogsetup` (*env.*) An empty argument list resets all initialized values to their defaults.

```

106 \NewDocumentEnvironment{typogsetup}{m}
107   {\def\typog@@arg{#1}%
108    \ifx\typog@@arg\empty
109      \typog@initialize@options
110    \else
111      \setkeys{typog}{#1}%
112    \fi
113    \ignorespaces}
114   {\ignorespacesafterend}

```

`\typogget` Access the package's configuration (name-)space.

```

115 \NewDocumentCommand{\typogget}{m}
116     {\csname typog@config@#1\endcsname}
117

```

`\typoggetnth` Access the n^{th} element of a comma-separated, list-like configuration item's value.

```

118 \ExplSyntaxOn
119 \cs_generate_variant:Nn \seq_set_split:Nnn {Nne}
120 \cs_new:Npn \typog_get_nth_csname:cnn #1#2#3
121   {
122     \seq_set_split:Nne \l_tmpa_seq {,} {\cs:w typog@config@#2 \cs_end:}
123     \cs_gset:cpn {#1} {\seq_item:Nn \l_tmpa_seq {#3}}
124   }
125 \cs_new:Npn \typog_get_nth_dimen:nnn #1#2#3
126   {
127     \seq_set_split:Nne \l_tmpa_seq {,} {\cs:w typog@config@#2 \cs_end:}
128     \dim_set:Nn {#1} {\seq_item:Nn \l_tmpa_seq {#3}}
129   }
130 \NewDocumentCommand{\typoggetnth}{m m m}{
131   \token_if_dim_register:NTF {#1}
132   {
133     \typog_get_nth_dimen:nnn {#1} {#2} {#3}
134   }
135   {
136     \typog_get_nth_csname:cnn {#1} {#2} {#3}
137   }
138 }
139 \ExplSyntaxOff
140

```


C.2 Information

`\typog@round@dim@to@tenths`

```

141 \ExplSyntaxOn
142 \newcommand*{\typog@round@dim@to@tenths}[1]
143   {\fp_to_decimal:n {round(10 * \dim_to_fp:n{#1} / 1\p@) / 10}}
144 \ExplSyntaxOff
145

```

`\typog@formatsizeinfo` Arguments 1 and 2 are the font size and the line spacing. The third parameter adds (decorative) units to both numbers.

```

146 \newcommand*{\typog@formatsizeinfo}[3]
147   {#1#3\kernedslash #2#3}
148

```

`\fontsizeinfo` All macros defined inside of `\fontsizeinfo` must be global because the call can occur inside of a group.

The two `\edefs` at the beginning capture the desired values at the point where the macro *is called*. The user-macro is tricky for we need a global macro with a constructed name and an associated starred version.

Implementation Note

`\@ifstar` caused too many problems which `\@ifnextchar` in combination with `\@gobble` avoid.

```

149 \NewDocumentCommand{\fontsizeinfo}{s m}
150   {\global\expandafter\edef\csname typog@fontsize@#2\endcsname
151     {\typog@round@dim@to@tenths{\fontdimen6\font}}}%
152   \global\expandafter\edef\csname typog@linespacing@#2\endcsname
153     {\typog@round@dim@to@tenths{\baselineskip}}}%
154   \protected\expandafter\gdef\csname #2\endcsname
155     {\@ifnextchar*{\typog@formatsizeinfo
156                   {\csname typog@fontsize@#2\endcsname}%
157                   {\csname typog@linespacing@#2\endcsname}%
158                   }% no unit
159                   \ignorespaces % eat spaces after star
160                   \@gobble} % consume the star itself
161   {\typog@formatsizeinfo
162     {\csname typog@fontsize@#2\endcsname}%
163     {\csname typog@linespacing@#2\endcsname}%
164     {\,pt}% decorative unit 'pt'
165   }}
166

```

`@default@inspect@id@prefix` Id-prefix for those `typoinspect` environments that were not identified by the user.

```

167 \newcommand*{\typog@default@inspect@id@prefix}{a-}

```

`typog@inspect@count` Counter to supply unique number and in turn *<id>* for those `typoinspect` environments that were not identified by the user.

```

168 \newcounter{typog@inspect@count}
169

```

`typoginspect` (*env.*)

```

170 \define@key[typog]{typoginspect}{tracingboxes}[\maxdimen]%
171     {\def\typog@@typoginspect@tracingboxes{#1}}
172 \NewDocumentEnvironment{typoginspect}{0}{ m}
173     {\def\typog@@typoginspect@tracingboxes{\m@ne}%
174     \setkeys[typog]{typoginspect}{#1}%

```

If the user does not supply an $\langle id \rangle$, we fall back to our own counter and construct a hopefully unique $\langle id \rangle$ from that.

```

175     \edef\typog@@arg{#2}%
176     \ifx\typog@@arg\empty
177         \stepcounter{typog@inspect@count}%
178         \edef\typog@@id{\typog@default@inspect@id@prefix
179                         \arabic{typog@inspect@count}}%
180     \else
181         \edef\typog@@id{\typog@trim@spaces{\typog@@arg}}%
182     \fi
183     \typeout{<typog-inspect\space
184             id="\typog@@id"\space
185             job="\jobname"\space
186             line="\the\inputlineno"\space
187             page="\the\value{page}">}%

```

Set both badness thresholds to absurdly low values as to activate T_EX's reports.

```

188     \hbadness=\m@ne
189     \vbadness=\m@ne

```

Carefully select the tracing functionality we want (to improve our typography). Too much trace data distracts and the user always can turn on more tracing at the beginning of the environment.

```

190     \tracingnone
191     \tracingpages=\@ne
192     \tracingparagraphs=\@ne
193     \showboxbreadth=\typog@@typoginspect@tracingboxes
194     \showboxdepth=\typog@@typoginspect@tracingboxes}
195     {\typeout{</typog-inspect>}}%
196     \ignorespacesafterend}

```

`typoginspectpar` (*env.*) Companion environment to `typoginspect` which adds a `\par` before the end of the group.

```

197 \NewDocumentEnvironment{typoginspectpar}{m}
198     {\typoginspect{#1}}
199     {\par\endtypoginspect}
200

```

C.3 Hyphenation

`\typog@allowhyphenation` Re-enable automatic hyphenation.

The same or almost the same implementation can be found in babel as macro `\bbl@allowhyphens` and hyphenat as macro `\prw@zbreak`.

```

201 \newcommand*{\typog@allowhyphenation}
202   {\ifvmode
203     \relax
204   \else
205     \nobreak
206     \hskip\z@skip
207   \fi}
208

```

`\allowhyphenation` Define a user-visible alias unless the name is already used.

```

209 \unless\ifdefined\allowhyphenation
210   \let\allowhyphenation=\typog@allowhyphenation
211 \fi
212

```

`\breakpoint` The starred form inhibits hyphenation of the right-hand component.

```

213 \NewDocumentCommand{\breakpoint}{s}
214   {\discretionary{}{}{}}%
215   \IfBooleanTF{#1}%
216     {\ignorespaces}%
217     {\typog@allowhyphenation}}
218

```

PDF-substitute definition

```

219 \typog@register@pdfsubstitute{
220   \def\breakpoint#1{\if*\detokenize{#1}\ignorespaces\fi}%
221 }
222

```

`hyphenmin (env.)` No trickery here.—We use the mandatory argument for the value of `\lefthyphenmin` if the optional argument has been omitted.

```

223 \NewDocumentEnvironment{hyphenmin}{o m}
224   {\lefthyphenmin=\IfNoValueTF{#1}{#2}{#1}%
225    \righthyphenmin=#2}
226   {}
227

```

C.4 Disable/Break Ligatures

`\typog@hyphen` We define our own hyphen so the user can override the definition in a pinch.

```

228 \newcommand*{\typog@hyphen}{\char‘-}
229

```

`\nolig`

```

230 \NewDocumentCommand{\nolig}{s o}

```

```

231 {\dimen0=\IfNoValueTF{#2}
232   {\typog@config@ligaturekern}
233   {#2\typog@dim@unit}%
234   \IfBooleanTF{#1}%
235   {\kern\dimen0\ignorespaces}%
236   {\discretionary{\typog@hyphen}{\kern\dimen0}%
237    \typog@allowhyphenation
238    \IfNoValueF{#2}{\ignorespaces}}}
239

```

The PDF-ready version of `\nolig` cannot be implemented with `\futurelet`.
Doh!

```

240 \typog@register@pdfsubstitute{
241   \RenewExpandableDocumentCommand{\nolig}{s o m}{%
242     \ifx\typog@TYPOG#3\typog@TYPOG
243       \relax
244     \else
245       \ifx\relax#3\relax
246         \relax
247       \else
248         \typog@missing@argument
249       \fi
250     \fi}
251 }
252

```

C.5 Manual Italic Correction

`@itcorr@text@unconditional` Fallback italic correction for text mode.

```

253 \newcommand*{\typog@itcorr@text@unconditional}[1]
254   {\kern#1\typog@config@text@italic@correction}

```

`\typog@itcorr@text` Conditional italic correction depending on the current font's own italic correction, i. e., `\fontdimen1`.

```

255 \newcommand*{\typog@itcorr@text}[1]
256   {\def\typog@@strength{#1}%
257    \dimen0=\fontdimen1\font
258    \ifdim\dimen0=\z@
259      \typog@itcorr@text@unconditional{\typog@@strength}%
260    \else
261      \kern\typog@@strength\dimen0
262    \fi}

```

`\typog@itcorr@math` Italic correction for math mode.

```

263 \newcommand*{\typog@itcorr@math}[1]
264   {\mkern#1\typog@config@math@italic@correction}

```

`\itcorr` If the font has no italic correction we fall back to our own length. In text mode, the starred version always uses the fallback. The star is a no-op in math mode.

```

265 \NewDocumentCommand{\itcorr}{s m}
266   {\ifmmode

```

```

267 \typog@itcorr@math{#2}%
268 \else
269 \IfBooleanTF{#1}%
270 {\typog@itcorr@text{#2}}%
271 {\typog@itcorr@text@unconditional{#2}}%
272 \fi}

PDF-substitute definition
273 \typog@register@pdfsubstitute{
274 \RenewExpandableDocumentCommand{\itcorr}{s m}{}
275 }
276

```

C.6 Apply Extra Kerning

Slash

`\typog@forwardslash` We define our own forward-slash so the user can override the definition in a pinch.

```
277 \newcommand*{\typog@forwardslash}{\char‘/}
```

`\kernedslash` Macro `\kernedslash` introduces a hyphenation possibility right after the dash, whereas the starred version does not.

By the way, `\slash` expands to `‘/\penalty\exhyphenpenalty’`.

```

278 \NewDocumentCommand{\kernedslash}{s}
279 {\hspace*{\typog@config@slashkern}%
280 \raisebox{-\typog@config@lowerslash}{\typog@forwardslash}%
281 \IfBooleanTF{#1}%
282 {\hspace*{\typog@config@slashkern}\ignorespaces}%
283 {\typog@breakpoint
284 \typog@allowhyphenation
285 \hspace*{\typog@config@slashkern}}}

```

PDF-substitute definition

```

286 \typog@register@pdfsubstitute{
287 \def\kernedslash#1{\if* \detokenize{#1}/\ignorespaces\else/#1\fi}%
288 }
289

```

Hyphen

`\kernedhyphen`

```

290 \NewDocumentCommand{\kernedhyphen}{s O{0} m m}
291 {\ifmmode
292 \mspace{\muexpr(#3 mu) * 18 / 1000}%
293 \raisebox{#2\typog@dim@unit}{\mathord{-}}%
294 \mspace{\muexpr(#4 mu) * 18 / 1000}%
295 \else
296 \def\typog@@auto{*}%
297 \def\typog@@optarg{#2}%
298 \hspace*{#3\typog@dim@unit}%
299 \raisebox{\ifx\typog@@optarg\typog@@auto

```

```

300             \typog@config@raisecapitalhyphen
301             \else
302             \typog@optarg\typog@dim@unit
303             \fi}{\typog@hyphen}%
304     \hspace{#4\typog@dim@unit}%
305     \IfBooleanT{#1}{\nobreak}%
306     \fi}

PDF-substitute definition
307 \typog@register@pdfsubstitute{
308   \RenewExpandableDocumentCommand{\kernedhyphen}{s o m m}{-}
309 }

```

One-argument shorthands.

`\leftkernedhyphen` Apply kerning on the left-hand side of the hyphen only.

```

310 \NewDocumentCommand{\leftkernedhyphen}{s O{0} m}
311 { \IfBooleanTF{#1}%
312   { \kernedhyphen*{#2}{#3}{0}\ignorespaces}%
313   { \kernedhyphen[#2]{#3}{0}}}

PDF-substitute definition
314 \typog@register@pdfsubstitute{
315   \RenewExpandableDocumentCommand{\leftkernedhyphen}{s o m}{-}
316 }
317

```

`\rightkernedhyphen` Apply kerning on the right-hand side of the hyphen only.

```

318 \NewDocumentCommand{\rightkernedhyphen}{s O{0} m}
319 { \IfBooleanTF{#1}%
320   { \kernedhyphen*{#2}{0}{#3}\ignorespaces}%
321   { \kernedhyphen[#2]{0}{#3}}}

PDF-substitute definition
322 \typog@register@pdfsubstitute{
323   \RenewExpandableDocumentCommand{\rightkernedhyphen}{s o m}{-}
324 }
325

```

En-Dash and Em-Dash

`\typog@wrap@endash` Wrapper for the en-dash. The first and second arguments are used to control the line breaking; the third argument is the actual en-dash macro.

```

326 \newcommand*{\typog@wrap@endash}[3]
327 { #1\hspace{\typog@config@endashspace}%
328   #3%
329   #2\hspace{\typog@config@endashspace}}

```

`\typog@wrap@emdash` Wrapper for the em-dash. The first and second arguments are used to control the line breaking; the third argument is the actual em-dash macro.

```

330 \newcommand*{\typog@wrap@emdash}[3]

```

```

331 {\hspace{\typog@config@emdashspace}%
332  #3%
333  #2\hspace{\typog@config@emdashspace}}
334

```

`\leftspacedendash` User-land macro for the left (aka opening or introducing) spaced en-dash. The unstarred variant introduces a breakpoint *before* the en-dash.

```

335 \NewDocumentCommand{\leftspacedendash}{s o}
336 {\IfBooleanTF{#1}
337  {\IfNoValueTF{#2}
338   {\typog@wrap@endash{\nobreak}{\nobreak}
339    {\textendash}}
340   {\typog@wrap@endash{\nobreak}{\nobreak}
341    {\raisebox{#2}{\textendash}}}}
342  {\IfNoValueTF{#2}
343   {\typog@wrap@endash{\relax}{\nobreak}
344    {\textendash}}
345   {\typog@wrap@endash{\nobreak}{\nobreak}
346    {\raisebox{#2}{\textendash}}}}}%
347 \ignorespaces}
348 \let\leftspaceddash=\leftspacedendash

```

PDF-substitute definition

```

349 \typog@register@pdfsubstitute{
350  \RenewExpandableDocumentCommand{\leftspacedendash}{s o m}{%
351   \ifx\typog@TYPOG#3\typog@TYPOG
352   \textendash
353   \else
354   \ifx\relax#3\relax
355   \textendash
356   \else
357   \typog@missing@argument
358   \fi
359   \fi}
360  \let\leftspaceddash=\leftspacedendash
361 }
362

```

`\rightspacedendash` User-land macro for the right (aka closing) spaced en-dash. The unstarred variant introduces a breakpoint *after* the en-dash.

```

363 \NewDocumentCommand{\rightspacedendash}{s o}
364 {\IfBooleanTF{#1}
365  {\IfNoValueTF{#2}
366   {\typog@wrap@endash{\nobreak}{\nobreak}
367    {\textendash}}
368   {\typog@wrap@endash{\nobreak}{\nobreak}
369    {\raisebox{#2}{\textendash}}}}
370  {\IfNoValueTF{#2}
371   {\typog@wrap@endash{\nobreak}{\relax}
372    {\textendash}}
373   {\typog@wrap@endash{\nobreak}{\nobreak}

```

```

374             {\raisebox{#2}{\textendash}}}%
375 \ignorespaces}
376 \let\rightspaceddash=\rightspacedendash
377 \let\spacedendash=\rightspacedendash
378 \let\spaceddash=\rightspacedendash

    PDF-substitute definition
379 \typog@register@pdfsubstitute{
380 \RenewExpandableDocumentCommand{\rightspacedendash}{s o m}{%
381 \ifx\typog@TYPOG#3\typog@TYPOG
382 \space\textendash\space
383 \else
384 \ifx\relax#3\relax
385 \space\textendash\space
386 \else
387 \typog@missing@argument
388 \fi
389 \fi
390 \ignorespaces}
391 \let\rightspaceddash=\rightspacedendash
392 \let\spacedendash=\rightspacedendash
393 \let\spaceddash=\rightspacedendash
394 }
395

```

\swapedashskip

```

396 \NewDocumentCommand{\swapedashskip}{m}
397 {\skip0=\lastskip
398 \unskip
399 #1%
400 \hskip\skip0}
401

```

\leftspacedendash User-land macro for the spaced em-dash.

The two \mboxes turn off automatic hyphenation of the adjacent words. This ensures ‘compatibility’ with the en-dash -- .

```

402 \NewDocumentCommand{\leftspacedemdash}{s o}
403 {\mbox{}%
404 \IfBooleanTF{#1}
405 {\IfNoValueTF{#2}
406 {\typog@wrap@emdash{\nobreak}{\nobreak}
407 {\textemdash}}
408 {\typog@wrap@emdash{\nobreak}{\nobreak}
409 {\raisebox{#2}{\textemdash}}}}
410 {\IfNoValueTF{#2}
411 {\typog@wrap@emdash{\relax}{\nobreak}
412 {\textemdash}}
413 {\typog@wrap@emdash{\nobreak}{\nobreak}
414 {\raisebox{#2}{\textemdash}}}}}%
415 \mbox{}%
416 \ignorespaces}

```


PDF-substitute definition

```

417 \typog@register@pdfsubstitute{
418   \RenewExpandableDocumentCommand{\leftspacedemdash}{s o m}{%
419     \ifx\typog@TYPOG#3\typog@TYPOG
420       \textemdash
421     \else
422       \ifx\relax#3\relax
423         \textemdash
424       \else
425         \typog@missing@argument
426       \fi
427     \fi}
428 }
429

```

`\rightspacedemdash` User-land macro for the right (aka closing) spaced em-dash. The unstarred variant introduces a breakpoint *after* the em-dash.

The two `\mboxes` turn off automatic hyphenation of the adjacent words. This ensures ‘compatibility’ with the em-dash `--`.

```

430 \NewDocumentCommand{\rightspacedemdash}{s o}
431 { \mbox{}%
432   \IfBooleanTF{#1}
433     { \IfNoValueTF{#2}
434       { \typog@wrap@emdash{\nobreak}{\nobreak}
435         { \textemdash} }
436       { \typog@wrap@emdash{\nobreak}{\nobreak}
437         { \raisebox{#2}{\textemdash}}} }
438     { \IfNoValueTF{#2}
439       { \typog@wrap@emdash{\nobreak}{\relax}
440         { \textemdash} }
441       { \typog@wrap@emdash{\nobreak}{\nobreak}
442         { \raisebox{#2}{\textemdash}}} } }%
443   \mbox{}%
444   \ignorespaces}
445 \let\spacedemdash=\rightspacedemdash

```

PDF-substitute definition

```

446 \typog@register@pdfsubstitute{
447   \RenewExpandableDocumentCommand{\rightspacedemdash}{s o m}{%
448     \ifx\typog@TYPOG#3\typog@TYPOG
449       \textemdash
450     \else
451       \ifx\relax#3\relax
452         \textemdash
453       \else
454         \typog@missing@argument
455       \fi
456     \fi
457     \ignorespaces}
458   \let\spacedemdash=\rightspacedemdash
459 }
460

```

C.7 Raise Selected Characters

`\typog@breakpoint` We want our own penalty for a line break at a particular point. The predefined `\allowbreak` is too eager. A package-private, user-configurable penalty fits best.

```
461 \newcommand*{\typog@breakpoint}
462   {\penalty\typog@config@breakpenalty}
```

`\capitalhyphen` Macro `\capitalhyphen` introduces a hyphenation possibility right after the dash, whereas the starred version does not.

```
463 \NewDocumentCommand{\capitalhyphen}{s}
464   {\raisebox{\typog@config@raisecapitalhyphen}{\typog@hyphen}%
465     \IfBooleanTF{#1}%
466       {\ignorespaces}%
467       {\typog@breakpoint\typog@allowhyphenation}}}
```

The non-hyperref version's code is straightforward. The `\pdfstringdef-DisableCommands` version must be expandable and must match the other version's signature. Yikes! We exploit the fact that conditions are expandable. However, we cannot use `\typog@hyphen` in the expansion as `\char` gets in the way. So, we fall back to the least common denominator and use a bare dash.

```
468 \typog@register@pdfsubstitute{
469   \def\capitalhyphen#1{%
470     \if*\detokenize{#1}%
471       -\ignorespaces
472     \else
473       -#1%
474     \fi}
475 }
476
```

`\capitalendash` Macro `\capitalendash` introduces a hyphenation possibility right after the dash; its starred version does not.

```
477 \NewDocumentCommand{\capitalendash}{s}
478   {\raisebox{\typog@config@raisecapitaldash}{\textendash}%
479     \IfBooleanTF{#1}%
480     {\ignorespaces}%
481     {\typog@breakpoint\typog@allowhyphenation}}
482 \let\capitaldash=\capitalendash
```

PDF-substitute definition

```
483 \typog@register@pdfsubstitute{
484   \def\capitalendash#1{%
485     \if*\detokenize{#1}%
486       \textendash\ignorespaces
487     \else
488       \textendash#1%
489     \fi}
490 \let\capitaldash=\capitalendash
491 }
492
```

`\capitalemdash` Macro `\capitalemdash` introduces a hyphenation possibility right after the dash; its starred version does not.

```
493 \NewDocumentCommand{\capitalemdash}{s}
494   {\raisebox{\typog@config@raisecapitaldash}{\textemdash}%
495     \IfBooleanTF{#1}%
496       {\ignorespaces}%
497       {\typog@breakpoint\typog@allowhyphenation}}}
```

PDF-substitute definition

```
498 \typog@register@pdfsubstitute{
499   \def\capitalemdash#1{%
500     \if*\detokenize{#1}%
501       \textemdash\ignorespaces
502     \else
503       \textemdash#1%
504     \fi}
505 }
506
```

`\leftspacedcapitalendash` Thanks to our wrapper macro the definition of `\leftspacedcapitalendash` is easy to write.

```
507 \NewDocumentCommand{\leftspacedcapitalendash}{s}
508   {\IfBooleanTF{#1}%
509     {\typog@wrap@endash{\nobreak}{\nobreak}{\capitalendash*}}
510     {\typog@wrap@endash{\relax}{\nobreak}{\capitalendash}}}%
511   \ignorespaces}
512 \let\leftspacedcapitaldash=\leftspacedcapitalendash
```

PDF-substitute definition

```
513 \typog@register@pdfsubstitute{
514   \def\leftspacedcapitalendash#1{%
515     \if*\detokenize{#1}%
516       \textendash\ignorespaces
517     \else
518       \textendash#1%
519     \fi}
520 \let\leftspacedcapitaldash=\leftspacedcapitalendash
521 }
522
```

`\rightspacedcapitalendash` Thanks to our wrapper macro the definition of `\rightspacedcapitalendash` is easy to write.

```
523 \NewDocumentCommand{\rightspacedcapitalendash}{s}
524   {\IfBooleanTF{#1}%
525     {\typog@wrap@endash{\nobreak}{\nobreak}{\capitalendash*}}
526     {\typog@wrap@endash{\nobreak}{\relax}{\capitalendash}}}%
527   \ignorespaces}
528 \let\rightspacedcapitaldash=\rightspacedcapitalendash
529 \let\spacedcapitalendash=\rightspacedcapitalendash
530 \let\spacedcapitaldash=\rightspacedcapitalendash
```

PDF-substitute definition

```

531 \typog@register@pdfsubstitute{
532   \def\rightrightdash#1{%
533     \if*\detokenize{#1}%
534       \textendash\ignorespaces
535     \else
536       \textendash#1%
537     \fi}
538 \let\rightrightdash=\rightrightdash
539 \let\spacedrightdash=\rightrightdash
540 \let\spacedrightdash=\rightrightdash
541 }
542

```

`\leftspacedcapitalemdash` Thanks to our wrapper macro the definition of `\leftspacedcapitalemdash` is easy to write.

```

543 \NewDocumentCommand{\leftspacedcapitalemdash}{s}
544 {
545   {\IfBooleanTF{#1}%
546     {\typog@wrap@emdash{\nobreak}{\nobreak}{\capitalemdash*}}
547     {\typog@wrap@emdash{\relax}{\nobreak}{\capitalemdash}}}%
548   \ignorespaces}

```

PDF-substitute definition

```

548 \typog@register@pdfsubstitute{
549   \def\leftspacedcapitalemdash#1{%
550     \if*\detokenize{#1}%
551       \textemdash\ignorespaces
552     \else
553       \textemdash#1%
554     \fi}
555 }
556

```

`\rightspacedcapitalemdash` Thanks to our wrapper macro the definition of `\rightspacedcapitalemdash` is easy to write.

```

557 \NewDocumentCommand{\rightspacedcapitalemdash}{s}
558 {
559   {\IfBooleanTF{#1}%
560     {\typog@wrap@emdash{\nobreak}{\nobreak}{\capitalemdash*}}
561     {\typog@wrap@emdash{\nobreak}{\relax}{\capitalemdash}}}%
562   \ignorespaces}
563 \let\spacedrightdash=\rightspacedcapitalemdash

```

PDF-substitute definition

```

563 \typog@register@pdfsubstitute{
564   \def\rightrightdash#1{%
565     \if*\detokenize{#1}%
566       \textendash\ignorespaces
567     \else
568       \textendash#1%
569     \fi}
570 \let\spacedrightdash=\rightspacedcapitalemdash
571 }

```

572

`\figuredash` Macro `\figuredash` introduces a hyphenation possibility right after the dash; its starred version does not.

```
573 \NewDocumentCommand{\figuredash}{s}
574   {\raisebox{\typog@config@raisefiguredash}{\textendash}%
575    \IfBooleanTF{#1}%
576      {\ignorespaces}%
577      {\typog@breakpoint\typog@allowhyphenation}}}
```

PDF-substitute definition

```
578 \typog@register@pdfsubstitute{\let\figuredash=\capitaldash}
579
```

`\textcolon` We introduce a macro for the plain colon to allow the user to override it in `\capitalcolon`.

```
580 \ProvideDocumentCommand{\textcolon}{}{\char‘\:}
```

`\capitalcolon` The raise-action of this macro actually is run-of-the-mill. However, two things have to be pointed out: Firstly, the amount to raise it is determined by a length with a different name. In particular, `\capitalcolon` does not have its own length associated to it. Secondly, the box raised is not a plain colon but the expansion of `\textcolon`.

```
581 \NewDocumentCommand{\capitalcolon}{}
582   {\raisebox{\typog@config@raisefiguredash}{\textcolon}}
583
```

`\capitaltimes`

```
584 \NewDocumentCommand{\capitaltimes}{}
585   {\ifmmode
586     \mathbin{\raisebox{\typog@config@raisecapitaltimes}
587               {$\m@th\times$}}%
588   \else
589     \raisebox{\typog@config@raisecapitaltimes}{\texttimes}%
590   \fi}
```

PDF-substitute definition

```
591 \typog@register@pdfsubstitute{
592   \RenewExpandableDocumentCommand{\capitaltimes}{}{\texttimes}
593 }
594
```

`\singleguillemetleft`

```
595 \NewDocumentCommand{\singleguillemetleft}{}
596   {\typog@allowhyphenation
597    \raisebox{\typog@config@raiseguillemets}{\guilsinglleft}}
```

PDF-substitute definition

```
598 \typog@register@pdfsubstitute{
599   \let\singleguillemetleft\guilsinglleft
600 }
```

`\singleguillemetright`

```
601 \NewDocumentCommand{\singleguillemetright}{}
602 {\raisebox{\typog@config@raiseguillemets}{\guilsinglright}%
603 \typog@allowhyphenation}
    PDF-substitute definition
604 \typog@register@pdfsubstitute{
605 \let\singleguillemetright\guilsinglright
606 }
```

`\doubleguillemetleft`

```
607 \NewDocumentCommand{\doubleguillemetleft}{}
608 {\typog@allowhyphenation
609 \raisebox{\typog@config@raiseguillemets}{\guillemotleft}}
    PDF-substitute definition
610 \typog@register@pdfsubstitute{
611 \let\doubleguillemetleft\guillemotleft
612 }
```

`\doubleguillemetright`

```
613 \NewDocumentCommand{\doubleguillemetright}{}
614 {\raisebox{\typog@config@raiseguillemets}{\guillemotright}%
615 \typog@allowhyphenation}
    PDF-substitute definition
616 \typog@register@pdfsubstitute{
617 \let\doubleguillemetright\guillemotright
618 }
```

`\Singleguillemetleft`

```
619 \NewDocumentCommand{\Singleguillemetleft}{}
620 {\typog@allowhyphenation
621 \raisebox{\typog@config@raisecapitalguillemets}
622 {\guilsinglleft}}
    PDF-substitute definition
623 \typog@register@pdfsubstitute{
624 \let\Singleguillemetleft\guilsinglleft
625 }
```

`\Singleguillemetright`

```
626 \NewDocumentCommand{\Singleguillemetright}{}
627 {\raisebox{\typog@config@raisecapitalguillemets}
628 {\guilsinglright}%
629 \typog@allowhyphenation}
    PDF-substitute definition
630 \typog@register@pdfsubstitute{
631 \let\Singleguillemetright\guilsinglright
632 }
```

\Doubleguillemetleft

```
633 \NewDocumentCommand{\Doubleguillemetleft}{}
634   {\typog@allowhyphenation
635     \raisebox{\typog@config@raisecapitalguillemets}
636       {\guillemotleft}}
    PDF-substitute definition
637 \typog@register@pdfsubstitute{
638   \let\Doubleguillemetleft\guillemotleft
639 }
```

\Doubleguillemetright

```
640 \NewDocumentCommand{\Doubleguillemetright}{}
641   {\raisebox{\typog@config@raisecapitalguillemets}
642     {\guillemotright}%
643     \typog@allowhyphenation}
    PDF-substitute definition
644 \typog@register@pdfsubstitute{
645   \let\Doubleguillemetright\guillemotright
646 }
647
```

We need three lengths for three (pairs of) inverted exclamation marks and inverted question marks.

onfig@raiseinvertedmarks@i

```
648 \newlength{\typog@config@raiseinvertedmarks@i}
```

nfig@raiseinvertedmarks@ii

```
649 \newlength{\typog@config@raiseinvertedmarks@ii}
```

fig@raiseinvertedmarks@iii

```
650 \newlength{\typog@config@raiseinvertedmarks@iii}
```

And the three (pairs of) inverted exclamation marks and inverted question marks themselves. Configurable.

inverted@exclamationmark@i

```
651 \newcommand*{\typog@inverted@exclamationmark@i}
652   {\textexclamdown}
```

og@inverted@questionmark@i

```
653 \newcommand*{\typog@inverted@questionmark@i}
654   {\textquestiondown}
```

nverted@exclamationmark@ii

```
655 \newcommand*{\typog@inverted@exclamationmark@ii}
656   {\textexclamdown}
```

g@inverted@questionmark@ii

```
657 \newcommand*{\typog@inverted@questionmark@ii}
658   {\textquestiondown}
```

verted@exclamationmark@iii

```
659 \newcommand*{\typog@inverted@exclamationmark@iii}
660 {\textexclamdown}
```

@inverted@questionmark@iii

```
661 \newcommand*{\typog@inverted@questionmark@iii}
662 {\textquestiondown}
```

\typog@raise@inverted@mark

The generic ‘raise’-macro handles all interesting cases. The first argument selects the mark type and the second the associated raise-amount.

If the raise-amount is exactly zero we shift-up the horizontal box with the mark to zero its depth.

```
663 \newcommand*{\typog@raise@inverted@mark}[2]
664 {\letcs{\@tempa}{\typog@inverted@#1@romannumeral#2}%
665 \letcs{\@tempb}{\typog@config@raiseinvertedmarks@romannumeral#2}%
666 \ifdim\@tempb=\z@
667   \setbox0=\hbox{\@tempa}%
668   \dimen0=\dp0
669 \else
670   \dimen0=\@tempb
671 \fi
672 \raisebox{\dimen0}{\@tempa}}
```

The user-side macros are almost trivial now.

talinvertedexclamationmark

```
673 \NewDocumentCommand{\capitalinvertedexclamationmark}{m}
674 {\typog@raise@inverted@mark{exclamationmark}{#1}%
675 \typog@allowhyphenation}
```

PDF-substitute definition

```
676 \typog@register@pdfsubstitute{
677 \def\capitalinvertedexclamationmark#1{%
678 \csname typog@inverted@exclamationmark@romannumeral#1\endcsname
679 }
680 }
```

apitalinvertedquestionmark

```
681 \NewDocumentCommand{\capitalinvertedquestionmark}{m}
682 {\typog@raise@inverted@mark{questionmark}{#1}%
683 \typog@allowhyphenation}
```

PDF-substitute definition

```
684 \typog@register@pdfsubstitute{
685 \def\capitalinvertedquestionmark#1{%
686 \csname typog@inverted@questionmark@romannumeral#1\endcsname
687 }
688 }
```


C.8 Vertically Adjusted Label Items

`uppercase@adjust@labelitem` Handle all possible requests for uppercase label item correction. Patch `itemize` environments.

```

689 \newcommand*{\@typog@uppercase@adjust@labelitem}[1]
690   {\@typog@maybe@patch@itemize
691     \ifstrequal{#1}{*}
692       {\setlength{\typog@adjust@labelitemi}
693         {\typog@adjust@uppercase@labelitemi}%
694         \setlength{\typog@adjust@labelitemii}
695           {\typog@adjust@uppercase@labelitemii}%
696         \setlength{\typog@adjust@labelitemiii}
697           {\typog@adjust@uppercase@labelitemiii}%
698         \setlength{\typog@adjust@labelitemiv}
699           {\typog@adjust@uppercase@labelitemiv}}
700     {\ifcase #1% 0
701       \relax % outside of any itemize environment
702     \or % 1
703       \setlength{\typog@adjust@labelitemi}
704         {\typog@adjust@uppercase@labelitemi}%
705     \or % 2
706       \setlength{\typog@adjust@labelitemii}
707         {\typog@adjust@uppercase@labelitemii}%
708     \or % 3
709       \setlength{\typog@adjust@labelitemiii}
710         {\typog@adjust@uppercase@labelitemiii}%
711     \or % 4
712       \setlength{\typog@adjust@labelitemiv}
713         {\typog@adjust@uppercase@labelitemiv}%
714     \else
715       \PackageError{typog}
716         {Itemize level out of range}
717         {Valid levels are 1, 2, 3, 4, and *}
718     \fi}}
719 
```

`lowercase@adjust@labelitem` Handle all possible requests for lowercase label item correction. Patch `itemize` environments.

```

720 \newcommand*{\@typog@lowercase@adjust@labelitem}[1]
721   {\@typog@maybe@patch@itemize
722     \ifstrequal{#1}{*}
723       {\setlength{\typog@adjust@labelitemi}
724         {\typog@adjust@lowercase@labelitemi}%
725         \setlength{\typog@adjust@labelitemii}
726           {\typog@adjust@lowercase@labelitemii}%
727         \setlength{\typog@adjust@labelitemiii}
728           {\typog@adjust@lowercase@labelitemiii}%
729         \setlength{\typog@adjust@labelitemiv}
730           {\typog@adjust@lowercase@labelitemiv}}
731     {\ifcase #1% 0
732       \relax % outside of any itemize environment

```

```

733         \or % 1
734             \setlength{\typog@adjust@labelitemi}{\z@}%
735             {\typog@adjust@lowercase@labelitemi}%
736         \or % 2
737             \setlength{\typog@adjust@labelitemii}{\z@}%
738             {\typog@adjust@lowercase@labelitemii}%
739         \or % 3
740             \setlength{\typog@adjust@labelitemiii}{\z@}%
741             {\typog@adjust@lowercase@labelitemiii}%
742         \or % 4
743             \setlength{\typog@adjust@labelitemiv}{\z@}%
744             {\typog@adjust@lowercase@labelitemiv}%
745         \else
746             \PackageError{typog}
747                 {Itemize level out of range}
748                 {Valid levels are 1, 2, 3, 4, and *}
749         \fi}}
750

```

`\@typog@noadjust@labelitem` Neutralize all label item corrections. This function *does not* request patching any itemize environment!

```

751 \newcommand*{\@typog@noadjust@labelitem}[1]
752 {\ifstrequal{#1}{*}
753     {\setlength{\typog@adjust@labelitemi}{\z@}%
754     \setlength{\typog@adjust@labelitemii}{\z@}%
755     \setlength{\typog@adjust@labelitemiii}{\z@}%
756     \setlength{\typog@adjust@labelitemiv}{\z@}}
757 {\ifcase #1% 0
758     \relax % outside of any itemize environment
759     \or % 1
760         \setlength{\typog@adjust@labelitemi}{\z@}%
761     \or % 2
762         \setlength{\typog@adjust@labelitemii}{\z@}%
763     \or % 3
764         \setlength{\typog@adjust@labelitemiii}{\z@}%
765     \or % 4
766         \setlength{\typog@adjust@labelitemiv}{\z@}%
767     \else
768         \PackageError{typog}
769             {Itemize level out of range}
770             {Valid levels are 1, 2, 3, 4, and *}
771     \fi}}
772

```

`\uppercaseadjustlabelitems` User macro that handles lists and the treats the empty list specially. We wrap the code into `\AfterPreamble` because it may be called in the document's preamble where we don't know whether package `enumitem` already has been loaded and we can patch its variant of `itemize`.

```

773 \NewDocumentCommand{\uppercaseadjustlabelitems}{m}
774 {\AfterPreamble{%
775     \ifblank{#1}

```

```

776             {\@typog@uppercase@adjust@labelitem{\@itemdepth}}
777             {\forcsvlist{\@typog@uppercase@adjust@labelitem}{#1}}%
778     \ignorespaces}}
779

```

`\lowercaseadjustlabelitems` User macro that handles lists and the treats the empty list specially.

```

780 \NewDocumentCommand{\lowercaseadjustlabelitems}{m}
781   {\AfterPreamble{%
782     \ifblank{#1}
783       {\@typog@lowercase@adjust@labelitem{\@itemdepth}}
784       {\forcsvlist{\@typog@lowercase@adjust@labelitem}{#1}}%
785     \ignorespaces}}
786

```

`\noadjustlabelitems` User macro that handles lists and the treats the empty list specially.

```

787 \NewDocumentCommand{\noadjustlabelitems}{m}
788   {\ifblank{#1}
789     {\@typog@noadjust@labelitem{\@itemdepth}}
790     {\forcsvlist{\@typog@noadjust@labelitem}{#1}}%
791   \ignorespaces}
792

```

Now we get to the dirty part. All the above definitions do not get called until we hack the existing `itemize`-environments, either the one of plain L^AT_EX or the one modified by package `enumitem`.

Here comes the result of `latexdef -c article -s itemize`, which was used to derive the patch code:

```

%   \def\itemize{%
%     \ifnum \@itemdepth > \thr@@
%       \@toodeep
%     \else
%       \advance\@itemdepth\@ne
%       \edef\@itemitem{labelitem\romannumeral\the\@itemdepth}%
%       \expandafter
%       \list
%       \csname\@itemitem\endcsname
%       {\def\makelabel##1{\hss\llap{##1}}}%
%     \fi}

```

`\@typog@itemize@patch` This is the additional code we inject into plain L^AT_EX's or package `enumitem`'s `\itemize`.

```

793 \newcommand*{\@typog@itemize@patch}

```

Save the original definition of `\@itemitem` for chain-calling it later on.

```

794   {\letcs{\@typog@old@itemitem}{\@itemitem}

```

Sneak in our own macro's name.

```

795   \edef\@itemitem{\@typog@labelitem\romannumeral\the\@itemdepth}

```

Redefine under the original macro's name so that our code gets called and the old code (`\@typog@old@itemitem`) is expanded.

```

796 \expandafter\def\csname\@itemitem\endcsname
797     {\raisebox{\csname typog@adjust@labelitem\romannumeral\the\@itemdepth\endcsname
798              {\@typog@old@itemitem}}}
799

```

If package `enumitem` has been loaded, we use the *same* patch. Here comes the result of `latexdef -c article -p enumitem -s enit@itemize@i` that explains, why no change is required:

```

% \def\enit@itemize@i#1#2#3#4{%
%   \ifnum #1 > #3 \relax
%     \enit@toodeep
%   \else
%     \enit@prelist\@ne{#1}{#2}%
%     \edef\@itemitem{label#2\romannumeral#1}%
%     \expandafter
%     \enit@list
%     \csname\@itemitem\endcsname
%     {\let\enit@calc\z@
%      \def\makelabel##1{\enit@align{\enit@format{##1}}}%
%      \enit@preset{#2}{#1}{#4}%
%      \enit@calcleft
%      \enit@before
%      \enit@negwidth}%
%     \enit@keyfirst
%   \fi}

```

`\@typog@patch@itemize` Unconditionally apply the patches that are just *single* macro calls to disturb the original macros as little as possible. If we detect `enumitem` to be present we modify its definition of `itemize` otherwise we wrestle L^AT_EX's macro.

```

800 \newcommand*{\@typog@patch@itemize}
801 {
802   \ifdefined\enit@itemize@i
803     \patchcmd{\enit@itemize@i}
804       {\expandafter}
805       {\@typog@itemize@patch\expandafter}
806       {\typog@debug@typeout{patching enumitem \string\enit@itemize@i\space
807         ceeded}}
808       {\PackageError{typog}
809         {Patching enumitem macro \string\enit@itemize@i\space
810         {}}}%
811   \else
812     \patchcmd{\itemize}
813       {\expandafter}
814       {\@typog@itemize@patch\expandafter}
815       {\typog@debug@typeout{patching \string\itemize\space suc-
816         ceeded}}
817     {\PackageError{typog}
818       {Patching plain LaTeX macro \string\itemize\space fai

```

```

816                                     {}}%
817   \fi}
818

```

@typog@maybe@patch@itemize Apply the patches only once.

```

819 \newbool{@typog@itemize@has@been@patched}
820 \newcommand*{@typog@maybe@patch@itemize}
821   {\ifbool{@typog@itemize@has@been@patched}
822     {\relax}
823     {@typog@patch@itemize
824       \booltrue{@typog@itemize@has@been@patched}}}
825

```

Freestanding height-adjusted label items.

\Adjustedlabelitemi

```

826 \NewDocumentCommand{\Adjustedlabelitemi}{}
827   {\raisebox{\typog@adjust@uppercase@labelitemi}
828     {\labelitemi}}

```

\adjustedlabelitemi

```

829 \NewDocumentCommand{\adjustedlabelitemi}{}
830   {\raisebox{\typog@adjust@lowercase@labelitemi}
831     {\labelitemi}}

```

\Adjustedlabelitemii

```

832 \NewDocumentCommand{\Adjustedlabelitemii}{}
833   {\raisebox{\typog@adjust@uppercase@labelitemii}
834     {\labelitemii}}

```

\adjustedlabelitemii

```

835 \NewDocumentCommand{\adjustedlabelitemii}{}
836   {\raisebox{\typog@adjust@lowercase@labelitemii}
837     {\labelitemii}}

```

\Adjustedlabelitemiii

```

838 \NewDocumentCommand{\Adjustedlabelitemiii}{}
839   {\raisebox{\typog@adjust@uppercase@labelitemiii}
840     {\labelitemiii}}

```

\adjustedlabelitemiii

```

841 \NewDocumentCommand{\adjustedlabelitemiii}{}
842   {\raisebox{\typog@adjust@lowercase@labelitemiii}
843     {\labelitemiii}}

```

\Adjustedlabelitemiv

```

844 \NewDocumentCommand{\Adjustedlabelitemiv}{}
845   {\raisebox{\typog@adjust@uppercase@labelitemiv}
846     {\labelitemiv}}

```

`\adjustedlabelitemiv`

```
847 \NewDocumentCommand{\adjustedlabelitemiv}{}
848   {\raisebox{\typog@adjust@lowercase@labelitemiv}
849     {\labelitemiv}}
850
```

And now for their PDFsubstitutes.

```
851 \typog@register@pdfsubstitute{
852   \def\Adjustedlabelitemi{\labelitemi}
853   \def\adjustedlabelitemi{\labelitemi}
854   \def\Adjustedlabelitemii{\labelitemii}
855   \def\adjustedlabelitemii{\labelitemii}
856   \def\Adjustedlabelitemiii{\labelitemiii}
857   \def\adjustedlabelitemiii{\labelitemiii}
858   \def\Adjustedlabelitemiv{\labelitemiv}
859   \def\adjustedlabelitemiv{\labelitemiv}
860 }
861
```

Here come our convenience macros to simplify an accurate setup of the label adjustments.

`\typog@hairline@width` Line width of the horizontal reference lines in our convenience macros.

```
862 \newcommand*{\typog@hairline@width}{.125pt}
```

`\typogadjuststairsfor` The arguments are: #1: *<scale-factor>*, #2: *<step-size>*, #3: *<number-of-steps>*, #4: *<sample>*, and #5: `\labelitem<N>`.

Generate an ascending stairs of argument #5.

```
863 \newcommand*{\typogadjuststairsfor}[5]
```

Store (half of) the space between two samples in `\dimen0`.

```
864   {\dimen0=1pt%
```

Load the *<number-of-steps>* and ensure that it is odd.

```
865   \count0=#3\relax
866   \unless\ifodd\count0
867     \advance\count0 by 1%
868   \fi
```

Set the iteration counter.

```
869   \setcounter{typog@@iteration}{1}%
```

Put the *<sample>* into a box so that we can measure it with `\ht`.

```
870   \setbox0=\hbox{#4}%
```

Box 1 is the accumulator for the raised samples.

```
871   \setbox1=\hbox{}%
```

Build the stairs.

```
872   \loop
873     \ifnum\thetypog@@iteration=\numexpr\count0 / 2\relax
874       \dimen1=3\dimen0
```

```

875     \else
876         \dimen1=\dimen0
877     \fi
878     \dimen2=\dimexpr#2 * (\thetypog@@iteration - \count0 / 2)\re-
lax
879     \setbox1=\hbox{\unhbox1\raisebox{\dimen2}{\kern\dimen1 #5\kern\dimen1}}%
880     \addtocounter{typog@@iteration}{1}%
881     \unless\ifnum\thetypog@@iteration>\count0
882     \repeat

```

Merge the stairs with a hairline at #1 times the height of *<sample>*. Answer just a single box.

```

883     \mbox{\rlap{\raisebox{\fpeval{#1}\ht0}{\rule{\wd1}{\typog@hairline@width}}}\bo
884

```

`\typogadjuststairs` The arguments are: #1: *<scale-factor>*, #2: *<step-size>*, #3: *<number-of-steps>*, and #4: *<sample>*.

```

885 \NewDocumentCommand{\typogadjuststairs}{0{.5} m m m}
886 { \begingroup
887   \unless\ifdim #2>\z@
888     \PackageError{typog}
889       {\string\typogadjuststairs\space non-positive step-
size}
890       {step-size must be a positive dimension}%
891   \fi
892   \ifnum #3<1
893     \PackageError{typog}
894       {\string\typogadjuststairs\space too few number-
of-steps}
895       {number-of-steps must at least be 1}%
896   \fi
897   \ifblank{#4}
898     {\PackageError{typog}
899       {sample must not be empty}
900       {supply either some uppercase or some low-
ercase letters}}
901   {}%
902   \def\arraystretch{1}%
903   \begin{tabular}{@{}c@{}}
904     \typogadjuststairsfor{#1}{#2}{#3}{#4}{\labelitemi} \\
905     \typogadjuststairsfor{#1}{#2}{#3}{#4}{\labelitemii} \\
906     \typogadjuststairsfor{#1}{#2}{#3}{#4}{\labelitemiii} \\
907     \typogadjuststairsfor{#1}{#2}{#3}{#4}{\labelitemiv}
908   \end{tabular}
909   \endgroup
910

```

`ercase@adjusted@labelitems` Return all four labelitems in a horizontal box after they have been adjusted with the uppercase-constants set.

```

911 \newcommand*{\typog@uppercase@adjusted@labelitems}
912 {\hbox{\raisebox{\typog@adjust@uppercase@labelitemi}{\labelitemi}%

```

```

913      \raisebox{\typog@adjust@uppercase@labelitemi}{\labelitemi}%
914      \raisebox{\typog@adjust@uppercase@labelitemii}{\labelitemii}%
915      \raisebox{\typog@adjust@uppercase@labelitemiv}{\labelitemiv}}}
```

`\typoguppercaseadjustcheck` We stuff the user's sample text into a box only to measure its height. We typeset all four labels and draw a hairline at half the height of the sample right through it.

```

916 \NewDocumentCommand{\typoguppercaseadjustcheck}{0{.5} m}
917   {\setbox0=\hbox{#2}%
918    \setbox1=\typog@uppercase@adjusted@labelitems
919    \mbox{\rlap{\raisebox{\fpeval{#1}\ht0}
920              {\rule{\wd1}{\typog@hairline@width}}}%
921           \box1}}
922
```

`ercase@adjusted@labelitems` Return all four labelitems in a horizontal box after they have been adjusted with the lowercase-constants set.

```

923 \newcommand*{\typog@lowercase@adjusted@labelitems}
924   {\hbox{\raisebox{\typog@adjust@lowercase@labelitemi}{\labelitemi}%
925           \raisebox{\typog@adjust@lowercase@labelitemii}{\labelitemii}%
926           \raisebox{\typog@adjust@lowercase@labelitemiii}{\labelitemiii}%
927           \raisebox{\typog@adjust@lowercase@labelitemiv}{\labelitemiv}}}
```

`\typoglowercaseadjustcheck` Same code as `\typoguppercaseadjustcheck` for lowercase.

```

928 \NewDocumentCommand{\typoglowercaseadjustcheck}{0{.5} m}
929   {\setbox0=\hbox{#2}%
930    \setbox1=\typog@lowercase@adjusted@labelitems
931    \mbox{\rlap{\raisebox{\fpeval{#1}\ht0}
932              {\rule{\wd1}{\typog@hairline@width}}}%
933           \box1}}
934
```

C.9 Align Last Line of a Paragraph

The code of environment `lastlinerraggedleftpar` has been inspired by macro `\lastlinerraggedleft` [40, Sec. 2].

`lastlinerraggedleftpar (env.)`

```

935 \NewDocumentEnvironment{lastlinerraggedleftpar}{}
936   {\lastlinefit=0%
937    \setlength{\leftskip}{\z@ \@plus 1fil}%
938    \setlength{\rightskip}{-\leftskip}%
939    \setlength{\parfillskip}{\leftskip}}
940   {\par}
```

`lastlineflushrightpar (env.)` Define `lastlineflushrightpar` as an alias of `lastlinerraggedleftpar`.

```

941 \let\lastlineflushrightpar=\lastlinerraggedleftpar
942 \let\endlastlineflushrightpar=\endlastlinerraggedleftpar
943
```


`lastlinecenteredpar` (*env.*) The code of environment `lastlinecenteredpar` has been inspired by *Tex By Topic* [12, Sec. 18.3.1].

```

944 \NewDocumentEnvironment{lastlinecenteredpar}{}
945   {\lastlinefit=0%
946    \setlength{\leftskip}{\z@ \@plus .5fil}%
947    \setlength{\rightskip}{-\leftskip}%
948    \setlength{\parfillskip}{\z@ \@plus 1fil}}
949   {\par}
950

```

C.10 Fill Last Line of a Paragraph

`shortenpar` (*env.*)

```

951 \NewDocumentEnvironment{shortenpar}{}
952   {\advance\looseness by -1
953    \ifnum\tracingparagraphs>0
954      \typeout{@ looseness \the\looseness}%
955    \fi}
956   {\par}
957

```

`prolongpar` (*env.*) We try to be prudent and inhibit hyphenation of the next-to-last line just in case the longer paragraph could be cheaply achieved by hyphenation – at the worst – of the last word.

```

958 \NewDocumentEnvironment{prolongpar}{}
959   {\finalhyphendemerits=100000001
960    \advance\looseness by 1
961    \ifnum\tracingparagraphs>0
962      \typeout{@ looseness \the\looseness}%
963    \fi}
964   {\par}
965

```

`xtindentpar@zero@parindent` This auxiliary macro and the following one are meant as an easy means to override the defaults of the user-visible environment `covernextindentpar`.

```

966 \newcommand*{\typog@covernextindentpar@zero@parindent}{2em}

```

`ndindentpar@nonzero@parindent`

```

967 \newcommand*{\typog@covernextindentpar@nonzero@parindent}{2\parindent}

```

`covernextindentpar` (*env.*)

```

968 \NewDocumentEnvironment{covernextindentpar}{o}
969   {\IfNoValueTF{#1}
970    {\ifdim\parindent=\z@
971     \dimen0=\dimexpr\linewidth - \typog@covernextindentpar@zero@parindent
972     \else
973     \dimen0=\dimexpr\linewidth - \typog@covernextindentpar@nonzero@parindent
974     \fi}
975    {\dimen0=\dimexpr\linewidth - (#1)}%
976    \parfillskip=\dimen0 \@minus \dimen0

```

```

977   \relax}
978   {\par}
979

```

`lastlinepar@zero@parindent` These auxiliary macros are meant as a means to override the defaults of the user-visible environment `openlastlinepar`.

```

980 \newcommand*{\typog@openlastlinepar@zero@parindent}{2em}

```

`lastlinepar@nonzero@parindent`

```

981 \newcommand*{\typog@openlastlinepar@nonzero@parindent}{2\parindent}

```

`openlastlinepar (env.)` Compare with the suggestion in reference 35.

```

982 \NewDocumentEnvironment{openlastlinepar}{o}
983   {\IfNoValueTF{#1}
984     {\ifdim\parindent=\z@
985       \skip0=\typog@openlastlinepar@zero@parindent
986       \@plus 1fil
987       \@minus \typog@openlastlinepar@zero@parindent
988     }else
989       \skip0=\typog@openlastlinepar@nonzero@parindent
990       \@plus 1fil
991       \@minus \typog@openlastlinepar@nonzero@parindent
992     }fi}
993   {\dimen0=\dimexpr#1\relax
994     \skip0=\dimen0 \@plus 1fil \@minus \dimen0}
995   \parfillskip=\skip0}
996   {\par}
997

```

`lastlinefitpar (env.)` Set value of `\lastlinefit` for a paragraph.

```

998 \NewDocumentEnvironment{lastlinefitpar}{0{1000}}
999   {\lastlinefit=#1\relax}
1000   {\par}
1001

```

C.11 Spacing

`\widespacestrength` Weight factor (“strength”) for `\fontdimen7`, the extra width of a sentence-ending space, we apply to construct our `\widespace` if `\fontdimen7` $\neq 0$. Can be increased to get a more pronounced effect.

```

1002 \newcommand*{\widespacestrength}{1.}

```

`\widespacescale` Scale factor we apply to the glue of the normal space to setup the glue of our `\widespacescale`. Also used in the fall-back calculation for the width if `\fontdimen7` = 0.

```

1003 \newcommand*{\widespacescale}{1.125}

```

`\widespace`

```

1004 \NewDocumentCommand{\widespace}{s}
1005   {\IfBooleanTF{#1}%

```

```

1006     {\dimen0=\widowspacescale\fontdimen2\font}%
1007     {\ifdim\fontdimen7\font=\z@
1008       \dimen0=\widowspacescale\fontdimen2\font
1009       \else
1010         \dimen0=\dimexpr\fontdimen2\font +
1011           \widowspacestrength\fontdimen7\font
1012       \fi}%
1013     \hskip \glueexpr\dimen0
1014         \@plus \widowspacescale\fontdimen3\font
1015         \@minus \widowspacescale\fontdimen4\font
1016     \ignorespaces}
1017

```

`\narrowospacestrength` Weight factor (“strength”) for `\fontdimen7`, the extra width of a sentence-ending space, we apply to construct our `\narrowospace` if `\fontdimen7` $\neq 0$. Can be increased to get a more pronounced effect.

```
1018 \newcommand*{\narrowospacestrength}{.5}
```

`\narrowowspacescale` Scale factor we apply to the glue of the normal space to setup the glue of our `\narrowowspacescale`. Also used in the fall-back calculation for the width if `\fontdimen7` = 0.

```
1019 \newcommand*{\narrowowspacescale}{.9375}
```

`\narrowospace`

```

1020 \NewDocumentCommand{\narrowospace}{s}
1021   {\IfBooleanTF{#1}%
1022     {\dimen0=\narrowowspacescale\fontdimen2\font}%
1023     {\ifdim\fontdimen7\font=\z@
1024       \dimen0=\narrowowspacescale\fontdimen2\font
1025       \else
1026         \dimen0=\dimexpr\fontdimen2\font -
1027           \narrowospacestrength\fontdimen7\font
1028       \fi}%
1029     \hskip \glueexpr\dimen0
1030         \@plus \narrowowspacescale\fontdimen3\font
1031         \@minus \narrowowspacescale\fontdimen4\font
1032     \ignorespaces}
1033

```

See also: TeX by Topic [12, ch. 20, p. 185–190].

`loosespacing` (*env.*)

```

1034 \NewDocumentEnvironment{loosespacing}{0{1}}
1035   {\dimen2=\fontdimen2\font
1036     \ifcase #1
1037       \spaceskip=\z@
1038       \or % 1          +5%
1039         \spaceskip=1.05\dimen2 \@plus .5\dimen2 \@minus .1\dimen2
1040       \or % 2          +10%
1041         \spaceskip=1.1\dimen2 \@plus .5\dimen2 \@minus .1\dimen2
1042       \or % 3          +20%

```

```

1043     \spaceskip=1.2\dimen2 \@plus .6\dimen2 \@minus .2\dimen2
1044     \else % >= 4      +30%
1045     \spaceskip=1.3\dimen2 \@plus .8\dimen2 \@minus .3\dimen2
1046     \fi
1047     \ignorespaces}
1048     {\ignorespacesafterend}
1049

```

`tightspacing` (*env.*)

```

1050 \NewDocumentEnvironment{tightspacing}{0{1}}
1051 {\dimen2=\fontdimen2\font
1052  \ifcase #1
1053    \spaceskip=\z@
1054    \or % 1      -1.25%
1055    \spaceskip=.9875\dimen2 \@plus .0125\dimen2 \@minus .5\dimen2
1056    \or % 2      -2.5%
1057    \spaceskip=.975\dimen2 \@plus .025\dimen2 \@minus .5\dimen2
1058    \or % 3      -5%
1059    \spaceskip=.95\dimen2 \@plus .05\dimen2 \@minus .5\dimen2
1060    \else % >= 4  -10%
1061    \spaceskip=.9\dimen2 \@plus .1\dimen2 \@minus .5\dimen2
1062    \fi
1063    \ignorespaces}
1064    {\ignorespacesafterend}
1065

```

C.12 Microtype Front-End

Tracking

`setfonttracking` (*env.*) To achieve the control we want, we must tinker with microtype's internals. Doh!

```

1066 \NewDocumentEnvironment{setfonttracking}{m}
1067 {\edef\MT@letterspace@{#1}%
1068  \lsstyle
1069  \ignorespaces}
1070  {\ignorespacesafterend}
1071

```

Font Expansion

`typog@setup@font@expansion` Note that we cannot factor the encodings into a macro; a single encoding would qualify, though. We need to support multiple encodings and thus go with the literal solution.

```

1072 \newcommand*{\typog@setup@font@expansion}
1073 {\SetExpansion
1074  [context = typog@shrink1,
1075   shrink = \typog@shrink@i,
1076   stretch = 0]%
1077  {encoding = {*}}}%
1078  {}
1079  \SetExpansion

```

```

1080     [context = typog@shrink2,
1081       shrink = \typog@shrink@ii,
1082       stretch = 0]%
1083     {encoding = {*}}%
1084     {}
1085   \SetExpansion
1086     [context = typog@shrink3,
1087       shrink = \typog@shrink@iii,
1088       stretch = 0]%
1089     {encoding = {*}}%
1090     {}
1091
1092   \SetExpansion
1093     [context = typog@stretch1,
1094       shrink = 0,
1095       stretch = \typog@stretch@i]%
1096     {encoding = {*}}%
1097     {}
1098   \SetExpansion
1099     [context = typog@stretch2,
1100       shrink = 0,
1101       stretch = \typog@stretch@ii]%
1102     {encoding = {*}}%
1103     {}
1104   \SetExpansion
1105     [context = typog@stretch3,
1106       shrink = 0,
1107       stretch = \typog@stretch@iii]%
1108     {encoding = {*}}%
1109     {}
1110
1111   \SetExpansion
1112     [context = typog@expand1,
1113       shrink = \typog@shrink@i,
1114       stretch = \typog@stretch@i]%
1115     {encoding = {*}}%
1116     {}
1117   \SetExpansion
1118     [context = typog@expand2,
1119       shrink = \typog@shrink@ii,
1120       stretch = \typog@stretch@ii]%
1121     {encoding = {*}}%
1122     {}
1123   \SetExpansion
1124     [context = typog@expand3,
1125       shrink = \typog@shrink@iii,
1126       stretch = \typog@stretch@iii]%
1127     {encoding = {*}}%
1128     {}

```

microtype@expansion@feature We cannot even parse the \iftypog@microtype@preloaded part further down unless the \ifMT@expansion conditional exists. So, we hoist this test

in a macro of its own. It only gets called if package microtype already has been sourced.

```

1129 \newcommand*{\typog@test@microtype@expansion@feature}
1130   {\ifMT@expansion
1131     \typog@typeout{microtype preloaded -- font expansion features avail-
1132       able}%
1133     \def\typog@require@microtype@expansion{\relax}
1134     \typog@setup@font@expansion
1135   }else
1136     \PackageWarning{typog}{microtype preloaded,\space
1137       but font expansion is disabled}%
1138     \def\typog@require@microtype@expansion
1139       {\PackageError{typog}
1140         {microtype font expansion disabled}
1141         {pass option 'expansion' to package microtype}}
1142   \fi}

```

`\typog@require@microtype@expansion` We are all set for the initialization of the font expansion, however, we must be careful in which (load-)state package microtype is in. Compare with the code for `\typog@require@microtype` and `\typog@require@preloaded@microtype`. Initialize our own flag and setup meaningful messages for later feature checks.

```

1142 \iftypog@microtype@preloaded
1143   \typog@test@microtype@expansion@feature
1144 }else
1145   \def\typog@require@microtype@expansion
1146     {\PackageError{typog}%
1147       {package microtype not (pre-)loaded, %
1148         which is required for typog's font expansion}%
1149       {require package microtype before package typog}}
1150 \fi
1151

```

`setfontshrink (env.)`

```

1152 \NewDocumentEnvironment{setfontshrink}{0{1}}
1153   {\typog@require@microtype@expansion
1154     \ifcase#1% 0
1155       \relax
1156     \or % 1
1157       \microtypecontext{expansion=typog@shrink1}%
1158     \or % 2
1159       \microtypecontext{expansion=typog@shrink2}%
1160     \else % >= 3
1161       \microtypecontext{expansion=typog@shrink3}%
1162     \fi
1163     \ignorespaces}
1164   {\ignorespacesafterend}
1165

```

`setfontstretch (env.)`

```

1166 \NewDocumentEnvironment{setfontstretch}{0{1}}
1167   {\typog@require@microtype@expansion

```

```

1168 \ifcase#1% 0
1169 \relax
1170 \or % 1
1171 \microtypecontext{expansion=typog@stretch1}%
1172 \or % 2
1173 \microtypecontext{expansion=typog@stretch2}%
1174 \else % >= 3
1175 \microtypecontext{expansion=typog@stretch3}%
1176 \fi
1177 \ignorespaces}
1178 {\ignorespacesafterend}
1179

```

setfontexpand (*env.*)

```

1180 \NewDocumentEnvironment{setfontexpand}{0{1}}
1181 {\typog@require@microtype@expansion
1182 \ifcase#1% 0
1183 \relax
1184 \or % 1
1185 \microtypecontext{expansion=typog@expand1}%
1186 \or % 2
1187 \microtypecontext{expansion=typog@expand2}%
1188 \else % >= 3
1189 \microtypecontext{expansion=typog@expand3}%
1190 \fi
1191 \ignorespaces}
1192 {\ignorespacesafterend}
1193

```

nofontexpansion (*env.*) Implementation: We proceed a different approach with respect to requiring package microtype. The semantics of the macro is to switch something off. If it is not ‘on’ because the necessary package was not loaded, a no-op is ok.

```

1194 \NewDocumentEnvironment{nofontexpansion}{}
1195 {\ifdefined\microtypesetup
1196 \microtypesetup{expansion=false}%
1197 \fi
1198 \ignorespaces}
1199 {\ignorespacesafterend}

```

nofontexpand (*env.*) Define nofontexpand as an alias of nofontexpansion.

```

1200 \let\nofontexpand=\nofontexpansion
1201 \let\endnofontexpand=\endnofontexpansion
1202

```

Character Protrusion

`nocharprotrusion` (*env.*) See ‘Implementation’ comment of `nofontexpansion`.

```

1203 \NewDocumentEnvironment{nocharprotrusion}{}
1204   {\ifdefined\microtypesetup
1205     \microtypesetup{protrusion=false}%
1206     \fi
1207     \ignorespaces}
1208   {\ignorespacesafterend}
1209 
```

C.13 Sloppy Paragraphs

`og@scaled@emergencystretch` Compute the correct scale factor for the emergency stretch, even if we do not have a valid `\linewidth`.

```

1210 \newcommand*{\typog@scaled@emergencystretch}[1]
1211   {\emergencystretch=\ifdim\linewidth=\z@
1212     #1%
1213     \else
1214       \dimexpr (#1) * \linewidth / \textwidth
1215     \fi}
1216 
```

`\slightlyloppy` Macro `\slightlyloppy` takes an optional *<loppiness>* index ranging from 0 to 8, where 0 means the same as `\fussy` and 8 or more works like `\sloppy`. The default *<loppiness>* is 1.

```

1217 \NewDocumentCommand{\slightlyloppy}{0{1}}
1218   {\ifcase #1% 0
1219     % \tolerance=200
1220     % \emergencystretch=\z@
1221     % \hfuzz=.1\p@
1222     % \vfuzz=\hfuzz
1223     \fussy
1224   \or % 1
1225     \pretolerance=165%
1226     \tolerance=330%
1227     \typog@scaled@emergencystretch{.375em}%
1228     \hfuzz=.15\p@
1229     \vfuzz=\hfuzz
1230   \or % 2
1231     \pretolerance=265%
1232     \tolerance=530%
1233     \typog@scaled@emergencystretch{.75em}%
1234     \hfuzz=.15\p@
1235     \vfuzz=\hfuzz
1236   \or % 3
1237     \pretolerance=435%
1238     \tolerance=870%
1239     \typog@scaled@emergencystretch{1.125em}%
1240     \hfuzz=.2\p@
1241     \vfuzz=\hfuzz

```



```

1242 \or % 4
1243   \pretolerance=705%
1244   \tolerance=1410%
1245   \typog@scaled@emergencystretch{1.5em}%
1246   \hfuzz=.3\p@
1247   \vfuzz=\hfuzz
1248 \or % 5
1249   \pretolerance=1155%
1250   \tolerance=2310%
1251   \typog@scaled@emergencystretch{1.875em}%
1252   \hfuzz=.35\p@
1253   \vfuzz=\hfuzz
1254 \or % 6
1255   \pretolerance=1880%
1256   \tolerance=3760%
1257   \typog@scaled@emergencystretch{2.25em}%
1258   \hfuzz=.4\p@
1259   \vfuzz=\hfuzz
1260 \or % 7
1261   \pretolerance=3065%
1262   \tolerance=6130%
1263   \typog@scaled@emergencystretch{2.625em}%
1264   \hfuzz=.45\p@
1265   \vfuzz=\hfuzz
1266 \else % >= 8
1267   % \tolerance=9999
1268   % \emergencystretch=3em
1269   % \hfuzz=.5\p@
1270   % \vfuzz=\hfuzz
1271   \sloppy
1272 \fi
1273 \ignorespaces}

```

Implementation Note

- The `\tolerance` values are calculated as the geometric mean of the extreme values 200 and 9999. This means the factor

$$f = \left(\frac{9999}{200} \right)^{1/8} \approx 1.63$$

defines additional tolerances which we generously round values in the actual implementation.

- The `\emergencystretch` is scaled linearly with $\langle sloppiness \rangle$ and the ratio of the actual `\linewidth` to the (maximum) `\textwidth`.
- The `\hfuzz` values are interpolated linearly with $\langle sloppiness \rangle$ between .1pt and .5pt.

Maxima code to calculate the intermediate values.

Initialize. `load("list_functions")$`

\tolerance: `logspace(log10(200), log10(9999), 9), numer;`

```

\emergencystretch: linspace(0, 3, 9), numer;
\hfuzz: linspace(.1, .5, 9);

```

slightlyloppypar (*env.*)

```

1274 \NewDocumentEnvironment{slightlyloppypar}{0{1}}
1275   {\par\slightlyloppy[#1]\ignorespaces}
1276   {\par}
1277

```

C.14 Vertically Partially-Tied Paragraphs

\typog@geometric@mean This is just the usual geometric mean of two values x and y : $\sqrt{xy}$.

```

1278 \ExplSyntaxOn
1279 \newcommand*{\typog@geometric@mean}[2]
1280   {\fp_to_int:n {sqrt((#1) * (#2))}}
1281 \ExplSyntaxOff
1282

```

typog@mean@penalty Reserve a private counter for the geometric-mean penalties.

```

1283 \newcounter{typog@mean@penalty}
1284

```

\vtietop

```

1285 \NewDocumentCommand{\vtietop}{0{3}}
1286   {\setcounter{typog@mean@penalty}
1287     {\typog@geometric@mean{\@M}{\clubpenalty}}}%
1288   \typog@debug@typeout{vtietop: penalties \the\@M--\the\value{typog@mean@penalty}
1289     -\the\clubpenalty}%
1289   \unless\ifnum\clubpenalty<\@M
1290     \PackageWarning{typog}{vtietop: clubpenalty=\the\clubpenalty\space>= 10000}%
1291   \fi
1292   \ifcase#1% 0
1293     \relax
1294   \or % 1
1295     \relax
1296   \or % 2
1297     \clubpenalties 3
1298     \@M
1299     \value{typog@mean@penalty}
1300     \clubpenalty
1301   \or % 3
1302     \clubpenalties 4
1303     \@M \@M
1304     \value{typog@mean@penalty}
1305     \clubpenalty
1306   \or % 4
1307     \clubpenalties 5
1308     \@M \@M \@M
1309     \value{typog@mean@penalty}
1310     \clubpenalty
1311   \or % 5

```

```

1312     \clubpenalties 6
1313         \@M \@M \@M \@M
1314         \value{typog@mean@penalty}
1315         \clubpenalty
1316     \or % 6
1317         \clubpenalties 7
1318             \@M \@M \@M \@M \@M
1319             \value{typog@mean@penalty}
1320             \clubpenalty
1321     \or % 7
1322         \clubpenalties 8
1323             \@M \@M \@M \@M \@M
1324             \value{typog@mean@penalty}
1325             \clubpenalty
1326     \or % 8
1327         \clubpenalties 9
1328             \@M \@M \@M \@M \@M
1329             \value{typog@mean@penalty}
1330             \clubpenalty
1331     \else % >= 9
1332         \clubpenalties 10
1333             \@M \@M \@M \@M \@M \@M \@M \@M \@M
1334             \value{typog@mean@penalty}
1335             \clubpenalty
1336     \fi}
1337

```

`vtietoppar (env.)`

```

1338 \NewDocumentEnvironment{vtietoppar}{0{3}}
1339 { \vtietop[#1]}
1340 { \par
1341   \ignorespacesafterend}
1342

```

`\splicevtietop`

```

1343 \NewDocumentCommand{\splicevtietop}{0{3}}
1344 { \let\typog@old@item=\@item
1345   \def\@item[##1]{\typog@old@item[##1]\vtietop[#1]}%
1346   \ignorespaces}
1347

```

We define an extra style for the users of `enumitem`. Its only drawback is that it hard-codes the default number of tied lines (3).

```

1348 \ifdefined\SetEnumitemKey
1349   \SetEnumitemKey{vtietop}{first=\splicevtietop}
1350 \fi
1351

```

`\vtiebot`

```

1352 \NewDocumentCommand{\vtiebot}{0{3}}
1353 { \setcounter{typog@mean@penalty}
1354   {\typog@geometric@mean{\@M}{\widowpenalty}}%

```

```

1355 \typog@debug@typeout{vtiebot: penalties \the\@M--\the\value{typog@mean@penalty
-\the\widowpenalty}%
1356 \unless\ifnum\widowpenalty<\@M
1357 \PackageWarning{typog}{vtiebot: widowpenalty=\the\widowpenalty\space>= 10000
1358 \fi
1359 \ifcase#1% 0
1360 \relax
1361 \or % 1
1362 \relax
1363 \or % 2
1364 \widowpenalties 3
1365 \@M
1366 \value{typog@mean@penalty}
1367 \widowpenalty
1368 \or % 3
1369 \widowpenalties 4
1370 \@M \@M
1371 \value{typog@mean@penalty}
1372 \widowpenalty
1373 \or % 4
1374 \widowpenalties 5
1375 \@M \@M \@M
1376 \value{typog@mean@penalty}
1377 \widowpenalty
1378 \or % 5
1379 \widowpenalties 6
1380 \@M \@M \@M \@M
1381 \value{typog@mean@penalty}
1382 \widowpenalty
1383 \or % 6
1384 \widowpenalties 7
1385 \@M \@M \@M \@M \@M
1386 \value{typog@mean@penalty}
1387 \widowpenalty
1388 \or % 7
1389 \widowpenalties 8
1390 \@M \@M \@M \@M \@M \@M
1391 \value{typog@mean@penalty}
1392 \widowpenalty
1393 \or % 8
1394 \widowpenalties 9
1395 \@M \@M \@M \@M \@M \@M \@M
1396 \value{typog@mean@penalty}
1397 \widowpenalty
1398 \else % >= 9
1399 \widowpenalties 10
1400 \@M \@M \@M \@M \@M \@M \@M \@M
1401 \value{typog@mean@penalty}
1402 \widowpenalty
1403 \fi}
1404

```

vtiebotpar (*env.*)

```

1405 \NewDocumentEnvironment{vtiebotpar}{0{3}}
1406   {\vtiebot[#1]}
1407   {\par
1408     \ignorespacesafterend}
1409

```

\typog@vtiebotdisp

```

1410 \NewDocumentCommand{\typog@vtiebotdisp}{m}
1411   {\setcounter{typog@mean@penalty}
1412     {\typog@geometric@mean{\@M}{\displaywidowpenalty}}%
1413     \typog@debug@typeout{vtiebotdisp: penalties \the\@M--\the\value{typog@mean@pen}
1414     -\the\displaywidowpenalty}%
1415     \unless\ifnum\displaywidowpenalty<\@M
1416       \PackageWarning{typog}{vtiebotdisp: displaywidowpenalty=\the\displaywidowpen}
1417     \fi
1418     \ifcase#1% 0
1419       \relax
1420     \or % 1
1421       \relax
1422     \or % 2
1423       \displaywidowpenalties 3
1424       \@M
1425       \value{typog@mean@penalty}
1426       \displaywidowpenalty
1427     \or % 3
1428       \displaywidowpenalties 4
1429       \@M \@M
1430       \value{typog@mean@penalty}
1431       \displaywidowpenalty
1432     \or % 4
1433       \displaywidowpenalties 5
1434       \@M \@M \@M
1435       \value{typog@mean@penalty}
1436       \displaywidowpenalty
1437     \or % 5
1438       \displaywidowpenalties 6
1439       \@M \@M \@M \@M
1440       \value{typog@mean@penalty}
1441       \displaywidowpenalty
1442     \or % 6
1443       \displaywidowpenalties 7
1444       \@M \@M \@M \@M \@M
1445       \value{typog@mean@penalty}
1446       \displaywidowpenalty
1447     \or % 7
1448       \displaywidowpenalties 8
1449       \@M \@M \@M \@M \@M \@M
1450       \value{typog@mean@penalty}
1451       \displaywidowpenalty
1452     \or % 8
1453       \displaywidowpenalties 9

```

```

1453      \@M \@M \@M \@M \@M \@M \@M
1454      \value{typog@mean@penalty}
1455      \displaywidowpenalty
1456    \else % >= 9
1457      \displaywidowpenalties 10
1458      \@M \@M \@M \@M \@M \@M \@M \@M
1459      \value{typog@mean@penalty}
1460      \displaywidowpenalty
1461    \fi}
1462

```

`vtiebotdisp (env.)`

```

1463 \NewDocumentEnvironment{vtiebotdisp}{0{3}}
1464 {\typog@vtiebotdisp{#1}}
1465 {\ignorespacesafterend}
1466

```

`vtiebotdisptoppar (env.)`

```

1467 \NewDocumentEnvironment{vtiebotdisptoppar}{0{3}o}
1468 {\postdisplaypenalty=\@M
1469 \predisplaypenalty=10001% in accordance with package ‘widows-
and-orphans’
1470 \edef\typog@@top@lines{\IfNoValueTF{#2}{#1}{#2}}%
1471 \edef\typog@@after@display@math{\vtietop[\typog@@top@lines]}%
1472 \PushPostHook{display}{\aftergroup\typog@@after@display@math}%
1473 \vtiebotdisp[#1]}
1474 {\par
1475 \PopPostHook{display}%
1476 \ignorespacesafterend}
1477

```

C.15 Breakable Displayed Equations

`breakabledisplay (env.)` We use a different default, 3, than `\allowdisplaybreaks` which utilizes 4 as its default.

```

1478 \newenvironment*{breakabledisplay}[1][3]
1479 {\allowdisplaybreaks[#1]}
1480 {\ignorespacesafterend}
1481

```

C.16 Setspace Front End

`\typog@iter@limit` The maximum number of iterations we perform before bailing out with an error. Can be changed by the user if convergence is slow.

```

1482 \newcommand*{\typog@setbaselineskip@iter@limit}{10}

```

`baselineskip@relative@error` The maximum relative error of the ratio we tolerate for the final `baselineskip` over the target `baselineskip`. Can also be changed by the user if necessary.

```

1483 \newcommand*{\typog@setbaselineskip@relative@error}{.001}

```

\typog@setbaselineskip Given the $\langle target-baselineskip \rangle$ as argument iterate setting \setstretch until the error drops below our threshold.

```

1484 \ExplSyntaxOn
1485 \cs_new:Npn \typog@setbaselineskip #1
1486 {
    Initialize our “emergency-stop” loop counter.
1487   \int_set:Nn \l_tmpa_int {1}
1488   \int_set:Nn \l_tmpb_int {\typog@setbaselineskip@iter@limit}
    Note that the call to \glueexpr is required to consume dimensions that carry
    stretchability via plus or minus.
1489   \dim_set:Nn \l_tmpa_dim {\glueexpr #1}
1490
1491   \typog@debug@typeout{\string\setbaselineskip:\space
1492     initial\space baselineskip:\space \the\baselineskip}
1493   \typog@debug@typeout{\string\setbaselineskip:\space
1494     target\space baselineskip:\space \dim_use:N \l_tmpa_dim}
1495
1496   \dim_compare:nNnTF {\baselineskip} > {\c_zero_dim}
1497   {}
1498   {
1499     \PackageError{typog}
1500       {\string\setbaselineskip:\space
1501         baselineskip\space not\space positive}
1502     {}
1503   }
1504
1505   \dim_compare:nNnTF {\l_tmpa_dim} > {\c_zero_dim}
1506   {}
1507   {
1508     \PackageError{typog}
1509       {\string\setbaselineskip:\space target\space
1510         baselineskip\space must\space be\space
1511         positive}
1512     {}
1513   }
1514
1515   \skip_if_eq:nNnTF {\l_tmpa_dim} {\glueexpr #1}
1516   {}
1517   {
1518     \PackageWarning{typog}
1519       {\string\setbaselineskip:\space argument\space
1520         is\space a\space skip;\space
1521         will\space ignore\space glue}
1522     {}
1523   }
1524
1525   \fp_set:Nn \l_tmpa_fp {\l_tmpa_dim / \baselineskip}
1526   \fp_until_do:nNnn {abs(\l_tmpa_dim / \baselineskip - 1)} <
1527     {\typog@setbaselineskip@relative@error}
1528   {

```

```

1529 \setstretch{\fp_use:N \l_tmpa_fp}
1530 \fp_set:Nn \l_tmpa_fp
1531      {\l_tmpa_fp * \l_tmpa_dim / \baselineskip}
1532
1533 \int_incr:N \l_tmpa_int
1534 \int_compare:nNnTF {\l_tmpa_int} > {\l_tmpb_int}
1535 {
1536   \PackageError{typog}
1537       {\string\setbaselineskip:\space excessive\space
1538        number\space of\space iterations:\space
1539         \int_use:N \l_tmpa_int\space >\space
1540         \int_use:N \l_tmpb_int}
1541   {}
1542 }
1543 {}
1544 }
1545
1546 \typog@debug@typeout{\string\setbaselineskip:\space
1547   final\space \string\setstretch\space argument:\space
1548   \fp_use:N \l_tmpa_fp}
1549 \typog@debug@typeout{\string\setbaselineskip:\space
1550   final\space baselineskip:\space \the\baselineskip}
1551 }
1552

```

`\setbaselineskip` Set the `\baselineskip` to an absolute length.

Implementation Note

Viewed as a standalone macro `\setbaselineskip` does not need the decoration `\AfterPreamble`. However, all of its siblings, `\setbaselineskippercentage`, `\setleading`, and `\setleadingpercentage` then would behave differently as they are delayed to the end of the preamble, but `\setbaselineskip` immediately becomes effective. For example, the successive calls

```

\setbaselineskippercentage{140}
\setbaselineskip{12.5pt}

```

in the preamble would set the baselineskip to 140% in the document. Therefore, `\setbaselineskip` is delayed too and the order of the calls thus preserved.

```

1553 \cs_new:Npn \setbaselineskip #1
1554 {
1555   \AfterPreamble{\typog@setbaselineskip{#1}}
1556   \ignorespaces
1557 }
1558

```

`\resetbaselineskip` Set the `\baselineskip` to ‘neutral’.

```

1559 \cs_new:Npn \resetbaselineskip
1560 {
1561   \AfterPreamble{\setstretch{1}}
1562 }

```


1563

`\typogfontsize` (*dimen*) Define the default font-size/quad size.

1564 `\dim_new:N \typogfontsize`

Initialize `\typogfontsize` at the end of the preamble, which is after all fonts have been setup.

1565 `\AfterEndPreamble{`1566 `\dim_set:Nn \typogfontsize {\fontdimen6\font}`1567 `\typog@debug@typeout{\string\typogfontsize =`1568 `\dim_use:N \typogfontsize\space`1569 `(at\space begin\space of\space document)}`1570 `}`

1571

`\setbaselineskippercentage`

1572 `\cs_new:Npn \setbaselineskippercentage #1`1573 `{`1574 `\AfterPreamble{`1575 `\dim_compare:nNnTF {\typogfontsize} > {\c_zero_dim}`1576 `{`1577 `\typog@setbaselineskip{`1578 `\fp_eval:n {(#1) / 100} \typogfontsize}`1579 `}`1580 `{`1581 `\PackageError{typog}`1582 `{\string\setbaselineskippercentage:\space`1583 `\string\typogfontsize <= 0}`1584 `{Maybe\space \string\typogfontsize\space`1585 `is\space uninitialized?}`1586 `}`1587 `}`1588 `\ignorespaces`1589 `}`

1590

`\setleading`

1591 `\cs_new:Npn \setleading #1`1592 `{`1593 `\AfterPreamble{`1594 `\dim_compare:nNnTF {\typogfontsize} > {\c_zero_dim}`1595 `{`1596 `\typog@setbaselineskip{\typogfontsize + \dimexpr #1}`1597 `}`1598 `{`1599 `\PackageError{typog}`1600 `{\string\setleading:\space`1601 `\string\typogfontsize <= 0}`1602 `{Maybe\space \string\typogfontsize\space`1603 `is\space uninitialized?}`1604 `}`

```

1605 }
1606 \ignorespaces
1607 }
1608

```

`\setleadingpercentage`

```

1609 \cs_new:Npn \setleadingpercentage #1
1610 {
1611   \AfterPreamble{
1612     \dim_compare:nNnTF {\typogfontsize} > {\c_zero_dim}
1613     {
1614       \typog@setbaselineskip{
1615         \fp_eval:n {1 + (#1) / 100} \typogfontsize}
1616     }
1617     {
1618       \PackageError{typog}
1619         {\string\setleadingpercentage:\space
1620          \string\typogfontsize <= 0}
1621         {Maybe\space \string\typogfontsize\space
1622          is\space uninitialized?}
1623     }
1624   }
1625   \ignorespaces
1626 }
1627 \ExplSyntaxOff
1628

```

C.17 Smooth Ragged

`\typog@repeat` As we shall have to repeat the line specifications for our paragraphs so often we introduce the two argument macro `\typog@repeat` that takes a *repeat-count* and a *body* that is repeated.

```

1629 \ExplSyntaxOn
1630 \cs_new_eq:NN \typog@repeat \prg_replicate:nn
1631

```

`\typog@mod` For error checking we shall need the modulo operation on integers, i. e., the remainder of an integral division.

```

1632 \newcommand*{\typog@mod}[2]{\int_mod:nn{#1}{#2}}
1633 \ExplSyntaxOff
1634

```

`\typog@triplet@max@lines` Maximum number of lines a smoothraggedright paragraph can have with the triplet generator. The number must be divisible by 3.

```

1635 \newcommand*{\typog@triplet@max@lines}{99}
1636

```

`aggedrightshapetriplet (env.)` Engine for 3-line repetitions.

```

1637 \define@key[typog]{smoothraggedrightshapetriplet}{leftskip}{%
1638   {\def\typog@triplet@leftskip{#1}}

```

```

1639 \define@key[typog]{smoothraggedrightshapetriplet}{parindent}%
1640         {\def\typog@@triplet@parindent{#1}}
1641 \NewDocumentEnvironment{smoothraggedrightshapetriplet}{0}{ m m m}
1642 {\def\typog@@triplet@leftskip{\z@}%
1643  \def\typog@@triplet@parindent{\z@}%
1644  \setkeys*[typog]{smoothraggedrightshapetriplet}{#1}%
1645  \skip0=\typog@@triplet@leftskip\relax
1646  \skip1=#2\relax
1647  \skip2=#3\relax
1648  \skip3=#4\relax
1649  \typog@debug@typeout{smoothraggedrightshapetriplet: skip0=\the\skip0}%
1650  \typog@debug@typeout{smoothraggedrightshapetriplet: skip1=\the\skip1}%
1651  \typog@debug@typeout{smoothraggedrightshapetriplet: skip2=\the\skip2}%
1652  \typog@debug@typeout{smoothraggedrightshapetriplet: skip3=\the\skip3}%
1653  \unless\ifnum\typog@mod{\typog@@triplet@max@lines}{3}=0
1654    \PackageError{typog}
1655        {Line number of triplet generator\space
1656         (\typog@@triplet@max@lines) not divisible by 3}
1657    {}
1658  \fi
1659  \edef\typog@@triplet@linespecs{%
1660    \glueexpr \skip0 + \typog@@triplet@parindent\relax
1661    \glueexpr \skip1 - \typog@@triplet@parindent\relax
1662    \skip0 \skip2 \skip0 \skip3
1663    \typog@repeat{\numexpr\typog@@triplet@max@lines / 3 - 1}
1664    {\skip0 \skip1 \skip0 \skip2 \skip0 \skip3}}
1665  \parshape=\typog@@triplet@max@lines\typog@@triplet@linespecs\relax
1666  {\par}
1667

```

`\typog@quintuplet@max@lines` Maximum number of lines a smoothraggedright paragraph can have with the quintuplet generator. The number must be divisible by 5.

```

1668 \newcommand*{\typog@quintuplet@max@lines}{95}
1669

```

`\smoothraggedrightshapequintuplet (env.)` Engine for 5-line repetitions.

```

1670 \define@key[typog]{smoothraggedrightshapequintuplet}{leftskip}
1671         {\def\typog@@quintuplet@leftskip{#1}}
1672 \define@key[typog]{smoothraggedrightshapequintuplet}{parindent}
1673         {\def\typog@@quintuplet@parindent{#1}}
1674 \NewDocumentEnvironment{smoothraggedrightshapequintuplet}{0}{ m m m m m}
1675 {\def\typog@@quintuplet@leftskip{\z@}%
1676  \def\typog@@quintuplet@parindent{\z@}%
1677  \setkeys*[typog]{smoothraggedrightshapequintuplet}{#1}%
1678  \skip0=\typog@@quintuplet@leftskip\relax
1679  \skip1=#2\relax
1680  \skip2=#3\relax
1681  \skip3=#4\relax
1682  \skip4=#5\relax
1683  \skip5=#6\relax
1684  \typog@debug@typeout{smoothraggedrightshapequintuplet: skip0=\the\skip0}%
1685  \typog@debug@typeout{smoothraggedrightshapequintuplet: skip1=\the\skip1}%

```

```

1686 \typog@debug@typeout{smoothraggedrightshapequintuplet: skip2=\the\skip2}%
1687 \typog@debug@typeout{smoothraggedrightshapequintuplet: skip3=\the\skip3}%
1688 \typog@debug@typeout{smoothraggedrightshapequintuplet: skip4=\the\skip4}%
1689 \typog@debug@typeout{smoothraggedrightshapequintuplet: skip5=\the\skip5}%
1690 \unless\ifnum\typog@mod{\typog@quintuplet@max@lines}{5}=0
1691   \PackageError{typog}
1692     {Line number of quintuplet generator\space
1693      (\typog@quintuplet@max@lines) not divisible by 5}
1694   {}
1695 \fi
1696 \edef\typog@@quintuplet@linespecs{%
1697   \glueexpr \skip0 + \typog@@quintuplet@parindent\relax
1698   \glueexpr \skip1 - \typog@@quintuplet@parindent\relax
1699   \skip0 \skip2 \skip0 \skip3
1700   \skip0 \skip4 \skip0 \skip5
1701   \typog@repeat{\numexpr\typog@quintuplet@max@lines / 5 - 1}
1702     {\skip0 \skip1 \skip0 \skip2 \skip0 \skip3
1703     \skip0 \skip4 \skip0 \skip5}}
1704 \parshape=\typog@quintuplet@max@lines\typog@@quintuplet@linespecs\relax}
1705 {\par}

```

\typog@septuplet@max@lines Maximum number of lines a smoothraggedright paragraph can have with the septuplet generator. The number must be divisible by 7.

```

1706 \newcommand*{\typog@septuplet@max@lines}{98}
1707

```

gedrightshapeseptuplet (*env.*) Engine for 7-line repetitions.

```

1708 \define@key[typog]{smoothraggedrightshapeseptuplet}{leftskip}%
1709   {\def\typog@@septuplet@leftskip{#1}}
1710 \define@key[typog]{smoothraggedrightshapeseptuplet}{parindent}%
1711   {\def\typog@@septuplet@parindent{#1}}
1712 \NewDocumentEnvironment{smoothraggedrightshapeseptuplet}{0{} m m m m m m m}
1713   {\def\typog@@septuplet@leftskip{\z@}%
1714   \def\typog@@septuplet@parindent{\z@}%
1715   \setkeys*{typog}{smoothraggedrightshapeseptuplet}{#1}%
1716   \skip0=\typog@@septuplet@leftskip\relax
1717   \skip1=#2\relax
1718   \skip2=#3\relax
1719   \skip3=#4\relax
1720   \skip4=#5\relax
1721   \skip5=#6\relax
1722   \skip6=#7\relax
1723   \skip7=#8\relax
1724   \typog@debug@typeout{smoothraggedrightshapeseptuplet: skip0=\the\skip0}%
1725   \typog@debug@typeout{smoothraggedrightshapeseptuplet: skip1=\the\skip1}%
1726   \typog@debug@typeout{smoothraggedrightshapeseptuplet: skip2=\the\skip2}%
1727   \typog@debug@typeout{smoothraggedrightshapeseptuplet: skip3=\the\skip3}%
1728   \typog@debug@typeout{smoothraggedrightshapeseptuplet: skip4=\the\skip4}%
1729   \typog@debug@typeout{smoothraggedrightshapeseptuplet: skip5=\the\skip5}%
1730   \typog@debug@typeout{smoothraggedrightshapeseptuplet: skip6=\the\skip6}%
1731   \typog@debug@typeout{smoothraggedrightshapeseptuplet: skip7=\the\skip7}%
1732   \unless\ifnum\typog@mod{\typog@septuplet@max@lines}{7}=0

```

```

1733 \PackageError{typog}
1734         {Line number of septuplet generator\space
1735         (\typog@septuplet@max@lines) not divisible by 7}
1736         {}
1737 \fi
1738 \edef\typog@@septuplet@linespecs{%
1739     \glueexpr \skip0 + \typog@@septuplet@parindent\relax
1740     \glueexpr \skip1 - \typog@@septuplet@parindent\relax
1741     \skip0 \skip2 \skip0 \skip3 \skip0 \skip4
1742     \skip0 \skip5 \skip0 \skip6 \skip0 \skip7
1743     \typog@repeat{\numexpr\typog@septuplet@max@lines / 7 - 1}
1744     {\skip0 \skip1 \skip0 \skip2 \skip0 \skip3 \skip0 \skip4
1745     \skip0 \skip5 \skip0 \skip6 \skip0 \skip7}}
1746 \parshape=\typog@septuplet@max@lines\typog@@septuplet@linespecs\relax
1747 {\par}
1748

```

smoothraggedrightfuzzfactor

```

1749 \newcommand*{\smoothraggedrightfuzzfactor}{1.0}

```

smoothraggedrightgenerator

```

1750 \newcommand*{\smoothraggedrightgenerator}{triplet}

```

smoothraggedrightleftskip (*skip*)

```

1751 \newlength{\smoothraggedrightleftskip}

```

smoothraggedrightparindent (*skip*)

```

1752 \newlength{\smoothraggedrightparindent}

```

smoothraggedrighttragwidth (*skip*)

```

1753 \newlength{\smoothraggedrighttragwidth}
1754 \setlength{\smoothraggedrighttragwidth}{2em}
1755

```

\typog@fuzzwidth (*dimen*)

```

1756 \newdimen{\typog@fuzzwidth}
1757

```

smoothraggedrightpar (*env.*) The longest line will be \linewidth wide unless overridden by optional argument linewidth.

```

1758 \define@key[typog]{smoothraggedrightpar}{linewidth}%
1759     {\def\typog@@linewidth{#1}}
1760

```

Convert generator name to an integer suitable for \ifcase. Indicate an unknown name with -1.

```

1761 \ExplSyntaxOn
1762 \cs_new:Npn \typog@generator@index:n #1 {

```

```

1763 \str_case:nnTF {#1}
1764 {
1765     {triplet} {0}
1766     {quintuplet} {1}
1767     {septuplet} {2}
1768 }
1769 {}
1770 {-1}
1771 }
1772 \cs_generate_variant:Nn \typog@generator@index:n {e}
1773 \let\typog@generator@index=\typog@generator@index:e
1774 \ExplSyntaxOff
1775
1776 \NewDocumentEnvironment{smoothraggedrightpar}{0{}}
1777 {\edef\typog@@linewidth{\linewidth}%
1778  \setkeys[typog]{smoothraggedrightpar}{#1}%
1779  \edef\typog@@generatorchoice{\typog@generator@index{\smoothraggedrightgenerator
Obey to the indentation prescribed by any list environment.
1780  \let\typog@@smoothraggedrightleftskip=\smoothraggedrightleftskip
1781  \ifnum\@listdepth>0
1782    \addtolength{\typog@@smoothraggedrightleftskip}{\leftmargin}%
1783  \fi
Scale the fuzz-width by the user's factor. Later we shall rescale again specifically
for each generator.
1784  \typog@fuzzwidth=\smoothraggedrightfuzzfactor\smoothraggedrightragwidth
    Now for the generator-specific code ...
1785  \ifcase\typog@@generatorchoice
    generator=triplet produces a “short line – long line – middle length
line” sequence.
1786    \typog@fuzzwidth=.25\smoothraggedrightragwidth
1787    \typog@debug@typeout{smoothraggedright: generator=triplet, ty-
pog@fuzzwidth=\the\typog@fuzzwidth}%
1788    \smoothraggedrightshapetriplet[leftskip=\typog@@smoothraggedrightleftskip,
1789    parindent=\glueexpr\smoothraggedrightparinden
indent,
1790    #1]%
1791    {\glueexpr \typog@@linewidth - \smoothraggedrightragwidth
1792     + \glueexpr \z@ \@plus \typog@fuzzwidth\relax}% (1)
1793    {\glueexpr \typog@@linewidth \@minus \typog@fuzzwidth}% (3)
1794    {\glueexpr (\typog@@linewidth * 2 - \smoothraggedrightrag-
width) / 2
1795     + \glueexpr \z@ \@plus \typog@fuzzwidth \@mi-
nus \typog@fuzzwidth\relax}% (2)
1796  \or
    generator=quintuplet.
1797    \typog@fuzzwidth=.125\smoothraggedrightragwidth
1798    \typog@debug@typeout{smoothraggedright: generator=quintuplet, ty-
pog@fuzzwidth=\the\typog@fuzzwidth}%
1799    \smoothraggedrightshapequintuplet[leftskip=\typog@@smoothraggedrightleftskip

```

```

1800                                     parindent=\glueexpr\smoothraggedrightparin
      indent,
1801                                     #1]%
1802      {\glueexpr (\typog@@linewidth * 4 - \smoothraggedrightrag-
width * 3) / 4
1803      + \glueexpr \z@ \@plus \typog@fuzzwidth \@mi-
nus \typog@fuzzwidth\relax}% (2)
1804      {\glueexpr \typog@@linewidth \@minus \typog@fuzzwidth\relax}% (5)
1805      {\glueexpr (\typog@@linewidth * 2 - \smoothraggedrightrag-
width) / 2
1806      + \glueexpr \z@ \@plus \typog@fuzzwidth \@mi-
nus \typog@fuzzwidth\relax}% (3)
1807      {\glueexpr (\typog@@linewidth * 4 - \smoothraggedrightrag-
width) / 4
1808      + \glueexpr \z@ \@plus \typog@fuzzwidth \@mi-
nus \typog@fuzzwidth\relax}% (4)
1809      {\glueexpr \typog@@linewidth - \smoothraggedrightragwidth
1810      + \glueexpr \z@ \@plus \typog@fuzzwidth\relax}% (1)
1811      \or
      generator=septuplet.
      Permutation 3-6-1-5-2-7-4 looks 'random' enough for our purposes.
1812      \typog@fuzzwidth=.08333\smoothraggedrightragwidth
1813      \typog@debug@typeout{smoothraggedright: generator=septuplet, ty-
pog@fuzzwidth=\the\typog@fuzzwidth}%
1814      \smoothraggedrightshapeseptuplet[leftskip=\typog@smoothraggedrightleftskip,
1815      parindent=\glueexpr\smoothraggedrightparind
      indent,
1816      #1]%
1817      {\glueexpr (\typog@@linewidth * 3 - \smoothraggedrightrag-
width * 2) / 3
1818      + \glueexpr \z@ \@plus \typog@fuzzwidth \@mi-
nus \typog@fuzzwidth\relax}% (3)
1819      {\glueexpr (\typog@@linewidth * 6 - \smoothraggedrightrag-
width) / 6
1820      + \glueexpr \z@ \@plus \typog@fuzzwidth \@mi-
nus \typog@fuzzwidth\relax}% (6)
1821      {\glueexpr \typog@@linewidth - \smoothraggedrightragwidth +
1822      + \glueexpr \z@ \@plus \typog@fuzzwidth\relax}% (1)
1823      {\glueexpr (\typog@@linewidth * 3 - \smoothraggedrightrag-
width) / 3
1824      + \glueexpr \z@ \@plus \typog@fuzzwidth \@mi-
nus \typog@fuzzwidth\relax}% (5)
1825      {\glueexpr (\typog@@linewidth * 6 - \smoothraggedrightrag-
width * 5) / 6
1826      + \glueexpr \z@ \@plus \typog@fuzzwidth \@mi-
nus \typog@fuzzwidth\relax}% (2)
1827      {\glueexpr \typog@@linewidth \@minus \typog@fuzzwidth\relax}% (7)
1828      {\glueexpr (\typog@@linewidth * 2 - \smoothraggedrightrag-
width) / 2
1829      + \glueexpr \z@ \@plus \typog@fuzzwidth \@mi-
nus \typog@fuzzwidth\relax}% (4)

```

```

1830 \else
1831 \PackageError{typog}
1832 {smoothraggedright: unknown generator name (\smooth-
raggedrightgenerator)}
1833 {valid generator names are triplet, quintuplet, and sep-
tuplet}
1834 \fi}
1835 {\ifcase\typog@@generatorchoice
1836 \endsmoothraggedrightshapetriplet
1837 \or
1838 \endsmoothraggedrightshapequintuplet
1839 \or
1840 \endsmoothraggedrightshapeseptuplet
1841 \fi}
1842

```

`smoothraggedright (env.)`

```

1843 \NewDocumentEnvironment{smoothraggedright}{0{}}
1844 {\PushPostHook{par}{\hskip-\parindent\smoothraggedrightpar[#1]\relax}}
1845 {\par\PopPostHook{par}}
1846

```


Change History

v0.1

General: Initial version. [i](#)

v0.2

`\narrowospace`: New macro. [105](#)

`\widespace`: Add fallback if `\fontdimen7` is zero. Extend with a starred version. [104](#)

v0.3

`hyphenmin`: New environment. [81](#)

`\resetbaselineskip`: New macro. [118](#)

`\setbaselineskip`: New macro. [118](#)

`\setbaselineskippercentage`: New macro. [119](#)

`\setleading`: New macro. [119](#)

`\setleadingpercentage`: New macro. [120](#)

`\typogfontsize`: New dimen. [119](#)

v0.4

`\lowercaseadjustlabelitems`: New macro. [97](#)

`\noadjustlabelitems`: New macro. [97](#)

`\typogadjuststairs`: New macro. [101](#)

`\typoggetnth`: New macro. [78](#)

`\typoglowercaseadjustcheck`: New macro. [102](#)

`\typoguppercaseadjustcheck`: New macro. [102](#)

`\uppercaseadjustlabelitems`: New macro. [96](#)

v0.4a

`smoothraggedrightshapeseptuplet`: Add missing backslash to `\typog@@septuplet@parindent` in local macro `\typog@@septuplet@linespecs` which rendered the septuplet generator unusable. [122](#)

`\typog@register@pdfsubstitute`: Using `\pdfstringdefDisableCommands` breaks package `pdfcomment`. So we just refrain from using the macro if `pdfcomment` was loaded. [74](#)

v0.5

General: **[BACKWARD INCOMPATIBLE CHANGE]** Rename package option `math-italicscorrection` to `mathitaliccorrection`. [77](#)

[BACKWARD INCOMPATIBLE CHANGE] Rename package option `textitalicscorrection` to `textitaliccorrection`. [77](#)

Rewrite parser for package option `lowercaselabelitemadjustments` to accept a star as a placeholder. [77](#)

Rewrite parser for package option `uppercaselabelitemadjustments` to accept a star as a placeholder. [77](#)

`\Adjustedlabelitemi`: New macro. [99](#)

`\adjustedlabelitemi`: New macro. [99](#)

`\Adjustedlabelitemii`: New macro. [99](#)

`\adjustedlabelitemii`: New macro. [99](#)

`\Adjustedlabelitemiii`: New macro. [99](#)

`\adjustedlabelitemiii`: New macro. [99](#)

`\Adjustedlabelitemiv`: New macro. [99](#)

`\adjustedlabelitemiv`: New macro. [100](#)
`\capitalinvertedexclamationmark`: New macro. [94](#)
`\capitalinvertedquestionmark`: New macro. [94](#)
`\kernedslash`: Allow for lowering (or raising) `\kernedslash` with package option `lowerslash`. [83](#)
`lastlinefitpar`: New environment. [104](#)
`\leftspacedcapitalemdash`: New macro. [90](#)
`\leftspacedcapitalendash`: New macro. [89](#)
`\leftspacedendash`: New macro. [85, 86](#)
`\rightspacedcapitalemdash`: New macro. [90](#)
`\rightspacedcapitalendash`: New macro. [89](#)
`\rightspacedemdash`: New macro. [87](#)
`\rightspacedendash`: New macro. [85](#)
`\swapendashskip`: New macro. [86](#)
`\typoglogo`: Make macro independent of typog setup. [73](#)

v0.6

General: New package option `pdfsubstitutes`. [77](#)
`\capitalcolon`: New macro. [91](#)
`\iftypog@setup@pdfsubstitutes`: New conditional to track the state of PDF-substitution setup. [73](#)
`\textcolon`: New macro. [91](#)
`\typog@register@pdfsubstitute`: Make the substitution configurable via a package option. [74](#)

References

- [1] ABRAHAM, PAUL W., KATHRYN A. HARGREAVES, and KARL BERRY. *T_EX for the Impatient*. 2020, <https://tug.ctan.org/info/impatient/book.pdf>.
- [2] AMERICAN MATHEMATICAL SOCIETY and the L^AT_EX3 PROJECT TEAM. *Package amsmath*. 2020, <https://ctan.org/pkg/amsmath>.
- [3] ARSENEAU, DONALD. *Package cite*. 2015, <https://ctan.org/pkg/cite>.
- [4] *The Chicago Manual of Style*, 17th ed. The University of Chicago Press, Chicago/IL, 2017.
- [5] BEZOS, JAVIER. *Package enumitem*. 2019, <https://ctan.org/pkg/enumitem>.
- [6] BEZOS, JAVIER. *Package babel*. 2021, <https://ctan.org/pkg/babel>. The original author of package babel was J. L. BRAAMS.
- [7] BREITENLOHNER, PETER and the N^TS TEAM. *e-T_EX*. 1998, https://mirrors.ctan.org/systems/doc/etex/etex_man.pdf.
- [8] BURNOL, JEAN-FRANÇOIS. *Package mathastext*. 2023, <https://ctan.org/pkg/mathastext>.
- [9] CARLISLE, DAVID. *Russian Paragraph Shapes*. Baskerville, 6(1), 13–15, 1996, <http://uk-tug-archive.tug.org/wp-installed-content/uploads/2008/12/61.pdf>.
- [10] CARLISLE, DAVID. *What do different \fontdimen<num> mean*. 2013–1–2, <https://tex.stackexchange.com/questions/88991/what-do-different-fontdimennum-mean>.
- [11] CUBITT, TOBY. *Package cleveref*. 2018, <https://ctan.org/pkg/cleveref>.
- [12] EIJKHOUT, VICTOR. *T_EX By Topic, A Texnician's Reference*. 2007, <https://www.eijkhout.net/tex/tex-by-topic.html>.
- [13] FORSSMAN, FRIEDRICH, and RALF DE JONG. *Detailtypographie*. 4. Aufl. Verlag Hermann Schmitz, Mainz, 2008.
- [14] HØGHOLM, MORTEN, LARS MADSEN, and the L^AT_EX3 PROJECT TEAM. *Package mathtools*. 2020, <https://ctan.org/pkg/mathtools>.
- [15] KHIREVICH, SIARHEI. *Tips on Writing a Thesis in L^AT_EX*. 2013, <https://www.khirevich.com/latex/microtype>.
- [16] KLEBER, JOSEF. *Package pdfcomment*. 2018, <https://ctan.org/pkg/pdfcomment>.
- [17] KNUTH, DONALD ERVIN. *The T_EXbook*. Addison Wesley, Reading/MA, 1986.

- [18] KOHM, MARKUS. *Package koma-script*. 2025, <https://ctan.org/pkg/koma-script>.
- [19] THE L^AT_EX PROJECT. *Package l3experimental*. 2024, <https://ctan.org/pkg/l3experimental>.
- [20] MCPHERSON, KENT. *Package layout*. 2014, <https://ctan.org/pkg/layout>. The package was converted to L^AT_EX 2_ε by J. L. BRAAMS and modified by H. UMEKI.
- [21] MIDDENDORP, JAN. *Shaping Text*. BIS publishers, Amsterdam, 2014.
- [22] MITTELBACH, FRANK. *Managing forlorn paragraph lines (a. k. a. widows and orphans) in L^AT_EX*. TUGboat, 39(3), 246–251, 2018, <https://tug.org/TUGboat/tb39-3/tb123mitt-widows.pdf>.
- [23] MITTELBACH, FRANK. *Package widows-and-orphans*. 2020, <https://ctan.org/pkg/widows-and-orphans>.
- [24] NIEDERBERGER, CLEMENS. *Package tasks*. 2022, <https://github.com/cgnieder/tasks>.
- [25] RAHTZ, SEBASTIAN, and FRANK MITTELBACH. *Package hyperref*. 2020, <https://ctan.org/pkg/hyperref>. The package is maintained by the L^AT_EX3 Project Team.
- [26] SCHLICHT, ROBERT. *Package microtype*. 2020, <https://ctan.org/pkg/microtype>.
- [27] SCHRÖDER, MARTIN. *Package ragged2e*. 2019, <https://ctan.org/pkg/ragged2e>.
- [28] SOLOMON, DAVID. *Output Routines: Examples and Techniques. Part I: Introduction and Examples*. TUGboat, 11(1), 69–85, 1990, <https://www.tug.org/TUGboat/Articles/tb11-1/tb27salomon.pdf>.
- [29] STRIZVER, ILENE. *Type rules!: the designer’s guide to professional typography*, 4th ed. John Wiley & Sons, Hoboken/NJ, 2014.
- [30] TOBIN, GEOFFREY, and ROBIN FAIRBAIRNS. *Package setspace*. 2011, <https://ctan.org/pkg/setspace>.
- [31] UMEKI, HIDEO. *Package geometry*. 2020, <https://ctan.org/pkg/geometry>.
- [32] WERMUTH, UDO. *Tracing paragraphs*. TUGboat, 37(3), 358–373, 2016, <https://tug.org/TUGboat/tb37-3/tb117wermuth.pdf>.
- [33] WERMUTH, UDO. *The optimal value for \emergencystretch*. TUGboat, 38(1), 65–86, 2017, <https://tug.org/TUGboat/tb38-1/tb118wermuth.pdf>.
- [34] WERMUTH, UDO. *A note on \linepenalty*. TUGboat, 38(3), 400–414, 2017, <https://tug.org/TUGboat/tb38-3/tb120wermuth.pdf>.

- [35] WERMUTH, UDO. *Experiments with `\parfillskip`*. TUGboat, 39(3), 276–303, 2018, <https://tug.org/TUGboat/tb39-3/tb123werimuth-parfillskip.pdf>.
- [36] WERMUTH, UDO. *An attempt at ragged-right typesetting*. TUGboat, 41(1), 73–94, 2020, <https://tug.org/TUGboat/tb41-1/tb127werimuth-ragged.pdf>.
- [37] WERMUTH, UDO. Personal communication. August 2, 2022.
- [38] WERMUTH, UDO. *Vertical alignments in plain $T_{\text{E}}\text{X}$* . TUGboat, 44(3), 427–440, 2023, <https://tug.org/TUGboat/tb44-3/tb138werimuth-valign.pdf>.
- [39] WILSON, PETER. *Package hyphenat*. 2004, <https://ctan.org/pkg/hyphenat>. The package is maintained by W. ROBERTSON.
- [40] WILSON, PETER. *Glisterings*. TUGboat, 28(2), 229–232, 2007, <https://tug.org/TUGboat/tb28-2/tb89glistier.pdf>.
- [41] WILSON, PETER. *Package needspace*. 2010, <https://ctan.org/pkg/needspace>. The package is maintained by W. ROBERTSON.

Index

Es ist schon alles gesagt,
nur noch nicht von allen.
— KARL VALENTIN

Numbers written in *italic style* refer to definitions; numbers in *regular style* refer to pages where an entry is used. References to code lines are prefixed with ‘*ℓ*’. Again, *italic style* refers to definitions and *regular style* to lines where the code is used.

Symbols

\: *ℓ*580

A

\Adjustedlabelitemi *32, ℓ*826, *ℓ*852
\adjustedlabelitemi *32, ℓ*829, *ℓ*853
\Adjustedlabelitemii *32, ℓ*832, *ℓ*854
\adjustedlabelitemii *32, ℓ*835, *ℓ*855
\Adjustedlabelitemiii *32, ℓ*838, *ℓ*856
\adjustedlabelitemiii *32, ℓ*841, *ℓ*857
\Adjustedlabelitemiv *32, ℓ*844, *ℓ*858
\adjustedlabelitemiv *32, ℓ*847, *ℓ*859
\allowhyphenation *11, ℓ*209
amsmath (package) *54*

B

baseline skip *55*
bookmark *16, 21*
breakabledisplay (env.) *54, ℓ*1478
breakpenalty (option) *2, 16, 18, 24–26*
\breakpoint *14, ℓ*213
\breakpoint* *14*

C

\capitalcolon *27, ℓ*581
\capitaldash *25, ℓ*482, *ℓ*490, *ℓ*578
\capitaldash* *25*
\capitalemdash *25, ℓ*493, *ℓ*545, *ℓ*546, *ℓ*559, *ℓ*560
\capitalemdash* *25*
\capitalendash *25, ℓ*477, *ℓ*509, *ℓ*510, *ℓ*525, *ℓ*526
\capitalendash* *25*
\capitalhyphen *24, ℓ*463

\capitalhyphen* *24*
\capitalinvertedexclamationmark *29, ℓ*673
\capitalinvertedquestionmark *29, ℓ*681
\capitaltimes *27, ℓ*584
code reference *73–126*
configuration *6*
covernextindentpar (env.) *40, ℓ*968
csquotes (package) *29*

D

debug (option) *2*
dimensions:
\smoothraggedrightleftskip *60, ℓ*1751, *ℓ*1780
\smoothraggedrightparindent *60, ℓ*1752, *ℓ*1789, *ℓ*1800, *ℓ*1815
\smoothraggedrightragwidth *61, ℓ*1753, *ℓ*1784, *ℓ*1786, *ℓ*1791, *ℓ*1794, *ℓ*1797, *ℓ*1802, *ℓ*1805, *ℓ*1807, *ℓ*1809, *ℓ*1812, *ℓ*1817, *ℓ*1819, *ℓ*1821, *ℓ*1823, *ℓ*1825, *ℓ*1828
\typogfontsize *56, ℓ*1564, *ℓ*1575, *ℓ*1578, *ℓ*1583, *ℓ*1584, *ℓ*1594, *ℓ*1596, *ℓ*1601, *ℓ*1602, *ℓ*1612, *ℓ*1615, *ℓ*1620, *ℓ*1621
\Doubleguillemetleft *28, ℓ*633
\doublinguilletleft *27, ℓ*607
\Doubleguillemetright *28, ℓ*640
\doublinguilletright *27, ℓ*613

E

emdashspace (option) *2*
endashspace (option) *2*
enumitem-keys:
vtietop *51*
environments:
breakabledisplay *54, ℓ*1478
covernextindentpar *40, ℓ*968

hyphenmin [14](#), [ℓ223](#)
 lastlinecenteredpar [36](#), [ℓ944](#)
 lastlinefitpar [42](#), [ℓ998](#)
 lastlineflushrightpar [34](#),
 [ℓ941](#)
 lastlineraggedleftpar [34](#),
 [ℓ935](#)
 loosespacing [43](#), [ℓ1034](#)
 nocharprotrusion [48](#), [ℓ1203](#)
 nofontexpand [48](#), [ℓ1200](#)
 nofontexpansion [48](#), [ℓ1194](#)
 openlastlinepar [40](#), [ℓ982](#)
 prolongpar [40](#), [ℓ958](#)
 setfontexpand [47](#), [ℓ1180](#)
 setfontshrink [47](#), [ℓ1152](#)
 setfontstretch [47](#), [ℓ1166](#)
 setfonttracking [46](#), [ℓ1066](#)
 shortenpar [40](#), [ℓ951](#)
 slightlysloppypar [48](#), [ℓ1274](#)
 smoothraggedright [60](#), [ℓ1843](#)
 smoothraggedrightpar [58](#), [ℓ1758](#)
 smoothraggedrightshape-
 quintuplet [58](#), [ℓ1670](#)
 smoothraggedrightshape-
 septuplet [58](#), [ℓ1708](#)
 smoothraggedrightshape-
 triplet [58](#), [ℓ1637](#)
 tightspacing [43](#), [ℓ1050](#)
 typoginspect [8](#), [ℓ170](#)
 typoginspectpar [8](#), [ℓ197](#)
 typogsetup [6](#), [ℓ106](#)
 vtiebotdisp [51](#), [ℓ1463](#)
 vtiebotdisptoppar [52](#), [ℓ1467](#)
 vtiebotpar [51](#), [ℓ1405](#)
 vtietoppar [50](#), [ℓ1338](#)
 extra spacing [18](#)

F

\figuredash [26](#), [ℓ573](#)
 \figuredash* [26](#)
 font
 encoding [27](#)
 expansion [39](#), [47](#)
 information [7](#)
 protrusion [48](#)
 size [7](#)
 spacing [39](#), [42](#)
 loose [42](#)
 tight [42](#)
 tracking [39](#), [46](#)
 typeface
 ADF Baskervald [28](#)

ADF Accanthis [35](#)
 ADF Venturis [26](#), [35](#)
 Alegreya [26](#)
 Aleo [35](#)
 Arvo [26](#)
 Bitter [26](#)
 Charis SIL [26](#), [28](#), [35](#)
 Clara [26](#)
 CM Roman [35](#)
 Cochineal [19](#), [35](#)
 Coelacanth [26](#)
 Crimson Pro [26](#)
 Crimson Text [26](#)
 Domitian [35](#)
 Droid Serif [26](#)
 EB Garamond [26](#), [28](#), [35](#)
 Erewhon [26](#)
 etbb [35](#)
 Extended Charter [35](#)
 fbb [26](#)
 Gentium [26](#), [28](#)
 Gentium Plus [35](#)
 GFS Artemisia [26](#), [28](#)
 GFS Bodoni [35](#)
 GFS Didot [28](#), [35](#)
 Ibarra Real Nova [26](#)
 IBM Plex Serif [35](#)
 INRIA Serif [26](#)
 KP Serif [35](#)
 Libertine [26](#)
 Libertinus Serif [26](#), [28](#), [35](#)
 Libre Baskerville [26](#)
 Libre Caslon [26](#)
 Merriweather [26](#), [28](#)
 ML Modern [35](#)
 PT Serif [26](#)
 Quattrocento [26](#)
 Roboto Slab [26](#)
 Source Serif Pro [26](#), [35](#)
 Spectral [26](#), [35](#)
 STIX [26](#), [35](#)
 T_EX Gyre Pagella [26](#)
 TX Fonts [26](#)
 URW Palladio [ii](#), [30](#), [35](#)
 Utopia [35](#)
 \fontsizeinfo [7](#), [ℓ149](#)
 forlorn line
 club [40](#), [50](#)
 display widow [51](#)
 orphan [40](#)
 widow [40](#), [51](#)

H

hyperref (package) 16, 21
 hyphenation 11
 empty discretionary 14
 first word 14
 \hskip 14
 re-enable automatic 11
 hyphenmin (env.) 14, ℓ223

I

information 7
 inverted
 exclamation mark 29
 question mark 29
 \itcorr 17, 30, ℓ265
 \itcorr* 17

K

\kernedhyphen 19, ℓ290, ℓ312, ℓ313,
 ℓ320, ℓ321
 \kernedhyphen* 19
 \kernedslash 18, ℓ147, ℓ278
 \kernedslash* 18
 kerning
 extra 18
 forward slash 18
 hyphen 19
 ligature 16
 known problems 62

L

label items 31
 lastlinecenteredpar (env.) 36,
 ℓ944
 \lastlinefit 10, 42, 71
 lastlinefitpar (env.) 42, 71, ℓ998
 lastlineflushrightpar (env.) 34,
 ℓ941
 lastlinetraggedleftpar (env.) 34,
 ℓ935
 leading 55
 \leftkernedhyphen 19, ℓ310, ℓ315
 \leftkernedhyphen* 19
 \leftspacedcapitaldash 25, ℓ512,
 ℓ520
 \leftspacedcapitaldash* 25
 \leftspacedcapitalemdash 25,
 ℓ543
 \leftspacedcapitalemdash* 25
 \leftspacedcapitalendash 25,
 ℓ507
 \leftspacedcapitalendash* 25

\leftspaceddash 21, ℓ348, ℓ360
 \leftspaceddash* 21
 \leftspacedemdash 22, ℓ402, ℓ418
 \leftspacedemdash* 22
 \leftspacedendash 21, ℓ335, ℓ402
 \leftspacedendash* 21
 ligature 16
 ligaturekern (option) 2, 16
 limitations 62
 line spacing 7
 list 50
 loosespacing (env.) 43, ℓ1034
 lowercaselabelitem-
 adjustments (option) 2, 31
 \lowercaseadjustlabelitems 31,
 ℓ780
 lowerslash (option) 3, 18

M

mathitaliccorrection (option) 3,
 17
 microtype (package) 46

N

narrow space 45
 \narrow space 45, ℓ1020
 \narrow space* 45
 \narrow spacescale 45, ℓ1019, ℓ1022,
 ℓ1024, ℓ1030, ℓ1031
 \narrow space strength 45, ℓ1018,
 ℓ1027
 \noadjustlabelitems 31, ℓ787
 nocharprotrusion (env.) 48, ℓ1203
 nodebug (option) 2
 nofontexpand (env.) 48, ℓ1200
 nofontexpansion (env.) 48, ℓ1194
 \nolig 16, ℓ230
 \nolig* 16
 nopdfsubstitutes (option) 3

O

openlastlinepar (env.) 40, ℓ982

P

package code 73–126
 package option 2–5
 breakpenalty 2, 16, 18, 24–26
 debug 2
 emdashspace 2
 endashspace 2
 ligaturekern 2, 16
 lowercaselabelitem-
 adjustments 2, 31
 lowerslash 3, 18
 mathitaliccorrection 3, 17
 nodebug 2
 nopdfsubstitutes 3
 pdfsubstitutes 3
 raise* 3
 raisecapitaldash 3, 25
 raisecapitalguillemets 3, 28
 raisecapitalhyphen 3, 19
 raisecapitaltimes 3, 27
 raisefiguredash 4, 26, 27
 raiseguillemets 4, 28
 raiseinvertedmarks 4, 29
 shrinklimits 4, 47
 slashkern 4, 18
 stretchlimits 4, 47
 textitaliccorrection 4, 17
 trackingttspacing 4, 46
 uppercaselabelitem-
 adjustments 5, 31
 page break 11, 54
 paragraph
 align last line 34
 centered 36
 flush right 34
 badness 10
 fill last line 37
 covernextindentpar 40
 \linebreak 39
 \mbox 38
 openlastlinepar 40
 prolongpar 40
 shortenpar 40
 tie 38
 sloppy 48
 vertically tied 50
 PDF 16, 21
 pdfsubstitutes (option) 3
 prolongpar (env.) 40, ℓ958
 \ProvideDocumentCommand ℓ580

Q

quick reference v–x

R

ragged right 58
 raise* (option) 3
 raisecapitaldash (option) 3, 25
 raisecapitalguillemets (option) 3, 28
 raisecapitalhyphen (option) 3, 19
 raisecapitaltimes (option) 3, 27
 raised character 24
 colon 27
 em-dash 25
 en-dash 25
 guillemets 27
 hyphen 24
 inverted exclamation mark 29
 inverted question mark 29
 multiplication sign 27
 number dash 26
 raisefiguredash (option) 4, 26, 27
 raiseguillemets (option) 4, 28
 raiseinvertedmarks (option) 4, 29
 reconfigure 6
 \resetbaselineskip 55, ℓ1559
 \rightkernedhyphen 19, ℓ318, ℓ323
 \rightkernedhyphen* 19
 \rightspacedcapitaldash 25, ℓ528, ℓ538
 \rightspacedcapitaldash* 25
 \rightspacedcapitalemdash 25, ℓ557
 \rightspacedcapitalemdash* 25
 \rightspacedcapitalendash 25, ℓ523
 \rightspacedcapitalendash* 25
 \rightspacedddash 21, ℓ376, ℓ391
 \rightspacedddash* 21
 \rightspacedemdash 22, ℓ430
 \rightspacedemdash* 22
 \rightspacedendash 21, ℓ363
 \rightspacedendash* 21

S

`\setbaselineskip` 55, [l1491](#), [l1493](#),
[l1500](#), [l1509](#), [l1519](#), [l1537](#), [l1546](#),
[l1549](#), [l1553](#)
`\setbaselineskippercentage` 56,
[l1572](#)
`setfontexpand` (env.) 47, [l1180](#)
`setfontshrink` (env.) 47, [l1152](#)
`setfontstretch` (env.) 47, [l1166](#)
`setfonttracking` (env.) 46, [l1066](#)
`\setleading` 56, [l1591](#)
`\setleadingpercentage` 56, [l1609](#)
`setup` 6
`shortenpar` (env.) 40, [l951](#)
`shrinklimits` (option) 4, 47
`\Singleguillemetleft` 28, [l619](#)
`\singleguillemetleft` 27, [l595](#)
`\Singleguillemetright` 28, [l626](#)
`\singleguillemetright` 27, [l601](#)
`slashkern` (option) 4, 18
`\slightlyloppy` 48, [l1217](#), [l1275](#)
`\slightlyloppypar` (env.) 48, [l1274](#)
`smoothraggedright` (env.) 60, [l1843](#)
`\smoothraggedrightfuzzfactor` 60,
[l1749](#), [l1784](#)
`\smoothraggedrightgenerator` 60,
[l1750](#), [l1779](#), [l1832](#)
`\smoothraggedrightleftskip`
(skip) 60, [l1751](#), [l1780](#)
`smoothraggedrightpar` (env.) 58,
[l1758](#)
`\smoothraggedrightparindent`
(skip) 60, [l1752](#), [l1789](#), [l1800](#),
[l1815](#)
`\smoothraggedrightragwidth`
(skip) 61, [l1753](#), [l1784](#), [l1786](#),
[l1791](#), [l1794](#), [l1797](#), [l1802](#), [l1805](#),
[l1807](#), [l1809](#), [l1812](#), [l1817](#), [l1819](#),
[l1821](#), [l1823](#), [l1825](#), [l1828](#)
`smoothraggedrightshape-`
`quintuplet` (env.) 58, [l1670](#)
`smoothraggedrightshape-`
`septuplet` (env.) 58, [l1708](#)
`smoothraggedrightshapetriplet`
(env.) 58, [l1637](#)
`spaced character`
em-dash 22
en-dash 21
`\spacedcapitaldash` [l530](#), [l540](#)
`\spacedcapitalemdash` 25, [l562](#),
[l570](#)
`\spacedcapitalemdash*` 25

`\spacedcapitalendash` [l529](#), [l539](#)
`\spacedddash` 21, [l378](#), [l393](#)
`\spacedddash*` 21
`\spacedemdash` 22, [l445](#), [l458](#)
`\spacedemdash*` 22
`\spacedendash` 21, [l377](#), [l392](#)
`\spacedendash*` 21
`\splicevtietop` 50, [l1343](#)
`stretchlimits` (option) 4, 47
`\swapendashskip` 22, [l396](#)

T

`\textcolon` [l580](#), [l582](#)
`\textemdash` 23, 26
`\textendash` 23, 26
`\texttexclamdown` 30
`textitaliccorrection` (option) 4,
[l17](#)
`\textquestiondown` 30
`tightspacing` (env.) 43, [l1050](#)
`trackingttspace` (option) 4, 46
`\typogadjuststairs` 33, [l885](#)
`\typogadjuststairsfor` [l863](#)
`\typogfontsize` (dim.) 56, [l1564](#),
[l1575](#), [l1578](#), [l1583](#), [l1584](#), [l1594](#),
[l1596](#), [l1601](#), [l1602](#), [l1612](#), [l1615](#),
[l1620](#), [l1621](#)
`\typogget` 6, [l115](#)
`\typoggetnth` 7, [l118](#)
`typoginspect` (env.) 8, [l170](#)
`typoginspectpar` (env.) 8, [l197](#)
`\typoglogo` [l11](#)
`\typoglowlowercaseadjustcheck` 33,
[l928](#)
`typogsetup` (env.) 6, [l106](#)
`\typoguppercaseadjustcheck` 33,
[l916](#)

U

`uppercaselabelitem-`
`adjustments` (option) 5, 31
`\uppercaseadjustlabelitems` 31,
[l773](#)

V

`\vtiebot` 51, [l1352](#), [l1406](#)
`\vtiebotdisp` [l1473](#)
`vtiebotdisp` (env.) 51, [l1463](#)
`vtiebotdisptoppar` (env.) 52, [l1467](#)
`vtiebotpar` (env.) 51, [l1405](#)
`\vtietop` 50, [l1285](#), [l1339](#), [l1345](#), [l1471](#)
`vtietop` (enumitem-key) 51

vtietoppar (env.) [50](#), [ℓ1338](#)

W

wide space [44](#)

\widespace [44](#), [ℓ1004](#)

\widespace* [44](#)

\widespacescale [44](#), [ℓ1003](#), [ℓ1006](#),
[ℓ1008](#), [ℓ1014](#), [ℓ1015](#)

\widespacestrength [44](#), [ℓ1002](#), [ℓ1011](#)